Database

Mattia Masina
CdL in Informatica – Università di Bologna

Anno Accademico 2026/2027

Indice

1 Basi di Dati e Sistemi di Gestione dei Dati (DBMS) 4
- 1.1 Sistemi Informativi, Dati e Informazione . . . 4
  - 1.1.1 Metodologia di Studio e Sistemi Informativi . . . 4
  - 1.1.2 Dati, Informazione e Codifica . . . 6
- 1.2 DBMS: Modelli, Architettura e Linguaggi . . . 7
  - 1.2.1 Basi di Dati, DBMS e Confronto con i File System . . . 7
  - 1.2.2 Modelli dei Dati, Architettura e Indipendenza . . . 11
  - 1.2.3 Linguaggi per Basi di Dati e Modalità di Utilizzo . . . 13
  - 1.2.4 Attori, Ruoli, Transazioni e Valutazione dei DBMS . . . 15

2 Il Modello Relazionale dei Dati 17
- 2.1 Struttura del Modello Relazionale . . . 17
  - 2.1.1 Origine, Modelli Logici e Paradigma Basato su Valori . . . 17
  - 2.1.2 Dalla Relazione Matematica alla Relazione Tabellare . . . 18
  - 2.1.3 Formalizzazione: Schemi, Istanze e Tuple . . . 20
  - 2.1.4 Strutture Dati Annidate e Appiattimento (*Unnesting*) . . . 22
  - 2.1.5 Informazione Parziale e Gestione del Valore NULL . . . 23
- 2.2 Vincoli di Integrità . . . 25
  - 2.2.1 Vincoli di Integrità e Loro Classificazione . . . 25
  - 2.2.2 Vincoli Intra-relazionali: Dominio e Tupla . . . 27
  - 2.2.3 Identificazione delle Tuple: Superchiavi e Chiavi . . . 28
  - 2.2.4 Chiavi, Valori NULL e la Chiave Primaria . . . 29
  - 2.2.5 Integrità Referenziale e Chiavi Esterne (*Foreign Key*) . . . 31
  - 2.2.6 Politiche di Reazione alle Violazioni e Vincoli Multipli . . . 32

3 Linguaggi Formali di Interrogazione 34
- 3.1 Algebra Relazionale . . . 34
  - 3.1.1 Linguaggi per Interrogare una Base di Dati . . . 34
  - 3.1.2 Operatori Insiemistici e Ridenominazione . . . 34
  - 3.1.3 Selezione e Proiezione . . . 35
  - 3.1.4 Join . . . 37
  - 3.1.5 Esempi di Interrogazioni . . . 39
  - 3.1.6 Equivalenza di Espressioni . . . 42
  - 3.1.7 Selezione e Valori NULL . . . 43
  - 3.1.8 Viste . . . 43
  - 3.1.9 Una Notazione Alternativa per i Join . . . 44
- 3.2 Calcolo Relazionale e Datalog . . . 45
  - 3.2.1 Calcolo Relazionale su Domini . . . 45
  - 3.2.2 Calcolo su Tuple con Dichiarazioni di Range . . . 46
  - 3.2.3 Limiti di Algebra e Calcolo: la Chiusura Transitiva . . . 48
  - 3.2.4 Datalog . . . 48
- 3.3 Esercizi di algebra relazionale . . . 49
  - 3.3.1 Esercizi visti in classe . . . 49
  - 3.3.2 Pattern utili da imparare . . . 58

4 SQL di base **62**
4.1 Definizione dei Dati (DDL) . . . 62
4.1.1 Origine, Standardizzazione ed Evoluzione del Linguaggio . . . 62
4.1.2 Database, Schemi, Tabelle e Vincoli . . . 63
4.1.3 Integrità Referenziale ed Evoluzione dello Schema . . . 66
4.2 Interrogazioni in SQL . . . 68
4.2.1 Il Comando SELECT: Struttura, Proiezione e Selezione . . . 68
4.2.2 Interrogazioni su Più Relazioni e Join . . . 72
4.2.3 Ordinamento e Operatori Insiemistici . . . 75
4.2.4 Interrogazioni Annidate e Visibilità delle Variabili . . . 77
4.2.5 Aggregazione e Raggruppamento . . . 80
4.3 Operazioni di Modifica dei Dati (DML): INSERT, DELETE e UPDATE . . . 84

5 SQL avanzato **86**
5.1 Vincoli Complessi e Viste . . . 86
5.1.1 Vincoli CHECK e Asserzioni . . . 86
5.1.2 Viste: Interrogazione, Aggiornamento e Annidamento . . . 87
5.2 Query Complesse, Ricorsione e Funzioni Scalari . . . 89
5.2.1 Query Complesse e Scomposizione con Viste Intermedie . . . 89
5.2.2 Query Ricorsive e Chiusura Transitiva (SQL:1999) . . . 91
5.2.3 Funzioni Scalari Predefinite e Manipolazione Dati . . . 93
5.3 Espressioni Condizionali e Sicurezza dei Dati . . . 94
5.3.1 Espressioni Condizionali: COALESCE, NULLIF e CASE . . . 94
5.3.2 Sicurezza, Privilegi e Ruoli . . . 96
5.4 Gestione delle Transazioni . . . 99
5.4.1 Transazioni e Proprietà ACID . . . 99
5.4.2 Comandi Transazionali ed Esempio Completo . . . 101
5.5 Concorrenza, affidabilità e indici . . . 102
5.5.1 Controllo di Concorrenza: Anomalie, Schedule e Serializzabilità . . . 102
5.5.2 Lock, Locking a Due Fasi e Livelli di Isolamento . . . 103
5.5.3 Affidabilità: Log, Ripresa dai Guasti e ARIES . . . 104
5.5.4 Strutture Fisiche di Accesso e Indici . . . 105

1 Basi di Dati e Sistemi di Gestione dei Dati (DBMS)

1.1 Sistemi Informativi, Dati e Informazione

1.1.1 Metodologia di Studio e Sistemi Informativi

Studio Individuale e Concetti Fondamentali. Per imparare a gestire le basi di dati non basta mandare a memoria definizioni astratte. È fondamentale sviluppare una forma mentale rigorosa: capire come rappresentare la realtà in modo formale e collegare sempre la teoria a problemi pratici e casi d'uso concreti.

Attività Applicativa e Strumenti Software. La teoria deve andare di pari passo con la pratica. Esercizi, laboratori e progetti d'esame richiedono l'uso operativo di sistemi software dedicati, detti **DBMS** (*Database Management System*). Nel mondo reale e sul mercato esistono diverse categorie di DBMS:

- **Sistemi Enterprise e Commerciali**: IBM DB2, Oracle Database, Microsoft SQL Server (pensati per grandi aziende con carichi elevati).
- **Sistemi Open Source Relazionali Standard**: PostgreSQL, MySQL (molto diffusi nel web e nei progetti moderni).
- **Sistemi Desktop ed Embedded**: Microsoft Access, SQLite (leggeri, spesso integrati dentro singole applicazioni o app per smartphone).
- **Piattaforme Cloud e Analitiche**: Google BigQuery (ottimizzate per analizzare moli immense di dati).

Punto di Vista Metodologico vs Tecnologico. La materia viene affrontata da due angolazioni complementari:

- **Punto di vista metodologico**: riguarda la *progettazione*. Ci si concentra sui modelli logici e concettuali usati per descrivere le informazioni del mondo reale in modo pulito, astraendo dai dettagli fisici del computer.
- **Punto di vista tecnologico**: riguarda il *software*. Esamina come è fatto internamente un DBMS: come scrive i dati sui dischi, come gestisce la memoria, come gestisce accessi simultanei e quali linguaggi offre.

I Quattro Contenuti Fondamentali della Disciplina

- **Modelli per l'organizzazione dei dati**: insiemi di regole e strutture matematiche per descrivere entità, relazioni e vincoli del mondo reale.
- **Linguaggi per l'utilizzo dei dati**: strumenti (come SQL) per inserire, leggere, aggiornare ed eliminare i record.
- **Sistemi per la gestione dei dati**: l'architettura dei DBMS, capace di garantire sicurezza, persistenza ed efficienza.

- **Metodologie di progettazione**: i passi guidati per passare dai requisiti parlati a uno schema di database corretto, privo di errori e facile da mantenere.

Il Sistema Informativo (SI). Il **sistema informativo** è l'insieme di persone, strumenti e procedure con cui un'organizzazione (azienda, università, ospedale) raccoglie, organizza e scambia le informazioni indispensabili per svolgere le proprie attività. Anche se un'azienda non ha un reparto chiamato formalmente "Ufficio Sistema Informativo", possiede comunque un sistema informativo che ne supporta il lavoro quotidiano.

Indipendenza dall'Automazione Informatica. Un sistema informativo **non deve essere per forza computerizzato**.
Esempio: Gli archivi notarili o gli uffici anagrafe comunali dell'Ottocento funzionavano benissimo usando faldoni cartacei, registri scritti a mano e fattorini; costituivano a tutti gli effetti un sistema informativo, pur in totale assenza di computer.

Il Ciclo di Gestione dell'Informazione. Qualsiasi informazione all'interno di un'organizzazione attraversa un ciclo di quattro passaggi:

- **Raccolta e Acquisizione**: reperire i dati (es. far compilare un modulo di iscrizione a uno studente).
- **Archiviazione e Conservazione**: salvare i dati in modo duraturo per poterli ritrovare in futuro (es. registrarli in un archivio o sul disco).
- **Elaborazione e Trasformazione**: calcolare nuovi risultati a partire dai dati grezzi (es. calcolare la media dei voti o il bilancio annuale).
- **Distribuzione e Comunicazione**: trasmettere i risultati a chi ne ha bisogno (es. inviare una ricevuta via email o mostrare una schermata all'operatore).

Il Sistema Informatico (IT System). Il **sistema informatico** è la parte tecnologica e automatizzata del sistema informativo. È composto dall'hardware (server, dischi, reti), dal software (DBMS, programmi applicativi) e dagli specialisti informatici che li gestiscono.

Gerarchia dei Sistemi Aziendali: Modello a Cerchi Concentrici

Dal contesto più ampio a quello più specifico, l'organizzazione si struttura in quattro livelli concentrici:

1. **Impresa (*Enterprise*)**: l'ente economico complessivo nel suo mercato (obiettivi di business, clienti, risorse).
2. **Organizzazione (*Organization*)**: l'insieme coordinato di persone, uffici, compiti e processi decisionali.
3. **Sistema Informativo (*Information System*)**: la parte dell'organizzazione dedicata specificamente alla gestione dei flussi informativi.

4. **Sistema Informatico (*IT System*)**: l'infrastruttura tecnologica digitale (computer e software) che automatizza il sistema informativo.

Relazione di inclusione: Enterprise $\supset$ Organization $\supset$ Information System $\supset$ IT System.

Tutti gli informatici vorrebbero che il sistema informativo e quello informatico fossero perfettamente allineati. Il problema è che in certi casi non conviene o non ha senso informatizzare alcune informazioni.

1.1.2 Dati, Informazione e Codifica

Distinzione Concettuale e Necessità di Interpretazione. Nella vita di tutti i giorni esprimiamo concetti in molti modi: parole parlate, testo scritto, disegni, numeri o gesti. Tali informazioni possono essere salvate nella nostra memoria biologica (il cervello), su carta oppure su supporti digitali. Nei calcolatori, però, tutta questa varietà deve essere ridotta e formalizzata sotto forma di **dati**.

I due termini hanno ruoli distinti:

- **Informazione**: la conoscenza o il significato che una persona ricava da una notizia, da un fatto o da un contesto.
- **Dato**: l'elemento grezzo, preciso e privo di ambiguità (es. una stringa o un numero), memorizzato come punto di partenza per il calcolo.

Il Dato è Inutile senza Interpretazione: L'Esempio dei Segnali Finlandesi

Si considerino tre cartelli stradali finlandesi di divieto di sosta con pannelli integrativi numerici:

Dato Grezzo	Significato Reale (Informazione Compresa)
8 – 17	Divieto attivo dalle 8:00 alle 17:00 nei giorni feriali (Lunedì–Venerdì).
(8 – 13)	Divieto attivo dalle 8:00 alle 13:00 nella giornata di Sabato (indicato tra parentesi).
8 – 14	Divieto attivo dalle 8:00 alle 14:00 nei giorni festivi e Domenica (indicato in rosso).

Analisi Semantica:
I numeri da soli sono solo cifre. Senza conoscere il codice stradale finlandese (il contesto), non sappiamo a cosa si riferiscano parentesi o colori.
$\Longrightarrow$ **Regola chiave**: Dato + Contesto / Chiave Interpretativa = Informazione.

Il Processo di Codifica dell'Informazione. Per consentire a un computer di elaborare la realtà, dobbiamo codificare le informazioni secondo regole fisse e ripetibili:

- Si prendono descrizioni del mondo reale ricche e discorsive (es. dati di un cittadino).
- Si scompone il testo in attributi precisi: *Nome*, *Cognome*, *Data di Nascita*.
- Si crea un identificatore standard sintetico e univoco: ad esempio il **Codice Fiscale**.

Perché i Dati Sono una Risorsa Strategica. All'interno di qualsiasi ente o azienda, quasi tutto cambia in fretta:

- I processi di business e le strategie commerciali si rinnovano continuamente.
- I computer, i server e i linguaggi di programmazione invecchiano e vengono rimpiazzati in pochi anni.
- Le persone cambiano ruolo o vanno in pensione.
- **I dati invece restano stabili nel tempo**: un contratto, uno storico di bonifici bancari o un registro delle nascite mantengono la propria validità e struttura invariate per decenni. I dati sono quindi il patrimonio più durevole dell'azienda.

1.2 DBMS: Modelli, Architettura e Linguaggi

1.2.1 Basi di Dati, DBMS e Confronto con i File System

Definizione di Base di Dati. L'espressione **base di dati** (*database*) viene usata con due significati:

- **Significato Generico**: qualsiasi archivio strutturato di dati usato per supportare le attività di un'organizzazione o di un individuo (anche una rubrica telefonica cartacea).
- **Significato Informatico**: una collezione organizzata di dati, salvata su memorie di massa persistenti (dischi SSD, hard disk) e gestita in modo centralizzato da un programma dedicato chiamato **DBMS**.

Il DBMS (DataBase Management System). Un **DBMS** è un sistema software concepito per gestire grandi collezioni di dati, garantendo che le operazioni siano sicure, affidabili ed efficienti.

Le Tre Proprietà Intrinseche dei Dati Gestiti da un DBMS

Un database gestito da un DBMS possiede tre proprietà distintive:

1. **Grandi Dimensioni (*Big*)**: i dati non possono risiedere per intero nella memoria RAM del computer; il sistema sfrutta la memoria secondaria (dischi fisici o storage cloud).
2. **Persistenza (*Persistent*)**: i dati sopravvivono alla chiusura delle applicazioni e allo spegnimento della macchina.
3. **Condivisione (*Shared*)**: la stessa base di dati è usata contemporaneamente da programmi diversi e da utenti diversi, evitando di dover duplicare gli archivi per ogni ufficio.

I Quattro Requisiti di Garanzia del DBMS. Un DBMS affidabile offre quattro garanzie fondamentali:

- **Riservatezza (*Privacy / Sicurezza*)**: ciascun utente vede e modifica solo i dati per cui ha un permesso esplicito (es. uno studente vede il proprio libretto, ma non quello degli altri).
- **Affidabilità (*Reliability*)**: in caso di guasto hardware o interruzione improvvisa della corrente elettrica, i dati memorizzati non si corrompono né vanno persi.
- **Efficienza (*Efficiency*)**: il sistema risponde velocemente alle richieste senza sprecare memoria o tempo di CPU, sfruttando algoritmi di indicizzazione mirati.
- **Efficacia (*Effectiveness*)**: mette a disposizione strumenti intuitivi, potenti e flessibili per estrarre e lavorare sui dati utili al lavoro quotidiano.

Disponibilità dei DBMS sul Mercato. I DBMS si possono installare sui propri server (*on-premise*) oppure noleggiare come servizi gestiti nel cloud (*Database as a Service*). Alcuni dei prodotti più usati sono Oracle, Microsoft SQL Server, PostgreSQL, MySQL, IBM DB2, Google BigQuery e SQLite.

Volumetrie e Ordini di Grandezza delle Basi di Dati. Quando diciamo che i database sono "grandi", intendiamo volumi di dati tali per cui caricare tutto il database in RAM sarebbe impossibile o insostenibile dal punto di vista dei costi.

Ordini di Grandezza nelle Basi di Dati

Tipologia di Base di Dati e Utilizzo	Dimensione Tipica
Database Operativi Quotidiani (OLTP - acquisti, pagamenti)	1 – 5 Terabyte
Database per Analisi e Report (OLAP / Data Warehouse)	30 – 50 Terabyte
Database Scientifici (meteo, fisica delle particelle)	500 – 800 Terabyte
Numero Complessivo di Record	$>$ 100 miliardi di righe

Nota: Per archiviare queste moli di dati, l'unico vero limite considerato dal DBMS è lo spazio fisico dei dischi (SSD o magnetici) e dello storage collegato in rete.

Condivisione, Ridondanza e Inconsistenza. In un'organizzazione complessa (es. un ateneo), uffici diversi (segreteria studenti, biblioteca, mensa) lavorano spesso sulle stesse informazioni (es. l'anagrafica degli studenti). Se ogni ufficio usasse un proprio archivio separato, nascerebbero grossi problemi. La frammentazione dei dati porta a due rischi tipici:

- **Ridondanza**: lo stesso dato viene duplicato in più archivi differenti (es. l'indirizzo di casa dello studente è salvato sia nel file della segreteria sia in quello della biblioteca).
- **Inconsistenza**: le copie duplicate non coincidono tra loro.
  Esempio: Lo studente cambia residenza e comunica il nuovo indirizzo solo alla segreteria; la biblioteca mantiene il vecchio indirizzo. Il sistema ora contiene due informazioni reciprocamente contraddittorie sullo stesso fatto.

Centralizzare i dati in un unico database gestito da un DBMS evita la ridondanza inutile e impedisce l'inconsistenza.

Evoluzione Storica dell'Architettura per la Gestione dei Dati. Il modo di archiviare i dati si è trasformato radicalmente nel corso dei decenni:

- **Anni '70 (Senza DBMS)**: i programmi leggevano e scrivevano direttamente su file di testo o file binari tramite il sistema operativo. Non c'era separazione tra il codice e il formato del file: se il file cambiava struttura, tutti i programmi andavano riscritti.
- **Anni '80 (Primi DBMS Relazionali)**: nasce il DBMS come strato intermedio. I programmi (scritti ad esempio in C o Pascal) non toccano più i file su disco, ma chiedono al DBMS le tabelle e i dati di cui hanno bisogno.
- **Anni '90 (Logica Condivisa e DBMS Attivi)**: parte delle regole di controllo viene spostata dentro il database con le **stored procedure** e i **trigger** (il database reagisce in automatico a modifiche e cancellazioni).
- **Anni 2000 (Architetture Web a Tre Livelli)**: separazione netta tra interfaccia e dati:

  $$\text{Browser (Client)} \longleftrightarrow \text{Server Applicativo (Java, PHP, Python)} \longleftrightarrow \text{DBMS}$$

- **Anni 2010 (Mobile e Cloud)**: decine di dispositivi diversi (app iOS, Android, web) si collegano ad API centralizzate che dialogano con basi di dati distribuite e scalabili.

Evoluzione della Logica Procedurale: Stored Procedure vs Trigger

Negli anni '90 i DBMS hanno iniziato a eseguire logica direttamente sul server:

- **Stored Procedure**: blocchi di codice salvati direttamente sul database ed eseguiti su richiesta delle applicazioni; riducono il traffico di rete ma spesso non sono facilmente portabili tra DBMS di marche diverse.
- **Trigger (DBMS Attivi)**: regole automatiche che scattano da sole secondo lo schema **Evento-Condizione-Azione**.
  Esempio: *Quando* viene inserito un nuovo ordine (*Evento*), *se* la quantità ordinata supera le scorte in magazzino (*Condizione*), *allora* blocca l'inserimento e invia una notifica (*Azione*).

Garanzie del DBMS: Riservatezza, Affidabilità e Transazioni. Poiché la base di dati è condivisa da molte persone, il DBMS applica regole di accesso granulari:

- *Esempio A*: Uno studente può visualizzare tutti i propri voti, ma ha il permesso di modificare soltanto la propria password e il proprio recapito telefonico.
- *Esempio B*: Il docente del corso può inserire e modificare i voti dei propri studenti, ma non può visualizzare o alterare i dati fiscali o bancari degli stessi.

Se salta la corrente o il computer va in crash mentre stiamo salvando un dato, il database non deve corrompersi né lasciare operazioni a metà. Per garantire questa affidabilità, il DBMS utilizza il meccanismo delle **transazioni**.

La Transazione: Proprietà Fondamentali (Prospettiva di Sistema)

Una **transazione** è un blocco di operazioni che il sistema tratta come un'unica unità di lavoro indivisibile, garantendo tre caratteristiche principali:

1. **Atomicità (Tutto o Niente)**: o tutte le istruzioni del blocco vanno a buon fine, oppure, al minimo errore o interruzione, il sistema annulla tutto e torna allo stato iniziale.
   Esempio del bonifico: Se trasferiamo 100€ dal conto A al conto B, il sistema deve togliere 100€ da A e aggiungere 100€ su B. Se la macchina va in crash subito dopo aver tolto i soldi da A, la transazione viene annullata e i 100€ ritornano su A.

2. **Coerenza e Gestione della Concorrenza**: più utenti possono operare contemporaneamente sugli stessi dati senza calpestarsi i piedi.
   Esempio del posto sul treno: Se due persone tentano di prenotare l'ultimo posto disponibile nello stesso identico secondo, il DBMS assegna il posto a una sola delle due e notifica l'esaurimento all'altra, evitando il sovrapprezzo o l'*overbooking*.

3. **Permanenza (Durabilità)**: una volta che l'operazione è confermata (*commit*), le modifiche sono scritte sul disco (e registrate in un file di log protetto). Anche se il computer si spegne improvvisamente un istante dopo, il dato salvato non andrà perso.

Il DBMS deve raggiungere due traguardi pratici:

- **Efficienza**: velocità di esecuzione e basso consumo di memoria, grazie a indici intelligenti (es. alberi B-Tree) e all'ottimizzatore di interrogazioni.

- **Efficacia**: fornire linguaggi semplici e potenti (come SQL) che permettano a programmatori e analisti di ottenere le risposte cercate senza dover scrivere algoritmi a basso livello.

Limiti del File System Tradizionale. Un normale sistema operativo permette di salvare dati organizzandoli in file e cartelle (**file system**).
Il limite principale dei file system è la **grana grossa**: un file va generalmente aperto, letto e riscritto per intero. Se abbiamo un file con 2 milioni di righe e vogliamo modificare il numero di telefono di una sola persona, il programma applicativo deve rileggere il file, localizzare la riga corretta in memoria e riscrivere il file sul disco. Il DBMS, al contrario, lavora a **grana fine**: individua e modifica direttamente il singolo record in frazioni di secondo.

Descrizione dei Dati e Catalogo Centralizzato. La differenza più profonda tra i due modelli sta nel modo in cui è descritta la forma dei dati:

- **Nei sistemi con File System**: ogni singolo programma deve sapere a memoria come è formattato il file (es. "i primi 20 byte contengono il cognome, i successivi 10 la data"). Se modifichiamo il formato del file aggiungendo un campo, tutti i programmi esistenti smettono di funzionare finché non vengono riscritti.

- **Nei DBMS**: la struttura dei dati è definita in un solo posto centrale chiamato **catalogo** (o **dizionario dei dati**). I programmi non devono conoscere i dettagli fisici di memorizzazione: chiedono semplicemente le informazioni per nome al DBMS.

1.2.2 Modelli dei Dati, Architettura e Indipendenza

Il Modello dei Dati. Un **modello dei dati** è una collezione di concetti usati per organizzare e descrivere i dati.
Ogni modello offre specifici **costruttori di tipo**, simili a quelli usati nella programmazione (come array o struct). Nel **modello relazionale**, il costruttore principale è la **relazione** (comunemente vista come una **tabella**), composta da righe omogenee (tuple).

Schema e Istanza. In qualunque base di dati si distingue nettamente tra forma e contenuto:

- **Schema (Aspetto Intensionale)**: l'intestazione, la struttura, i tipi di dato e i vincoli delle tabelle. È fisso e quasi mai soggetto a modifiche nel tempo.
- **Istanza (Aspetto Estensionale)**: i dati concreti memorizzati nelle righe in un determinato momento. È molto dinamica, poiché righe vengono inserite, aggiornate o cancellate di continuo.

Esempio: Schema e Istanza nella Tabella Orario Lezioni

Si consideri una semplice tabella che organizza l'orario delle lezioni:

Schema della Relazione (Intestazione fissa, definita una volta sola)			
Corso (*Course*)	**Docente (*Lecturer*)**	**Aula (*Room*)**	**Ora (*Time*)**
Istanza della Relazione (Le righe di dati salvate in questo momento)			
Analisi I	Luigi Neri	N1	8:00
Basi di Dati	Pier Rossi	N2	9:45
Chimica	Nicola Mori	N1	9:45
Fisica I	Mario Bruni	N1	11:45
Fisica II	Mario Bruni	N3	9:45
Sistemi Informatici	Pier Rossi	N3	8:00

Le intestazioni definiscono **cosa** può contenere la tabella (lo *schema*); le sei righe di valori rappresentano il contenuto effettivo attuale (l'*istanza*).

Modelli Logici vs Modelli Concettuali. Esistono due grandi famiglie di modelli usate in momenti diversi della progettazione:

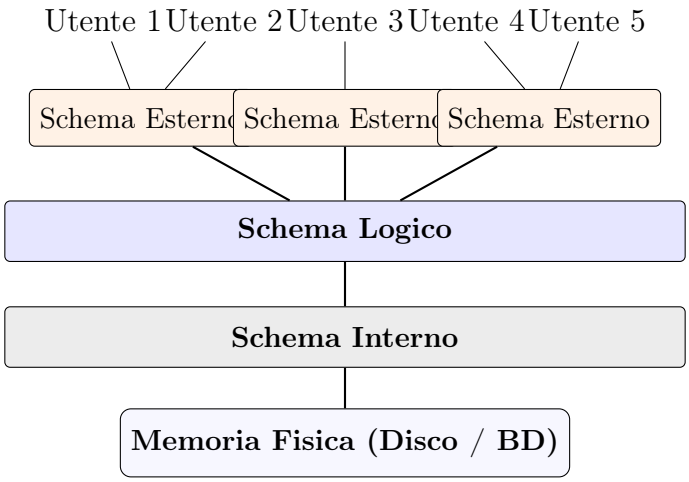
- **Modelli Logici**: sono i modelli compresi e usati dai DBMS per organizzare i dati su disco. Sono indipendenti dai dettagli fisici del disco, ma già vicini all'implementazione software.
  Esempi: il modello **relazionale** (a tabelle), il modello gerarchico, a oggetti o documentale/-JSON (NoSQL).
- **Modelli Concettuali**: sono formalismi grafici e intuitivi usati per dialogare con i committenti e descrivere la realtà senza pensare a quale software verrà usato in futuro.
  Il più celebre: il **Modello Entità-Relazione (E-R)**.

Architettura dei DBMS: Livelli e Astrazione. In prima approssimazione, un sistema di gestione dati separa due aspetti (Architettura Semplificata a Due Livelli):

- **Schema Logico**: la vista complessiva della struttura (quali tabelle esistono, quali colonne contengono, quali relazioni ci sono tra loro).
- **Schema Interno (o Fisico)**: come quei record sono disposti sui blocchi del disco, quali puntatori e quali file fisici vengono usati.

Architettura Standard ANSI/SPARC a Tre Livelli. Articola la visione dei dati su tre livelli:

1. **Livello Esterno (Schemi Esterni / Viste)**: schermate e finestre ritagliate su misura per utenti specifici. Ogni gruppo di utenti vede solo i dati di proprio interesse.
2. **Livello Logico (Schema Logico)**: la descrizione completa, unica e priva di ridondanze dell'intera base di dati.
3. **Livello Interno (Schema Interno)**: l'organizzazione fisica su disco, i file di memoria e le strutture di indicizzazione.



Il Meccanismo delle Viste. Una **vista** è una tabella virtuale: non memorizza dati duplicati su disco, ma calcola le righe al momento eseguendo una query sulle tabelle reali sottostanti.

Esempio di Costruzione di una Vista: Accoppiamento Corsi-Aule

Consideriamo due tabelle di base memorizzate nel database:

Tabella Lezioni

Corso	Docente	Aula
Basi di Dati	Rossi	DS3
Sistemi	Neri	N3
Reti	Bruni	N3
Controlli	Bruni	G

Tabella Aule

Nome	Edificio	Piano
DS1	OMI	Terra
N3	OMI	Terra
G	Pincherle	Primo

Se agli studenti interessa solo sapere dove andare a lezione (senza vedere il nome del docente o le aule vuote), il DBMS può creare al volo la vista **AuleLezioni**, collegando le due tabelle tramite l'aula:

Corso	Aula	Edificio	Piano
Sistemi	N3	OMI	Terra
Reti	N3	OMI	Terra
Controlli	G	Pincherle	Primo

In questo modo l'utente vede una tabella pulita e pronta all'uso, senza che i dati siano stati duplicati sul disco.

Le Due Forme di Indipendenza dei Dati. Separare i tre livelli garantisce il principio di **indipendenza dei dati**:

- **Indipendenza Fisica**: possiamo cambiare il modo in cui i file sono scritti su disco, spostare il database su un SSD più veloce o aggiungere un indice per velocizzare le ricerche, **senza dover cambiare una sola riga** dei programmi applicativi o delle query SQL.
- **Indipendenza Logica**: possiamo aggiungere nuove tabelle o nuove colonne al database **senza rompere** i programmi preesistenti, che continuano a funzionare interagendo con le loro viste abituali.

1.2.3 Linguaggi per Basi di Dati e Modalità di Utilizzo

Classificazione Funzionale: DDL vs DML. Nei database si usano due categorie distinte di comandi:

- **DDL (Data Definition Language)**: comandi per creare o modificare la struttura delle tabelle (lo schema).
  Esempi: `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`.
- **DML (Data Manipulation Language)**: comandi per inserire, cercare, modificare ed eliminare i record effettivi (l'istanza).
  Esempi: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

```
CREATE TABLE orario (
    corso   CHAR(20),
    docente CHAR(20),
    aula    CHAR(4),
    ora     CHAR(5)
);
```

Listing 1: Esempio DDL: creazione dello schema di una tabella

I Quattro Paradigmi di Interazione con la Base di Dati. Un utente o un programma può dialogare con il DBMS in quattro modi:

1. Linguaggio Interattivo Testuale (SQL a riga di comando). L'utente scrive direttamente comandi SQL in una console per ottenere subito i risultati a video:

```sql
SELECT Corso, Aula, Piano
FROM Aule, Lezioni
WHERE Nome = Aula
  AND Piano = 'Terra';
```

Listing 2: Query SQL interattiva

2. SQL Immerso in un Linguaggio Ospite (*Embedded SQL*). I comandi SQL vengono inseriti all'interno di un linguaggio tradizionale (es. C o Pascal). Poiché SQL restituisce insiemi di righe mentre le variabili dei linguaggi procedurali leggono un valore alla volta, si usa un **cursore** per scorrere i risultati riga per riga:

```
write('Nome della citta? '); readln(citta);
EXEC SQL DECLARE P CURSOR FOR
    SELECT NAME, INCOME
    FROM PEOPLE
    WHERE CITY = :citta;
EXEC SQL OPEN P;
EXEC SQL FETCH P INTO :name, :income;
while SQLCODE = 0 do begin
    write('Nome persona: ', name, ' Aumento? ');
    readln(aumento);
    EXEC SQL UPDATE PEOPLE
        SET INCOME = INCOME + :aumento
        WHERE CURRENT OF P;
    EXEC SQL FETCH P INTO :name, :income;
end;
EXEC SQL CLOSE CURSOR P;
```

Listing 3: Esempio di Embedded SQL in Pascal con cursore

3. Linguaggi Procedurali Integrati nel DBMS (es. Oracle PL/SQL). Si tratta di linguaggi che uniscono istruzioni SQL a costrutti classici di programmazione (`IF`, cicli `WHILE`, gestione degli errori):

```
DECLARE
    Stip NUMBER;
BEGIN
    SELECT SALARY INTO Stip FROM EMPLOYEE
    WHERE NUMBER = '575488' FOR UPDATE OF SALARY;

    IF Stip > 30 THEN
        UPDATE EMPLOYEE SET SALARY = SALARY * 1.1
        WHERE NUMBER = '575488';
    ELSE
        UPDATE EMPLOYEE SET SALARY = SALARY * 1.15
        WHERE NUMBER = '575488';
    END IF;

    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        INSERT INTO ERRORS
```

```
19          VALUES ('MATRICOLA NON ESISTENTE', SYSDATE);
20  END;
```

Listing 4: Esempio di blocco procedurale PL/SQL

4. Interfacce Grafiche (GUI e Maschere a Schermo). Gli utenti non tecnici (es. operatori di sportello, cassieri) non scrivono comandi SQL: usano maschere grafiche con campi da compilare, pulsanti e menu a tendina (es. Microsoft Access o pagine web gestionali). Dietro le quinte, il software genera ed esegue i comandi SQL corrispondenti.

1.2.4 Attori, Ruoli, Transazioni e Valutazione dei DBMS

I Quattro Gruppi di Ruoli Operativi. Nel mondo delle basi di dati lavorano quattro figure principali:

1. **Costruttori del DBMS**: gli ingegneri informatici che scrivono il motore del DBMS (progettano gli ottimizzatori di query, la gestione della memoria su disco e i compilatori interni).
2. **Progettisti del Database e DBA**: chi decide come strutturare le tabelle per uno specifico problema e ne amministra il funzionamento quotidiano.
3. **Sviluppatori Applicativi**: i programmatori che scrivono i programmi e le interfacce utente per interagire con i dati.
4. **Utenti**:
   - *Utenti Finali (Operativi)*: usano interfacce guidate e maschere predefinite per compiti di routine (es. l'impiegato postale che registra un pacco).
   - *Utenti Casuali (Casual Users)*: analisti o manager che interrogano il database formulando query SQL personalizzate per prendere decisioni strategiche.

L'Amministratore della Base di Dati (DBA). Il **DBA** (*Database Administrator*) è il custode e responsabile dell'efficienza e della salute del database. I suoi compiti principali sono:

- Monitorare le prestazioni del sistema e configurare gli indici per non rallentare le ricerche.
- Pianificare i salvataggi periodici (**backup**) e le procedure di ripristino dei dati in caso di guasti o disastri (*disaster recovery*).
- Gestire gli account, le password e le autorizzazioni di sicurezza di tutti gli utenti.
- Spesso collabora anche alla progettazione iniziale e all'evoluzione delle tabelle.

I Due Punti di Vista sulla Transazione. La parola "transazione" ha due sfumature:

- **Per l'utente finale**: è un'azione applicativa compiuta con uno scopo chiaro (es. fare un bonifico, comprare un biglietto aereo, iscriversi a un esame).
- **Per il DBMS**: è una serie indivisibile di comandi che devono essere eseguiti con successo al 100% oppure annullati del tutto, salvaguardando la coerenza dell'archivio anche in caso di errori o accessi concorrenti.

I Vantaggi (Pros) nell'Adozione di un DBMS. Scegliere un DBMS porta chiari vantaggi:

- **Dati Condivisi**: un'unica fonte di verità centralizzata per tutti i reparti dell'azienda.
- **Niente Duplicazioni Inutili**: si eliminano i rischi di avere dati disallineati e contraddittori.
- **Servizi Integrati**: il DBMS gestisce da solo sicurezza, controllo della concorrenza, transazioni e salvataggi.
- **Indipendenza dei Dati**: se si modifica l'hardware o la disposizione fisica dei file, non serve riscrivere i software applicativi.

Gli Svantaggi e i Costi (Cons). Un DBMS non è sempre la soluzione ideale in qualunque scenario, a causa di due fattori:

- **Costi Economici e Complessità**: le licenze enterprise sono onerose, richiedono server potenti e personale altamente qualificato (i DBA) per l'amministrazione.
- **Struttura Impegnativa (*Overhead*)**: le funzioni di controllo transazionale, sicurezza e logging sono sempre attive e consumano risorse. Per compiti minuscoli o file di log elementari, un semplice file di testo gestito dal sistema operativo può risultare più veloce ed economico rispetto a un intero DBMS.

2 Il Modello Relazionale dei Dati

2.1 Struttura del Modello Relazionale

2.1.1 Origine, Modelli Logici e Paradigma Basato su Valori

Evoluzione dei Modelli Logici Tradizionali. Prima dell'avvento del modello relazionale, la gestione dei dati nelle applicazioni aziendali si basava su due grandi modelli logici:

- **Modello Gerarchico**: organizza i record secondo strutture ad albero piramidali; ogni record figlio possiede un unico record genitore, replicando fedelmente le gerarchie rigide ma rendendo complessa la gestione di relazioni molti-a-molti.
- **Modello Reticolare**: generalizza il modello gerarchico consentendo strutture a grafo; un record può avere più record proprietari (*owner*) e record membri (*member*).

Entrambi questi modelli presentano un limite strutturale insidioso: le associazioni tra i dati sono espresse mediante **riferimenti espliciti** (puntatori di memoria fisici o logici gestiti a basso livello). Chi programma deve conoscere nel dettaglio la topologia dei collegamenti per navigare l'archivio record per record.

Modelli Alternativi e Successivi. Negli anni successivi sono emerse ulteriori proposte:

- **Modello a Oggetti (*Object-Oriented*)**: estende ai database i concetti della programmazione a oggetti (incapsulamento, metodi, ereditarietà, identità tramite OID). Sebbene potente, ha trovato diffusione solo in nicchie specifiche e complesse (es. CAD/CAM, sistemi geografici).
- **Modello Documentale e Semistrutturato (XML / JSON)**: nato per gestire dati privi di uno schema rigido a priori o fortemente gerarchici, oggi ampiamente adottato nei sistemi NoSQL ed è da considerarsi complementare al modello relazionale.

La Rivoluzione di Edgar F. Codd (1970). Il modello relazionale fu proposto formalmente da **Edgar F. Codd** nel celebre articolo del 1970 con un obiettivo primario: raggiungere la piena **indipendenza logica e fisica dei dati**. Nei modelli basati su puntatori, ogni modifica alla struttura interna dei file su disco obbligava a riscrivere il software applicativo. Codd propose di rappresentare l'intera base di dati mediante una struttura concettuale pulita e matematica: la **relazione**.

L'adozione commerciale effettiva arrivò circa un decennio dopo, nel **1981**, poiché garantire l'indipendenza dei dati senza sacrificare le prestazioni computazionali e l'affidabilità su memorie di massa richiese anni di ricerca algoritmica (in particolare per lo sviluppo di optimizer di query e indici su albero).

Puntatori Espliciti vs Paradigma Basato su Valori (*Value-Based*)

La distinzione cruciale tra i sistemi pre-relazionali e il modello relazionale risiede nel meccanismo con cui si stabiliscono i legami tra le entità:

- **Sistemi basati su puntatori (Gerarchico/Reticolare)**: i collegamenti tra record

diversi sono rappresentati da indirizzi o identificatori fisici gestiti dal motore di sistema. Il legame è intrinsecamente **direzionale** (un puntatore va da un record A a un record B, ma per risalire da B ad A occorrono strutture aggiuntive).

- **Modello relazionale (Basato su Valori)**: le correlazioni tra i dati memorizzati in relazioni diverse sono espresse **esclusivamente attraverso i valori** contenuti nelle tuple stesse.

Esempio concreto: se una tabella **Esami** deve indicare quale studente ha sostenuto la prova, essa non conterrà l'indirizzo su disco del record dello studente, bensì il valore della sua matricola (es. `3456`). Tale valore coincide con la chiave memorizzata nella tabella **Studenti**.

I Vantaggi Chiave della Struttura Basata su Valori. L'adozione del paradigma *value-based* porta benefici sostanziali:

- **Indipendenza dalla struttura fisica**: i puntatori fisici cambiano quando i dati vengono riallocati o deframmentati su disco; i valori informativi rimangono inalterati indipendentemente dal supporto hardware.
- **Pertinenza applicativa**: nel database si salvano esclusivamente i dati che hanno un chiaro significato informativo per il dominio applicativo, scartando metadati di navigazione fittizi.
- **Visione uniforme per utente e sviluppatore**: chi interroga la base di dati osserva le medesime strutture tabellari su cui ragiona il programmatore, senza dettagli nascosti.
- **Interoperabilità e portabilità**: dati basati su valori possono essere esportati, serializzati e scambiati con facilità tra piattaforme e DBMS eterogenei.
- **Simmetria e navigazione bidirezionale**: mentre un puntatore impone una traiettoria prefissata, l'uguaglianza sui valori consente di navigare la correlazione con pari facilità in entrambi i versi (trovare tutti gli esami sostenuti dallo studente X oppure risalire all'anagrafica dello studente che ha superato l'esame Y).

2.1.2 Dalla Relazione Matematica alla Relazione Tabellare

Chiarimento Terminologico Fondamentale. L'uso del termine "relazione" genera spesso equivoci tra mondi contigui:

- **Relazione logico-matematica (Teoria degli Insiemi)**: sottoinsieme del prodotto cartesiano tra insiemi di elementi.
- **Relazione nel Modello Relazionale**: costrutto logico rappresentabile come una tabella bidimensionale con intestazioni dotate di significato.
- **Relationship (Associazione o Correlazione)**: concetto del **Modello Entità-Relazione (E-R)**, che descrive una classe di fatti o un legame concettuale tra due o più entità del mondo reale.

La Relazione Matematica Insiemistica. Siano $D_1, D_2, \ldots, D_n$ insiemi detti **domini** (non necessariamente distinti tra loro).

- Il **prodotto cartesiano** $D_1 \times D_2 \times \cdots \times D_n$ è l'insieme di tutte le n-uple ordinate $(d_1, d_2, \ldots, d_n)$ tali per cui:
$$d_1 \in D_1, \quad d_2 \in D_2, \quad \ldots, \quad d_n \in D_n$$
- Una **relazione matematica** r sui domini $D_1, \ldots, D_n$ è un qualsiasi sottoinsieme del loro prodotto cartesiano:
$$r \subseteq D_1 \times D_2 \times \cdots \times D_n$$
I domini $D_1, \ldots, D_n$ prendono il nome di **domini della relazione**, mentre il numero n rappresenta il **grado** della relazione.

Proprietà della Relazione Insiemistica: Notazione Posizionale

In base alla rigorosa definizione della teoria degli insiemi:

1. **Una relazione è un insieme**: ne discende che tra le ennuple non esiste alcun ordinamento intrinseco e che non possono esistere due ennuple identiche (le tuple sono tutte distinte).
2. **Ogni ennupla è una sequenza ordinata**: la posizione dell'i-esimo elemento determina in modo rigido da quale dominio esso provenga. L'accesso ai valori si basa sulla loro **posizione ordinale** $(1°, 2°, \ldots, n°)$.

Il Limite della Posizionalità: L'Esempio del Calendario Partite. Si consideri una relazione $Partite \subseteq \text{String} \times \text{String} \times \text{Int} \times \text{Int}$ che descrive i risultati di un girone calcistico:

1° Componente	2° Componente	3° Componente	4° Componente
Barcellona	Bayern	3	1
Real Madrid	Bayern	2	0
Barcellona	Juventus	0	2
Juventus	Real Madrid	0	1

In questo schema insiemistico, il dominio String appare due volte e il dominio Int compare due volte. Il loro ruolo semantico è affidato unicamente alla posizione:

- Posizione 1: squadra ospitante (*Home*).
- Posizione 2: squadra ospitata (*Away*).
- Posizione 3: reti realizzate dalla squadra di casa (*GoalsH*).
- Posizione 4: reti realizzate dalla squadra ospite (*GoalsA*).

Se scambiassimo di posto le prime due colonne senza mutare i dati, il significato del record risulterebbe stravolto.

Dalla Struttura Posizionale alla Struttura Non Posizionale. Nei database reali, affidare il significato alla posizione è rischioso e scomodo. Il modello relazionale supera questo limite

introducendo gli **attributi**: a ciascuna colonna viene assegnato un nome simbolico univoco che ne definisce formalmente il ruolo.

L'Attributo e l'Indipendenza dall'Ordine delle Colonne

Un **attributo** è un nome unico associato a un dominio per esplicitarne il significato applicativo all'interno della relazione.
Con l'introduzione degli attributi, la tabella precedente viene modellata come:

SquadraCasa	SquadraOspite	GolCasa	GolOspite
Barcellona	Bayern	3	1
Bayern	Real Madrid	2	0
Barcellona	Juventus	0	2
Juventus	Real Madrid	0	1

Poiché ciascun dato è identificato dal nome dell'attributo e non dal suo indice numerico, **l'ordine orizzontale delle colonne è del tutto irrilevante**:

SquadraOspite	SquadraCasa	GolOspite	GolCasa
Bayern	Barcellona	1	3
Real Madrid	Bayern	0	2
Juventus	Barcellona	2	0
Real Madrid	Juventus	1	0

Entrambe le tabelle rappresentano la medesima informazione logica: la struttura dei dati è pienamente **non posizionale**.

Proprietà di una Tabella come Relazione. Una tabella su video o su carta rappresenta una valida relazione relazionale se e solo se rispetta le seguenti condizioni:

1. Tutte le righe sono distinte tra loro (nessuna tupla duplicata).
2. L'ordine delle righe è irrilevante.
3. Tutte le intestazioni di colonna (attributi) sono distinte all'interno della stessa tabella.
4. L'ordine delle colonne è irrilevante.
5. I valori all'interno di ciascuna colonna sono omogenei rispetto al dominio dell'attributo.

2.1.3 Formalizzazione: Schemi, Istanze e Tuple

Schema di Relazione. Uno **schema di relazione** $R(X)$ è costituito da:

- Un nome simbolico R (es. `STUDENTI`).
- Un insieme di nomi di attributi $X = \{A_1, A_2, \ldots, A_n\}$.
- A ciascun attributo A_i è associato un insieme di valori ammissibili, denominato dominio $\text{dom}(A_i)$.

Viene compattamente indicato con $R(A_1, A_2, \ldots, A_n)$.

Schema di Base di Dati. Uno **schema di base di dati relazionale** $\mathcal{R}$ è una collezione finita di schemi di relazione con nomi distinti:

$$\mathcal{R} = \{R_1(X_1), R_2(X_2), \ldots, R_k(X_k)\}$$

Definizione Formale di Tupla. Data una relazione con schema $R(X)$, una **tupla** t su X non è una semplice sequenza ordinata, bensì una **funzione matematica** che mappa ogni attributo $A \in X$ in un valore del suo rispettivo dominio:

$$t : X \longrightarrow \bigcup_{A \in X} \text{dom}(A) \quad \text{tale che} \quad t[A] \in \text{dom}(A) \quad \forall A \in X$$

La notazione $t[A]$ (oppure $t.A$) indica il valore assunto dalla tupla t in corrispondenza dell'attributo A.

Esempio Completo: Schemi e Istanze in un Contesto Didattico

Si consideri lo schema di base di dati universitario:

$$\begin{aligned}\mathcal{R} = \{&\text{STUDENTI}(\text{Matricola}, \text{Cognome}, \text{Nome}, \text{DataNascita}),\\ &\text{CORSI}(\text{Codice}, \text{NomeCorso}, \text{Docente}),\\ &\text{ESAMI}(\text{Candidato}, \text{Voto}, \text{Corso})\}\end{aligned}$$

Una possibile **istanza di relazione** per la tabella **STUDENTI** è l'insieme di tuple:

Tupla	**Matricola**	**Cognome**	**Nome**	**DataNascita**
t_1	6554	Rossi	Mario	1978/12/05
t_2	8765	Neri	Paolo	1976/11/03
t_3	9283	Verdi	Luisa	1979/11/12
t_4	3456	Rossi	Maria	1978/02/01

Se consideriamo la prima tupla t_1, possiamo estrarre i singoli componenti mediante la notazione funzionale:

$$t_1[\text{Matricola}] = 6554, \quad t_1[\text{Cognome}] = \text{'Rossi'}, \quad t_1[\text{DataNascita}] = \text{'1978/12/05'}$$

Istanza di Relazione e di Base di Dati.

- Un'**istanza di relazione** r su uno schema $R(X)$ è un insieme finito di tuple definite su X.
- Un'**istanza di base di dati** su uno schema $\mathcal{R} = \{R_1(X_1), \ldots, R_k(X_k)\}$ è l'insieme delle rispettive istanze di relazione:
$$r = \{r_1, r_2, \ldots, r_k\}$$
dove ogni r_i è un'istanza conforme allo schema $R_i(X_i)$.

Relazioni con un Solo Attributo (Unarie). Una relazione può essere definita su un unico attributo ($n = 1$). Tali relazioni sono molto utili per rappresentare sottoinsiemi, stati o proprietà del dominio senza dover introdurre colonne booleane ridondanti.

STUDENTI_LAVORATORI
Matricola
6554
3456

La presenza del valore indica l'appartenenza dello studente alla categoria dei lavoratori.

2.1.4 Strutture Dati Annidate e Appiattimento (*Unnesting*)

La Complessità dei Dati del Mondo Reale. Nei documenti cartacei e nelle ricevute contabili i dati si presentano spesso in forma gerarchica e ricorsiva. Si pensi a uno scontrino rilasciato da un ristorante:

- Informazioni di testata generali: nome del locale, indirizzo, data, numero scontrino fiscale e conto totale.
- Corpo dello scontrino: una lista variabile di consumazioni (ciascuna con quantità, descrizione della portata e prezzo unitario o complessivo).

Il Vincolo della Prima Forma Normale (1NF). Nel modello relazionale canonico vige il vincolo di **atomicità dei domini**: i valori contenuti all'incrocio tra una riga e una colonna devono essere indivisibili. Non è consentito inserire come valore di un attributo una lista, un vettore o una sotto-tabella (strutture non in *First Normal Form* o ¬1NF).

Il Processo di Decomposizione e Appiattimento (*Unnesting*)

Per rappresentare fedelmente uno scontrino nel modello relazionale, la struttura annidata deve essere scomposta (*unnested*) in due relazioni distinte ma correlate per valore:

Relazione RICEVUTE

Numero	Data	Totale
1235	12/10/2002	39,20
1240	13/10/2002	39,00

Relazione PORTATE (Dettaglio Scontrino)

Ricevuta	Riga	Descrizione	Prezzo
1235	1	3 Coperti	3,00
1235	2	2 Antipasti	6,20
1235	3	3 Primi	12,00
1235	4	2 Bistecche	18,00
1240	1	2 Coperti	2,00
1240	2	2 Antipasti	7,00

Analisi delle Scelte Progettuali:

- Il campo **Ricevuta** nella tabella **PORTATE** funge da connettore logico verso la tabella **RICEVUTE**.
- È stato introdotto l'attributo esplicito **Riga** per preservare l'ordinamento originario con cui i piatti sono stati battuti alla cassa. Poiché il modello relazionale non assegna alcun significato all'ordine di memorizzazione delle righe, l'unico modo per tracciare una sequenza è renderla un dato informativo esplicito.
- L'introduzione della riga d'ordine consente inoltre di ammettere portate duplicate consumate in momenti diversi dello stesso pasto (es. due ordinazioni successive della stessa bevanda).

2.1.5 Informazione Parziale e Gestione del Valore NULL

La Rigidità dello Schema e il Problema dei Valori Mancanti. Il modello relazionale impone una struttura stretta: ogni tupla conforme a uno schema $R(A_1, \ldots, A_n)$ deve possedere un valore per ciascuno degli attributi previsti. Nella realtà dei fatti, tuttavia, le informazioni raccolte possono essere incomplete o temporaneamente non disponibili.

Si consideri l'anagrafica di quattro personaggi storici memorizzata in uno schema composto da `Nome`, `SecondoNome` (*Middle Name*) e `Cognome`:

Nome	**SecondoNome**	**Cognome**
Franklin	Delano	Roosevelt
Winston	*?*	Churchill
Charles	*?*	De Gaulle
Josip	*?*	Stalin

Franklin D. Roosevelt possiede effettivamente un secondo nome, mentre gli altri statisti ne sono sprovvisti o non è registrato. Come gestire la casella vuota?

Perché Non Usare Valori Sentinella del Dominio. Un approccio ingenuo consiste nell'adottare valori speciali appartenenti al dominio (es. stringa vuota `""`, numeri convenzionali come `0`, `-1` o `9999`). Questa tecnica presenta gravi difetti:

- **Mancanza di valori disponibili**: in molti domini continui o aperti non esistono valori non validi o inutilizzati a priori.
- **Collisioni future**: valori scelti come sentinella potrebbero acquisire un significato reale nel tempo (es. uno zero per la temperatura o per un saldo bancario non significa assenza di dato).
- **Onere applicativo ed errori di calcolo**: i programmatori devono ricordare a memoria le sentinelle e gestirle manualmente nel codice con condizioni `IF`. Se la sentinella non viene filtrata, altera funzioni aggregate come medie e conteggi.

La Soluzione Relazionale: Il Marcatore Speciale NULL

Per superare i limiti delle sentinelle, il modello relazionale introduce un costrutto universale dedicato: il valore **NULL**.

- NULL denota in modo formale l'**assenza di un valore informativo valido**.
- NULL **non appartiene ad alcun dominio di tipo**: si estende la definizione formale di tupla ammettendo che:

$$t[A] \in \text{dom}(A) \cup \{\text{NULL}\}$$

- Nei database moderni, l'ammissibilità dei valori NULL può essere inibita specificando nello schema vincoli dedicati (es. la clausola `NOT NULL` del linguaggio SQL).

Le Tre Sfumature Semantiche del Valore Mancante. Dal punto di vista concettuale, l'assenza di un dato può manifestarsi sotto tre forme profondamente differenti:

1. **Valore Sconosciuto (*Unknown*)**: il valore esiste realmente nel mondo reale, ma al momento della registrazione non è noto al sistema.
   Esempio: la data di nascita o l'indirizzo di residenza di una persona che non ha ancora compilato il modulo anagrafico.
2. **Valore Inesistente o Non Applicabile (*Inexistent / Not Applicable*)**: l'attributo non ha alcuna ragione logica di esistere per quella specifica entità.
   Esempio: il numero di patente di guida per un individuo che non ha mai conseguito la licenza, o il cognome da nubile per un soggetto di sesso maschile.
3. **Valore Non Informativo o Riservato (*No Information*)**: non è possibile determinare se il dato esista o meno, oppure la legislazione sulla privacy ne preclude l'archiviazione.

Trattamento nei DBMS e il Rischio di Proliferazione (*Too Many NULLs*). Sebbene la teoria riconosca la triplice sfumatura di significato, i DBMS commerciali non distinguono internamente tra valore sconosciuto e valore inesistente: entrambi sono registrati indistintamente tramite il bit di marcatura NULL.

L'abuso dei valori NULL introduce severe complicazioni:

- Richiede l'adozione di una logica a tre valori di verità (*True*, *False*, *Unknown*) nell'elaborazione dei predicati delle query SQL.
- Rende ambigue le operazioni di correlazione e join tra tabelle diverse.
- Se una tupla accumula troppi valori NULL (come illustrato nella tabella sottostante), perde la sua identità informativa e cessa di essere utile all'applicazione.

Tabella STUDENTI con Eccessiva Presenza di Valori NULL

Matricola	Cognome	Nome	DataNascita
6554	Rossi	Mario	1978/12/05
8765	Neri	Paolo	1976/11/03
9283	Verdi	Luisa	1979/11/12
NULL	Rossi	Maria	1978/02/01

Come evidenziato nell'ultima riga, se il valore NULL si presenta su un attributo deputato a identificare la riga (la matricola), il record non può più essere collegato in modo univoco agli esami sostenuti o ad altre tabelle del database. Nasce così la necessità di regole strutturali rigide: i **vincoli di integrità**.

2.2 Vincoli di Integrità

2.2.1 Vincoli di Integrità e Loro Classificazione

Limiti della Correttezza Sintattica. Non tutte le istanze di basi di dati sintatticamente conformi a uno schema descrivono una realtà plausibile o corretta per l'applicazione. Si consideri la seguente istanza di prova:

Relazione ESAMI

Matricola	Voto	Lode	Corso
276545	**32**		01
276545	30	Lode	02
787643	**27**	**Lode**	03
739430	24		04

Relazione STUDENTI

Matricola	Cognome	Nome
276545	Rossi	Mario
787643	Neri	Piero
787643	Bianchi	Luca

Dal punto di vista dei tipi di dato l'istanza è impeccabile (i voti sono interi, i nomi sono stringhe), eppure contiene gravi anomalie semantiche:

- Un voto d'esame pari a 32, valore inammissibile nella scala di valutazione universitaria ($18 \leq$ voto ≤ 30).
- L'assegnazione della lode a una prova superata con 27, combinazione priva di senso regolamentare.
- La presenza di due studenti differenti registrati con la medesima matricola (`787643`).

Definizione Formale di Vincolo di Integrità

Un **vincolo di integrità** è un predicato logico (una funzione booleana) associato a uno schema di base di dati che associa a ogni possibile istanza di relazione o di base di dati il

valore di verità **Vero** (*True*) o **Falso** (*False*):

$$\mathcal{C} : \text{Istanze}(R) \longrightarrow \{\text{True}, \text{False}\}$$

- Un'istanza di base di dati si definisce **valida** (o **legale**) se soddisfa tutti i vincoli di integrità definiti sullo schema ($\mathcal{C}(r) = \text{True}$).
- Un'istanza è **non valida** se viola anche un solo vincolo ($\mathcal{C}(r) = \text{False}$).

Motivazioni per l'Introduzione dei Vincoli. La definizione esplicita dei vincoli è essenziale per molteplici scopi:

- **Accuratezza Semantica**: consente di descrivere in modo rigoroso e formale le regole del mondo reale modellato nel database.
- **Qualità dei Dati (*Data Quality*)**: funge da barriera protettiva contro errori di digitazione dell'utente o anomalie dei programmi applicativi.
- **Guida nella Progettazione**: aiuta a strutturare gli schemi e a identificare precocemente ridondanze o difetti di normalizzazione.
- **Ottimizzazione delle Query**: l'ottimizzatore interno del DBMS sfrutta la certezza dell'integrità dei dati per semplificare i piani di esecuzione delle interrogazioni, scartando scansioni inutili di blocchi su disco.

Supporto dei Vincoli nei DBMS. I DBMS non offrono il medesimo livello di supporto per qualsiasi tipo di vincolo:

- **Vincoli Dichiarativi Nativi**: sono supportati direttamente dal linguaggio DDL del DBMS (es. chiavi primarie, vincoli di unicità, integrità referenziale e controlli generici con la clausola `CHECK`). Il DBMS ne verifica automaticamente il rispetto durante ogni operazione di scrittura, respingendo le transazioni illegali.
- **Vincoli Procedurali Esterni**: vincoli complessi che coinvolgono predicati articolati o computazioni aggregate su molte tabelle spesso non sono esprimibili tramite DDL standard. Devono pertanto essere implementati dal programmatore tramite **trigger**, **stored procedure** o controlli inseriti nel software applicativo esterno.

Tassonomia dei Vincoli di Integrità. I vincoli si dividono in due grandi classi:

1. **Vincoli Intra-relazionali**: il predicato deve essere valutato esaminando i dati contenuti all'interno di una singola relazione.
   - *Vincoli su valori (o di dominio)*: coinvolgono un singolo attributo isolato.
   - *Vincoli di tupla*: coinvolgono più attributi ma all'interno della medesima tupla.
2. **Vincoli Inter-relazionali**: il predicato coinvolge tuple appartenenti a relazioni distinte (o alla stessa relazione esaminata su istanze diverse), coordinando la coerenza complessiva della base di dati.

2.2.2 Vincoli Intra-relazionali: Dominio e Tupla

Vincoli di Dominio. Un **vincolo di dominio** è una condizione imposta sul singolo attributo di una relazione, limitando l'insieme dei valori ammissibili a un sottoinsieme proprio del dominio di tipo:

$$t[A] \in D' \subseteq \text{dom}(A)$$

Esempi: imporre che il voto di un esame sia compreso nell'intervallo $[18, 30]$, che l'età di un dipendente sia strettamente positiva o che il campo `Genere` ammetta unicamente i caratteri `'M'`, `'F'` o `'N'`.

Vincoli di Tupla. Un **vincolo di tupla** esprime una regola che coinvolge una combinazione di più attributi appartenenti alla medesima riga, **in modo del tutto indipendente** dalle altre tuple memorizzate nella tabella.

Sintassi dei Vincoli di Tupla. Formalmente, un vincolo di tupla viene espresso come un'espressione booleana composta da atomi collegati da connettivi logici (`AND`, `OR`, `NOT`). Ciascun atomo può essere:

- Un confronto tra valori o attributi del dominio (es. $(\text{Voto} \geq 18) \wedge (\text{Voto} \leq 30)$).
- Un confronto tra espressioni aritmetiche calcolate sugli attributi della tupla.

Esempio: Vincolo di Tupla Contabile sulla Busta Paga

Si consideri la tabella **STIPENDI**, dove ogni riga rappresenta la retribuzione mensile di un dipendente suddivisa in imponibile lordo, trattenute fiscali e compenso netto percepito:

Dipendente	StipendioLordo	Ritenute	StipendioNetto
Rossi	55.000	12.500	42.500
Neri	45.000	10.000	35.000
Bruni	47.000	11.000	36.000

Il dominio contabile impone un rigoroso **vincolo di tupla aritmetico**:

$$\text{StipendioLordo} = \text{Ritenute} + \text{StipendioNetto}$$

Tutte le tuple dell'istanza sopra riportata soddisfano il vincolo:

- Tupla di Rossi: $55.000 = 12.500 + 42.500$ (*Verificato*).
- Tupla di Neri: $45.000 = 10.000 + 35.000$ (*Verificato*).
- Tupla di Bruni: $47.000 = 11.000 + 36.000$ (*Verificato*).

Esempio di Violazione del Vincolo. Si supponga che un aggiornamento errato alteri la riga del dipendente Bruni come segue:

Dipendente	StipendioLordo	Ritenute	StipendioNetto
Bruni	**50.000**	11.000	36.000

Poiché $11.000 + 36.000 = 47.000 \neq 50.000$, la tupla viola il vincolo di integrità di tupla; il DBMS deve respingere la modifica lasciando inalterato lo stato della base di dati.

2.2.3 Identificazione delle Tuple: Superchiavi e Chiavi

Il Problema dell'Identificazione Univoca. Nel paradigma relazionale basato su valori, non esistono puntatori fisici o numeri di riga impliciti. L'unico modo per individuare una specifica tupla, accedere alle sue informazioni e collegarla ad altre relazioni consiste nel fare leva sui valori dei suoi attributi.

Definizioni Formali: Superchiave e Chiave

Sia r un'istanza di relazione conforme allo schema $R(X)$ con $X = \{A_1, A_2, \ldots, A_n\}$:

1. **Superchiave**: un insieme di attributi $K \subseteq X$ è una **superchiave** per r se l'istanza non contiene due tuple distinte $t_1, t_2 \in r$ che coincidano su tutti gli attributi di K:

$$\forall t_1, t_2 \in r, \quad (t_1 \neq t_2) \implies t_1[K] \neq t_2[K]$$

2. **Chiave (o Chiave Candidata)**: un insieme di attributi $K \subseteq X$ è una **chiave** per r se è una **superchiave minimale**, ossia:
   - K è una superchiave per r.
   - Nessun sottoinsieme proprio $K' \subset K$ è una superchiave per r (eliminando anche un solo attributo da K, la proprietà di unicità decade).

Esempio Dettagliato: Ricerca delle Chiavi. Si consideri la seguente istanza della relazione **STUDENTI**:

Matricola	Cognome	Nome	CorsoDiStudi	DataNascita
27655	Rossi	Mario	Ingegneria Informatica	1978/12/05
78763	Rossi	Mario	Ingegneria Informatica	1976/11/03
65432	Neri	Piero	Ingegneria Meccanica	1979/07/10
87654	Neri	Mario	Ingegneria Informatica	1976/11/03
67653	Rossi	Piero	Ingegneria Meccanica	1978/12/05

Analizziamo le possibili combinazioni di attributi:

- $K_1 = \{\text{Matricola}\}$: ciascuna tupla possiede un numero di matricola univoco. È una superchiave ed è composta da un singolo attributo, quindi è **minimale per definizione**: K_1 è una **chiave**.
- $K_2 = \{\text{Cognome}, \text{Nome}, \text{DataNascita}\}$: non esistono due studenti con stesso cognome, stesso nome e medesima data di nascita. È una superchiave. Inoltre:
  - $\{\text{Cognome}, \text{Nome}\}$ non basta (ci sono due *Mario Rossi*).
  - $\{\text{Cognome}, \text{DataNascita}\}$ non basta (ci sono due *Rossi* nati il 1978/12/05).
  - $\{\text{Nome}, \text{DataNascita}\}$ non basta (ci sono due *Mario* nati il 1976/11/03).

  Poiché nessun sottoinsieme proprio identifica le tuple, K_2 è minimale ed è una **seconda chiave candidata**.

- $K_3 = \{\text{Matricola}, \text{Cognome}\}$: identifica univocamente le tuple, ma contiene la chiave $\{\text{Matricola}\}$ come sottoinsieme proprio. È una **superchiave non minimale**, quindi non è una chiave.

Schemi, Vincoli e Stati: Il Rischio delle Chiavi Accidentali. Si consideri la combinazione $K_4 = \{\text{Cognome}, \text{CorsoDiStudi}\}$. Nell'istanza specifica mostrata nella tabella precedente, per pura coincidenza, non figurano due tuple uguali su questa coppia di attributi. Possiamo affermare che $\{\text{Cognome}, \text{CorsoDiStudi}\}$ sia una chiave?

Vincoli a Livello di Schema vs Proprietà dello Stato

La risposta è tassativamente **no**:

- I vincoli di integrità riflettono **proprietà del mondo reale modellato** e sono definiti a livello di **schema**.
- Un vincolo deve essere soddisfatto da **tutte le istanze legali ammissibili**, presenti e future, non soltanto dai dati momentaneamente inseriti.
- L'unicità di $\{\text{Cognome}, \text{CorsoDiStudi}\}$ è una coincidenza dello stato attuale (*by chance*): nel mondo reale nulla impedisce a due fratelli o omonimi con il cognome *Rossi* di iscriversi entrambi a *Ingegneria Informatica*.

$\Longrightarrow$ Le chiavi non si desumono mai guardando i dati di una tabella campione, ma unicamente dall'analisi dei requisiti e della semantica del dominio.

Il Teorema di Esistenza della Chiave. Poiché per definizione assiomatica del modello relazionale una relazione è un **insieme di tuple distinte**, ne consegue una proprietà teorica di fondamentale importanza:

1. L'insieme formato da **tutti gli attributi della relazione** $X = \{A_1, A_2, \ldots, A_n\}$ garantisce l'assenza di tuple identiche ed è perciò sempre una **superchiave**.
2. Poiché l'insieme degli attributi è finito, esso deve contenere almeno un sottoinsieme minimale (che nel caso limite peggiore coincide con l'intero insieme X).

$\Longrightarrow$ **Ogni relazione possiede sempre almeno una chiave**.

L'Importanza Sistemistica delle Chiavi. L'esistenza garantita di almeno una chiave assicura due requisiti cardine:

- **Accessibilità univoca**: qualunque elemento informativo all'interno della base di dati può essere reperito in modo deterministico specificando il nome della relazione, il nome dell'attributo e il valore della chiave.
- **Fondamento delle relazioni per valore**: le chiavi costituiscono l'ancora logica per connettere relazioni distinte senza ambiguità.

2.2.4 Chiavi, Valori NULL e la Chiave Primaria

L'Incompatibilità tra Valori NULL e Chiavi. Se gli attributi appartenenti a una chiave contenessero valori NULL, la funzione stessa della chiave verrebbe irrimediabilmente compromessa:

- **Perdita di identificazione**: due tuple con un valore NULL sulla chiave non possono essere confrontate (NULL non è uguale a NULL nella logica a tre valori), rendendo impossibile verificare se rappresentino la stessa entità o entità distinte.
- **Impossibilità di referenziazione**: un'altra tabella che desideri puntare a quella specifica riga non avrebbe un valore concreto da registrare nella propria colonna di collegamento.

Esempio: La Corruzione dell'Identità delle Tuple con Valori NULL

Si consideri una tabella di studenti con valori NULL distribuiti sulle possibili chiavi:

Matricola	Cognome	Nome	CorsoDiStudi	DataNascita
NULL	**NULL**	Mario	Ing. Informatica	1978/12/05
78763	Rossi	Mario	Ingegneria Civile	1976/11/03
65432	Neri	Piero	Ingegneria Meccanica	1979/07/10
87654	Neri	Mario	Ing. Informatica	**NULL**
NULL	Neri	Mario	Ing. Informatica	**NULL**

Nell'ultima riga, la matricola è ignota e la data di nascita è assente: né {Matricola} né {Cognome, Nome, DataNascita} possono essere utilizzate per distinguere questa persona dagli altri iscritti o per verbalizzare un esame sostenuto.

La Scelta della Chiave Primaria (*Primary Key*). Tra tutte le chiavi candidate individuate su una relazione, il progettista deve selezionarne una specifica che assuma il ruolo di **chiave primaria**:

- La chiave primaria funge da identificatore formale, privilegiato e immutabile delle tuple della tabella.
- Tutte le altre chiavi candidate non prescelte prendono il nome di **chiavi alternative** o secondarie (garantite dal vincolo `UNIQUE` in SQL).
- Si prediligono chiavi compatte (composte da pochi attributi), stabili nel tempo (i cui valori non cambiano mai) e preferibilmente numeriche.

Vincolo di Integrità dell'Entità (*Entity Integrity Constraint*)

Regola Aurea: Gli attributi che compongono la **chiave primaria** di una relazione **non possono mai assumere il valore NULL** (*NOT NULL implicito*).

Convenzione Grafica e Notazionale:
Negli schemi testuali e nelle tabelle relazionali, la chiave primaria viene evidenziata applicando una **sottolineatura** agli attributi che la compongono:

STUDENTI(Matricola, Cognome, Nome, CorsoDiStudi, DataNascita)

2.2.5 Integrità Referenziale e Chiavi Esterne (*Foreign Key*)

Il Collegamento Inter-relazionale per Valore. Nel modello relazionale, informazioni memorizzate in tabelle distinte vengono poste in correlazione mediante la sovrapposizione di valori condivisi. In particolare, una tabella referenziante include tra i propri attributi i valori della chiave primaria della tabella referenziata.

Affinché la base di dati resti affidabile, tali collegamenti devono essere **coerenti**: non deve mai essere possibile fare riferimento a un'entità inesistente o cancellata.

Definizione Formale di Vincolo di Integrità Referenziale

Siano date due relazioni R_1 e R_2 con rispettivi schemi. Un **vincolo di integrità referenziale** (o **chiave esterna** / *foreign key*) tra un insieme di attributi X di R_1 (detta **relazione referenziante**) e la chiave primaria K di R_2 (detta **relazione referenziata**) impone che:

- Per ogni tupla $t_1 \in r_1$, se il valore di $t_1[X]$ è interamente valorizzato (non contiene NULL), allora deve esistere una tupla $t_2 \in r_2$ tale che:

$$t_1[X] = t_2[K]$$

- In notazione insiemistica di algebra relazionale:

$$\pi_X(r_1) \setminus \{\text{NULL}\} \subseteq \pi_K(r_2)$$

Esempio Completo: Il Sistema di Verbalizzazione delle Infrazioni. Si consideri il sistema di gestione del comando di polizia locale, modellato su tre relazioni:

Relazione INFRAZIONI

Codice	Data	Vigile	Stato	Targa
34321	1995/02/01	3987	Italia	MI39548K
53524	1995/03/04	3295	Italia	AJ046EZ
64521	1996/04/05	3295	Francia	ET234RY
73321	1998/02/05	9345	Finlandia	MMG-418

Relazione VIGILI

Matricola	Cognome	Nome
3987	Rossi	Luca
3295	Neri	Piero
9345	Neri	Mario
7543	Mori	Gino

Relazione AUTO

Stato	Targa	Cognome	Nome
Italia	MI39548K	Rossi	Mario
Italia	AJ046EZ	Verdi	Giuseppe
Francia	ET234RY	Debussy	Claude
Finlandia	MMG-418	Sibelius	Jean

Sulla tabella **INFRAZIONI** insistono due distinti vincoli di integrità referenziale:

1. L'attributo singolo `Vigile` referenzia la chiave primaria della tabella **VIGILI**:

$$\text{INFRAZIONI[Vigile]} \xrightarrow{\text{FK}} \text{VIGILI[Matricola]}$$

2. La coppia di attributi {Stato, Targa} referenzia la chiave primaria composita della tabella **AUTO**:

$$\text{INFRAZIONI[Stato, Targa]} \xrightarrow{\text{FK}} \text{AUTO[Stato, Targa]}$$

Violazione dell'Integrità Referenziale. Una violazione del vincolo si verificherebbe qualora venisse inserita nella tabella **INFRAZIONI** una tupla con:

- `Vigile` = 8888, ma tale matricola non compare in **VIGILI**.
- `Stato` = 'Italia', `Targa` = 'XX000YY', ma tale veicolo non risulta registrato nella tabella **AUTO**.

In entrambi i casi, l'infrazione punterebbe al vuoto (*dangling reference*), compromettendo la qualità e l'affidabilità della base di dati.

2.2.6 Politiche di Reazione alle Violazioni e Vincoli Multipli

Integrità Referenziale e Valori NULL. Il vincolo di chiave esterna non impone necessariamente che il riferimento esista: ammette che l'attributo assuma il valore NULL, a patto che la colonna non sia marcata con il vincolo aggiuntivo `NOT NULL`.

Esempio: Partecipazione Opzionale a Progetti

Si consideri una base di dati aziendale composta dalle relazioni:

DIPENDENTI(Matricola, Cognome, Progetto)
PROGETTI(Codice, DataInizio, DurataMesi, CostoMilioni)

Matricola	Cognome	Progetto
34321	Rossi	IDEA
53524	Neri	XYZ
64521	Verdi	**NULL**
73032	Bianchi	IDEA

Codice	DataInizio	Mesi	Costo
IDEA	2000/01	36	200
XYZ	2001/07	24	120
BOH	2001/09	24	150

La presenza di **NULL** sul dipendente *Verdi* è lecita: significa semanticamente che il dipendente è momentaneamente senza allocazione progettuale. Per i dipendenti che hanno un valore specificato, invece, il codice deve esistere tassativamente nella tabella **PROGETTI**.

Gestione degli Effetti Collaterali: Azioni Compensative. Quando un utente o un programma richiede la **cancellazione** o la **modifica della chiave** di una tupla nella tabella referenziata (es. eliminare il progetto `XYZ` da **PROGETTI**), le tuple della tabella referenziante (**DIPENDENTI**) rimarrebbero con una chiave esterna orfana, violando il vincolo.

I DBMS commerciali permettono di gestire l'anomalia configurando quattro distinte **azioni compensative**:

1. **Rifiuto / Blocco (*Restrict / No Action*)**: il DBMS impedisce tassativamente l'operazione di cancellazione o aggiornamento, segnalando un errore applicativo fintanto che esistono record figli collegati. È il comportamento predefinito di sicurezza.

2. **Cancellazione a Cascata (*Cascade*)**: l'eliminazione della tupla genitore provoca l'**eliminazione a catena** automatica di tutte le tuple figlie collegate nella tabella referenziante. Se si cancella il progetto `XYZ`, vengono automaticamente eliminati anche tutti i dipendenti assegnati a quel progetto.

3. **Assegnazione a NULL (*Set NULL*)**: la tupla genitore viene regolarmente eliminata; nei record referenzianti collegati, il valore della chiave esterna viene commutato automaticamente al marcatore speciale NULL (il dipendente *Neri* non viene licenziato, ma la sua colonna `Progetto` diventa NULL).

4. **Assegnazione al Default (*Set Default*)**: il campo della chiave esterna nelle tuple figlie viene reimpostato su un valore predefinito valido specificato nello schema (es. un progetto jolly aziendale generico).

Vincoli Referenziali su Attributi Multipli e Corrispondenza Posizionale. Una relazione può possedere più chiavi esterne indipendenti rivolte verso la medesima tabella di destinazione. Si consideri la relazione **INCIDENTI** che descrive sinistri stradali tra due autovetture:

Relazione INCIDENTI

Codice	**StatoA**	**TargaA**	**StatoB**	**TargaB**
34321	Italia	AJ046EZ	Italia	MI39548K
53524	Finlandia	MMG-418	Francia	ET234RY

In questa tabella sussistono due vincoli distinti verso la tabella AUTO(Stato, Targa):

$$\mathcal{C}_1: \quad (\text{StatoA}, \text{TargaA}) \xrightarrow{\text{FK}} \text{AUTO}(\text{Stato}, \text{Targa})$$
$$\mathcal{C}_2: \quad (\text{StatoB}, \text{TargaB}) \xrightarrow{\text{FK}} \text{AUTO}(\text{Stato}, \text{Targa})$$

La Rigidità dell'Ordinamento Posizionale nella Definizione dei Vincoli

Quando un vincolo referenziale è definito su una chiave composita formata da più attributi $(A_1, A_2, \ldots, A_m) \to (K_1, K_2, \ldots, K_m)$:

- L'**ordine posizionale** degli attributi specificato nella clausola è determinante: l'attributo in posizione i della chiave esterna viene confrontato esclusivamente con l'attributo in posizione i della chiave primaria di destinazione.
- Scambiare l'ordine (es. definire erroneamente $(\text{TargaA}, \text{StatoA}) \to (\text{Stato}, \text{Targa})$) genera un errore immediato di tipo di dato incompatibile oppure tenta di abbinare una targa a uno stato geografico, corrompendo l'integrità logica del modello.

3 Linguaggi Formali di Interrogazione

3.1 Algebra Relazionale

3.1.1 Linguaggi per Interrogare una Base di Dati

Su una base di dati si fanno due tipi di operazioni:

- **operazioni sullo schema**, con il **DDL** (*Data Definition Language*): servono a creare o modificare la struttura delle tabelle;
- **operazioni sui dati**, con il **DML** (*Data Manipulation Language*): comprendono le **interrogazioni** (*query*, cioè leggere i dati) e gli **aggiornamenti** (inserire nuovi dati o modificare quelli esistenti).

I linguaggi di interrogazione si dividono in due famiglie:

- **Dichiarativi**: dicono **cosa** vogliamo ottenere, cioè le proprietà del risultato.
- **Procedurali**: dicono **come** ottenere il risultato, passo dopo passo.

I principali linguaggi per le basi di dati relazionali sono:

- **Algebra relazionale**: procedurale;
- **Calcolo relazionale**: dichiarativo (teorico, non implementato nei DBMS);
- **SQL** (*Structured Query Language*): parzialmente dichiarativo, è quello usato davvero;
- **QBE** (*Query By Example*): dichiarativo, implementato.

L'**algebra relazionale** è definita da un insieme di **operatori** che:

- lavorano su relazioni;
- producono come risultato **altre relazioni**;
- possono quindi essere **composti** tra loro (l'output di uno diventa l'input di un altro).

Gli operatori che vedremo sono: unione, intersezione, differenza, ridenominazione, selezione, proiezione e join (naturale, prodotto cartesiano, theta-join).

3.1.2 Operatori Insiemistici e Ridenominazione

Le relazioni sono **insiemi** di tuple, e anche i risultati devono essere insiemi. Possiamo quindi usare le classiche operazioni sugli insiemi: **unione** ($\cup$), **intersezione** ($\cap$) e **differenza** ($-$). C'è però un vincolo: le due relazioni coinvolte devono avere **lo stesso schema** (stessi attributi).

Prendiamo come esempio le relazioni **Laureati** e **Tecnici**, entrambe con schema (Matricola, Nome, Età):

Laureati		
Matricola	**Nome**	**Età**
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Tecnici		
Matricola	**Nome**	**Età**
9297	Neri	33
7432	Neri	54
9824	Verdi	45

- **Laureati** $\cup$ **Tecnici**: tutte le tuple che stanno in almeno una delle due (7274, 7432, 9824, 9297). Le tuple comuni compaiono una sola volta.
- **Laureati** $\cap$ **Tecnici**: solo le tuple presenti in entrambe (7432 Neri, 9824 Verdi).
- **Laureati** $-$ **Tecnici**: le tuple dei laureati che non sono tecnici (solo 7274 Rossi).

Il problema degli schemi diversi. Consideriamo Paternità(Padre, Figlio) e Maternità(Madre, Figlio). Vorremmo unirle per ottenere tutte le coppie genitore–figlio, ma l'unione non è ammessa perché gli schemi sono diversi ($Padre \neq Madre$). Ha senso, ma non si può fare direttamente: serve la ridenominazione.

Ridenominazione (ρ). È un operatore **unario** (ha un solo argomento) che **cambia il nome degli attributi** lasciando i dati invariati:

$$\rho_{\text{NuovoNome}\leftarrow\text{VecchioNome}}(\text{Relazione})$$

Con la ridenominazione il problema di prima si risolve:

$$\rho_{\text{Genitore}\leftarrow\text{Padre}}(\text{Paternità}) \ \cup \ \rho_{\text{Genitore}\leftarrow\text{Madre}}(\text{Maternità})$$

Ora entrambe le relazioni hanno schema (Genitore, Figlio) e l'unione è lecita.

Si possono rinominare più attributi alla volta, rispettando l'ordine posizionale. Ad esempio, date Impiegati(Cognome, Ufficio, Stipendio) e Operai(Cognome, Fabbrica, Salario):

$$\rho_{\text{Sede,Paga}\leftarrow\text{Ufficio,Stipendio}}(\text{Impiegati}) \ \cup \ \rho_{\text{Sede,Paga}\leftarrow\text{Fabbrica,Salario}}(\text{Operai})$$

produce un'unica relazione con schema (Cognome, Sede, Paga).

3.1.3 Selezione e Proiezione

Selezione (σ). Operatore unario che **filtra le righe**:

$$\sigma_{\text{Predicato}}(\text{Relazione})$$

- il risultato ha **lo stesso schema** dell'input;
- contiene solo le tuple che **soddisfano il predicato**, cioè un sottoinsieme delle tuple di partenza;
- il predicato è un'espressione booleana sugli attributi (confronti con $=, \neq, <, >, \leq, \geq$, combinati con AND, OR, NOT).

Esempio, sulla relazione **Impiegati**:

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Milano	44
5698	Neri	Napoli	64

- Impiegati che guadagnano più di 50: $\sigma_{\text{Stipendio}>50}(\text{Impiegati}) \Rightarrow$ 7309, 5998, 5698.
- Che guadagnano più di 50 **e** lavorano a Milano: $\sigma_{\text{Stipendio}>50 \wedge \text{Filiale}=\text{'Milano'}}(\text{Impiegati}) \Rightarrow$ solo 5998.
- Che hanno cognome uguale alla città della filiale: $\sigma_{\text{Cognome}=\text{Filiale}}(\text{Impiegati}) \Rightarrow$ solo 9553. Qui si confrontano due attributi della stessa tupla, non un attributo con una costante.

Proiezione (π). Operatore unario che **tiene solo alcune colonne**:

$$\pi_{\text{ListaAttributi}}(\text{Relazione})$$

- lo schema del risultato è un **sottoinsieme** di quello di partenza;
- vengono considerate **tutte** le tuple dell'input, ma solo sugli attributi indicati.

Supponiamo ora che Impiegati contenga: (7309, Neri, Napoli, 55), (5998, Neri, Milano, 64), (9553, Rossi, Roma, 44), (5698, Rossi, Roma, 64).

- $\pi_{\text{Matricola,Cognome}}(\text{Impiegati})$ restituisce 4 tuple, una per impiegato.
- $\pi_{\text{Cognome,Filiale}}(\text{Impiegati})$ restituisce solo **3** tuple: (Neri, Napoli), (Neri, Milano), (Rossi, Roma). La coppia (Rossi, Roma) comparirebbe due volte, ma una relazione è un insieme e i duplicati vengono eliminati.

Cardinalità della Proiezione

Il risultato di una proiezione ha **al più** tante tuple quante l'input, e può averne meno perché le tuple ripetute vengono scartate.

Se la lista di attributi X è una **superchiave** di R, allora $\pi_X(R)$ ha **esattamente** lo stesso numero di tuple di R: i valori su X sono tutti distinti, quindi non possono nascere duplicati.

Selezione e proiezione sono operatori **ortogonali**: la selezione taglia la tabella **orizzontalmente** (toglie righe), la proiezione **verticalmente** (toglie colonne). Combinandoli si estrae solo l'informazione che interessa. Ad esempio, matricola e cognome degli impiegati che guadagnano più di 50:

$$\pi_{\text{Matricola,Cognome}}\big(\sigma_{\text{Stipendio}>50}(\text{Impiegati})\big)$$

Limite. Con selezione e proiezione si lavora su **una sola relazione**: non si possono mettere in relazione tuple di tabelle diverse, né tuple diverse della stessa tabella. Per questo serve il join.

3.1.4 Join

Il **join** è l'operatore più importante dell'algebra relazionale: permette di **collegare tuple di relazioni diverse**.

Un esempio intuitivo. In un concorso pubblico ogni compito è anonimo e ha associata una busta chiusa con il nome del candidato; compito e busta hanno lo stesso numero identificativo. Abbiamo quindi due tabelle, (ID, Voto) e (ID, Candidato). Per sapere il voto di ciascun candidato basta accoppiare le righe con lo stesso ID, ottenendo (ID, Candidato, Voto). Questo è proprio un join.

Join naturale ($\bowtie$). È un operatore binario (si estende a più relazioni perché è associativo). Il risultato:

- ha come schema l'**unione degli attributi** delle due relazioni;
- contiene le tuple ottenute combinando una tupla della prima e una della seconda che hanno **gli stessi valori sugli attributi in comune**.

Formalmente, date $R_1(X_1)$ e $R_2(X_2)$:

$$R_1 \bowtie R_2 = \{\, t \text{ su } X_1X_2 \mid \exists t_1 \in R_1,\ t_2 \in R_2 \text{ con } t[X_1] = t_1 \text{ e } t[X_2] = t_2 \,\}$$

Esempio, con Impiegati(Impiegato, Reparto) e Reparti(Reparto, Capo):

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

$\bowtie$

Reparto	Capo
A	Mori
B	Bruni

$=$

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	B	Bruni
Bianchi	B	Bruni

Qui ogni tupla di entrambe le relazioni contribuisce al risultato: si parla di join **completo**. Non è sempre così:

- **Join non completo**: se Reparti contiene solo B e C, la tupla di Rossi (reparto A) e quella del reparto C non trovano corrispondenza e vengono **escluse** dal risultato.
- **Join vuoto**: se nessun valore di Reparto è in comune, il risultato non ha tuple (ma ha comunque lo schema Impiegato, Reparto, Capo).
- **Join con $n \times m$ tuple**: se tutte le tuple hanno lo stesso valore (es. tutti nel reparto B, e B compare due volte in Reparti), ogni tupla si combina con tutte le altre.

Quante tuple produce un join?

In generale:

$$0 \leq |R_1 \bowtie R_2| \leq |R_1| \times |R_2|$$

- Se il join coinvolge una **chiave** di R_2, ogni tupla di R_1 può combinarsi con **al più una** tupla di R_2 (i valori della chiave sono unici), quindi:

$$0 \leq |R_1 \bowtie R_2| \leq |R_1|$$

- Se in più c'è un **vincolo di integrità referenziale** da R_1 verso quella chiave di R_2, ogni tupla di R_1 trova **esattamente una** corrispondenza, quindi:

$$|R_1 \bowtie R_2| = |R_1|$$

Outer join. Il join "normale" (detto **inner join**) scarta le tuple senza corrispondenza, e quindi si perde informazione. L'**outer join** invece le mantiene, completandole con valori **NULL**. Ne esistono tre versioni:

- **Left** ($R_1 \mathbin{⟕} R_2$): mantiene tutte le tuple del **primo** operando; dove manca la corrispondenza a destra mette NULL.
- **Right** ($R_1 \mathbin{⟖} R_2$): mantiene tutte le tuple del **secondo** operando. È definito come $R_2 \mathbin{⟕} R_1$.
- **Full** ($R_1 \mathbin{⟗} R_2$): mantiene tutte le tuple di **entrambi**; è l'unione dei due precedenti.

Con Impiegati = {(Rossi, A), (Neri, B), (Bianchi, B)} e Reparti = {(B, Mori), (C, Bruni)}:

Full outer join		
Impiegato	**Reparto**	**Capo**
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni

Il left outer join contiene le prime tre righe, il right outer join la prima, la seconda e la quarta.

Prodotto cartesiano. È un join naturale tra due relazioni **senza attributi in comune**: non c'è nessuna condizione da rispettare, quindi ogni tupla della prima si combina con ogni tupla della seconda. Il risultato ha esattamente $|R_1| \times |R_2|$ tuple. Ad esempio, Impiegati(Impiegato, Reparto) $\bowtie$ Reparti(Codice, Capo) dà $3 \times 2 = 6$ tuple, la maggior parte senza senso (Rossi del reparto A abbinato al capo del reparto B...). Per questo il prodotto cartesiano è utile solo se seguito da una selezione:

$$\sigma_{\text{condizione}}(R_1 \bowtie R_2)$$

Theta-join. La combinazione "prodotto cartesiano + selezione" è così comune da avere un nome e un simbolo propri:

$$R_1 \bowtie_{\text{condizione}} R_2 \;=\; \sigma_{\text{condizione}}(R_1 \bowtie R_2)$$

La condizione è di solito una congiunzione (AND) di confronti del tipo $A_1 \,\theta\, A_2$, dove θ è un operatore di confronto ($=, >, <, \dots$) e A_1, A_2 sono attributi delle due relazioni.

Equi-join. È un theta-join in cui tutti i confronti sono **uguaglianze**. È molto più usato del theta-join generico perché permette di dire su quali attributi fare il join senza doverli rinominare:

$$\text{Impiegati} \bowtie_{\text{Reparto=Codice}} \text{Reparti}$$

Il risultato ha schema (Impiegato, Reparto, Codice, Capo), con Reparto e Codice sempre uguali. Si ottiene quasi lo stesso risultato con un join naturale dopo aver rinominato l'attributo:

$$\text{Impiegati} \bowtie \rho_{\text{Reparto}\leftarrow\text{Codice}}(\text{Reparti})$$

che però ha una colonna in meno (Reparto compare una sola volta).

3.1.5 Esempi di Interrogazioni

Consideriamo le relazioni:

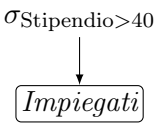
Impiegati			
Matricola	**Nome**	**Età**	**Stipendio**
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

Supervisione	
Impiegato	**Capo**
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

In Supervisione sia Impiegato sia Capo sono matricole di Impiegati.

1. **Tutti i dati degli impiegati che guadagnano più di 40:**

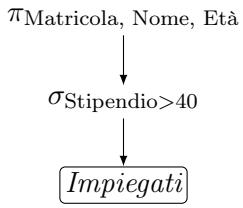
$$\sigma_{\text{Stipendio}>40}(\text{Impiegati})$$



Matricola	**Nome**	**Età**	**Stipendio**
7309	Rossi	34	45
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

2. **Matricola, nome ed età degli impiegati che guadagnano più di 40:**

$$\pi_{\text{Matricola,Nome,Età}}\big(\sigma_{\text{Stipendio}>40}(\text{Impiegati})\big)$$

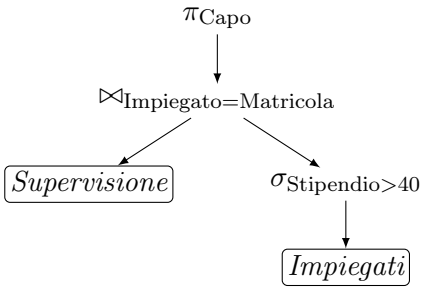


Matricola	**Nome**	**Età**
7309	Rossi	34
5698	Bruni	43
4076	Mori	45
8123	Lupi	46

3. **I capi di impiegati che guadagnano più di 40:** si selezionano gli impiegati che guadagnano più di 40, si collegano a Supervisione tramite la matricola e si tiene solo il capo.

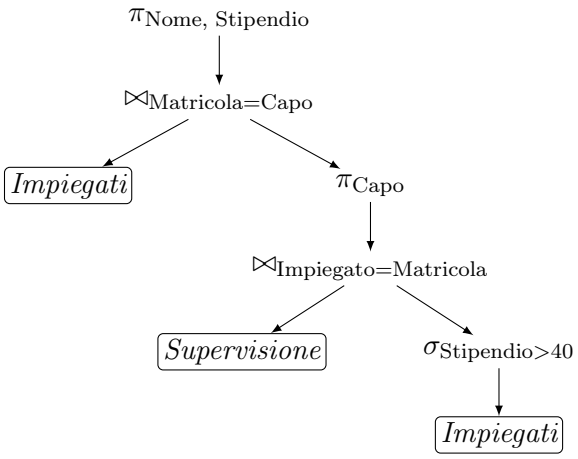
$$\pi_{\text{Capo}}\big(\text{Supervisione} \bowtie_{\text{Impiegato=Matricola}} \sigma_{\text{Stipendio}>40}(\text{Impiegati})\big)$$

Capo
5698
4076
8123



4. **Nome e stipendio di quei capi:** serve un secondo join con Impiegati, questa volta sulla matricola del capo.

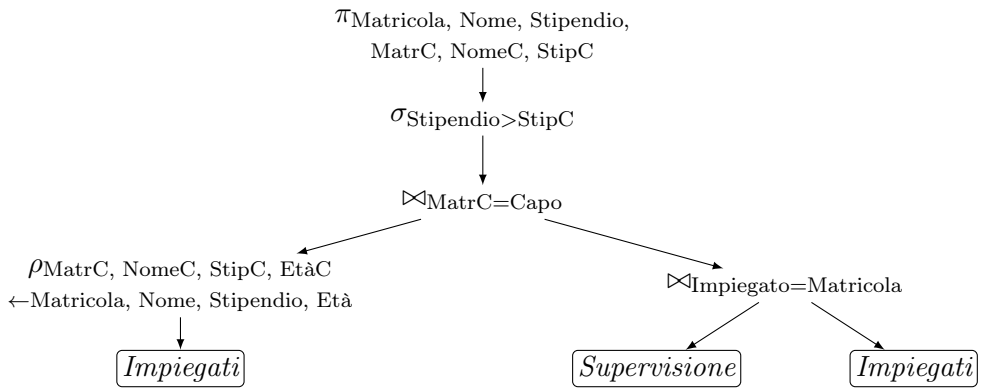
$$\pi_{\text{Nome,Stipendio}}\Big(\text{Impiegati} \bowtie_{\text{Matricola=Capo}} \pi_{\text{Capo}}\big(\text{Supervisione} \bowtie_{\text{Impiegato=Matricola}} \sigma_{\text{Stipendio>40}}(\text{Impiegati})\big)\Big)$$



Nome	Stipendio
Bruni	42
Mori	50
Lupi	60

5. **Impiegati che guadagnano più del proprio capo**, con matricola, nome e stipendio di entrambi. Bisogna usare Impiegati **due volte**: una per l'impiegato e una per il capo. Per non confondere gli attributi, la seconda copia viene rinominata (MatrC, NomeC, StipC, EtàC):

$$\pi_{\text{Matricola,Nome,Stipendio,MatrC,NomeC,StipC}}\Big(\sigma_{\text{Stipendio>StipC}}\Big(\rho_{\substack{\text{MatrC,NomeC,StipC,EtàC}\\ \leftarrow\text{Matricola,Nome,Stipendio,Età}}}(\text{Impiegati}) \bowtie_{\text{MatrC=Capo}} (\text{Supervisione} \bowtie_{\text{Impiegato=Matricola}} \text{Impiegati})\Big)\Big)$$



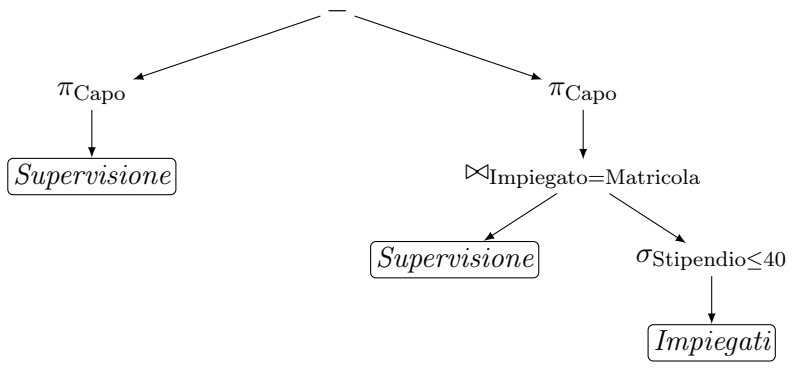
Matricola	Nome	Stipendio	MatrC	NomeC	StipC
7309	Rossi	45	5698	Bruni	42

Gli altri impiegati guadagnano meno del capo (o quanto lui): ad esempio Bianchi (38) ha come capo Bruni (42).

6. **Matricole dei capi i cui impiegati guadagnano <u>tutti</u> più di 40.** Il "tutti" non si esprime direttamente: si prendono tutti i capi e si **tolgono** quelli che hanno almeno un impiegato che guadagna 40 o meno.

$$\pi_{\text{Capo}}(\text{Supervisione}) - \pi_{\text{Capo}}\big(\text{Supervisione} \bowtie_{\text{Impiegato=Matricola}} \sigma_{\text{Stipendio}\leq 40}(\text{Impiegati})\big)$$

Questo schema ("tutti" = "tutti meno quelli che violano la condizione") con la differenza è molto ricorrente.



Passo per passo:

π_{Capo}(Supervisione)
5698
4076
8123

$-$

capi con un impiegato ≤ 40	
5698	(Bianchi, 38)
4076	(Neri, 35)

$=$

Capo
8123

3.1.6 Equivalenza di Espressioni

Due espressioni dell'algebra relazionale E_1 ed E_2 sono **equivalenti** se producono lo stesso risultato **qualunque sia lo stato** della base di dati. Le regole di equivalenza sono importanti perché i DBMS le usano per riscrivere le query in forme equivalenti ma **più efficienti**.

Le principali regole (con A, B attributi e X, Y insiemi di attributi):

1. **Atomizzazione delle selezioni**: una cascata di selezioni equivale a una selezione con AND.
$$\sigma_{A>20}\big(\sigma_{B=\text{'Rossi'}}(R)\big) = \sigma_{A>20 \wedge B=\text{'Rossi'}}(R)$$
2. **Idempotenza della proiezione**: proiettare prima su più attributi e poi su meno non cambia nulla.
$$\pi_X(R) = \pi_X\big(\pi_{XY}(R)\big)$$
3. **Anticipazione della selezione** rispetto al join (se A è un attributo di R_2):
$$\sigma_{A=10}(R_1 \bowtie R_2) = R_1 \bowtie \sigma_{A=10}(R_2)$$
È la regola più utile: filtrando **prima** del join si riducono molto i risultati intermedi, e quindi il costo.
4. **Anticipazione della proiezione**: se R_1 ha schema X_1, R_2 ha schema X_2 e $Y_2 \subseteq X_2$ contiene tutti gli attributi comuni ($X_1 \cap X_2 \subseteq Y_2$), gli attributi di $X_2 - Y_2$ non servono al join e si possono eliminare subito:
$$\pi_{X_1 Y_2}(R_1 \bowtie R_2) = R_1 \bowtie \pi_{Y_2}(R_2)$$
5. **Inglobamento della selezione nel theta-join**:
$$\sigma_{A=10}(R_1 \bowtie R_2) = R_1 \bowtie_{A=10} R_2$$
6. **Proprietà distributive**:
$$\sigma_{A=10}(R_1 \cup R_2) = \sigma_{A=10}(R_1) \cup \sigma_{A=10}(R_2) \qquad \sigma_{A=10}(R_1 - R_2) = \sigma_{A=10}(R_1) - \sigma_{A=10}(R_2)$$
$$\pi_X(R_1 \cup R_2) = \pi_X(R_1) \cup \pi_X(R_2) \qquad R \bowtie (R_1 \cup R_2) = (R \bowtie R_1) \cup (R \bowtie R_2)$$
Attenzione: la proiezione **non** è distributiva rispetto alla differenza.
7. **Selezioni e operatori insiemistici**:
$$\sigma_{A=10 \vee B=\text{'Rossi'}}(R) = \sigma_{A=10}(R) \cup \sigma_{B=\text{'Rossi'}}(R) \qquad \sigma_{A=10 \wedge B=\text{'Rossi'}}(R) = \sigma_{A=10}(R) \cap \sigma_{B=\text{'Rossi'}}(R)$$
$$\sigma_{A=10 \wedge \neg(B=\text{'Rossi'})}(R) = \sigma_{A=10}(R) - \sigma_{B=\text{'Rossi'}}(R)$$

Esempio di ottimizzazione. Capi degli impiegati con meno di 30 anni:

$$\pi_{\text{Capo}}\big(\sigma_{\text{Matricola=Impiegato} \wedge \text{Età<30}}(\text{Impiegati} \bowtie \text{Supervisione})\big)$$

Con la regola 1 si spezza la selezione; con la 5 la condizione Matricola = Impiegato diventa un theta-join; con la 3 la selezione sull'età viene anticipata; con la 4 si tiene solo la matricola prima del join:

$$\pi_{\text{Capo}}\Big(\pi_{\text{Matricola}}\big(\sigma_{\text{Età<30}}(\text{Impiegati})\big) \bowtie_{\text{Matricola=Impiegato}} \text{Supervisione}\Big)$$

Nel corso non è richiesto scrivere query efficienti (meglio corrette e chiare): l'ottimizzazione la fa il DBMS.

3.1.7 Selezione e Valori NULL

Se un attributo vale NULL, una condizione atomica su di esso (es. Età > 40) **non è soddisfatta**: la tupla viene esclusa. Questo porta a risultati poco intuitivi:

$$\sigma_{\text{Età}>30}(\text{Persone}) \cup \sigma_{\text{Età}\leq 30}(\text{Persone}) \neq \text{Persone} \qquad \sigma_{\text{Età}>30 \vee \text{Età}\leq 30}(\text{Persone}) \neq \text{Persone}$$

perché le persone con età NULL non rientrano in nessuna delle due condizioni.

Per gestire i NULL esistono due condizioni atomiche apposite: **IS NULL** e **IS NOT NULL**. Così:

$$\sigma_{\text{Età}>30 \vee \text{Età}\leq 30 \vee \text{Età IS NULL}}(\text{Persone}) = \text{Persone}$$

e, ad esempio, $\sigma_{(\text{Età}>40) \vee (\text{Età IS NULL})}(\text{Impiegati})$ restituisce sia chi ha più di 40 anni sia chi ha età sconosciuta. Si potrebbe definire una logica a tre valori (vero, falso, *sconosciuto*), ma qui non è necessario.

3.1.8 Viste

Le **viste** sono rappresentazioni diverse degli stessi dati. Si distinguono:

- **relazioni di base**: hanno un contenuto proprio, memorizzato;
- **relazioni derivate** (viste): il loro contenuto è definito da una **query** sulle altre relazioni (anche su altre viste).

Le viste corrispondono agli **schemi esterni** dell'architettura ANSI/SPARC: ogni gruppo di utenti vede la base di dati attraverso le proprie viste, costruite sopra lo schema logico.

Esempio, date Afferenza(Impiegato, Reparto) e Direzione(Reparto, Capo):

$$\text{Supervisione} = \pi_{\text{Impiegato,Capo}}(\text{Afferenza} \bowtie \text{Direzione})$$

Una vista può essere realizzata in due modi:

- **Vista materializzata**: il risultato viene salvato fisicamente. È subito disponibile per altre query, ma è ridondante, rallenta gli aggiornamenti (va tenuta allineata) ed è supportata da pochi DBMS.
- **Relazione virtuale**: viene salvata solo la definizione. Supportata da tutti i DBMS: quando una query usa la vista, il suo nome viene **sostituito dalla query che la definisce**. Ad esempio $\sigma_{\text{Capo='Leoni'}}(\text{Supervisione})$ viene eseguita come $\sigma_{\text{Capo='Leoni'}}\big(\pi_{\text{Impiegato,Capo}}(\text{Afferenza} \bowtie \text{Direzione})\big)$.

Perché usare le viste.

- Ogni utente vede solo ciò che gli interessa, nel formato che preferisce, e solo ciò che è **autorizzato** a vedere.
- Semplificano la scrittura di query complesse in cui una sotto-espressione si ripete. Ad esempio, gli impiegati che hanno lo stesso capo di Rossi:

  $$\pi_{\text{Impiegato}}\Big(\text{Supervisione} \bowtie \rho_{\text{ImpR}\leftarrow\text{Impiegato}}\big(\sigma_{\text{Impiegato='Rossi'}}(\text{Supervisione})\big)\Big)$$

  Senza vista bisognerebbe scrivere due volte Afferenza $\bowtie$ Direzione.

- Permettono di usare programmi già esistenti su schemi ristrutturati.

Le viste però **non migliorano l'efficienza** delle query.

Aggiornare una vista. Significa modificare le relazioni di base in modo che la vista rifletta la modifica. Il problema è che la traduzione può essere **ambigua**: se nella vista Supervisione voglio inserire "Lupi ha come capo Bruni", in quale reparto va Lupi? Bruni dirige sia B sia C. E se voglio inserire "Belli ha come capo Falchi", Falchi non dirige alcun reparto. Per questo sulle viste sono ammessi solo pochi tipi di aggiornamento.

3.1.9 Una Notazione Alternativa per i Join

Nella pratica (ed è l'approccio usato da SQL) si adotta spesso una notazione diversa:

- si **ignora il join naturale**: non si assume nessuna condizione implicita sugli attributi con lo stesso nome, ogni join è un theta-join esplicito;
- gli attributi con lo stesso nome in relazioni diverse si distinguono con la sintassi **Relazione.Attributo**;
- per usare due volte la stessa relazione si crea una **vista** con un nuovo nome (es. Capi := Impiegati); la ridenominazione degli attributi serve solo per l'unione.

L'esempio degli impiegati che guadagnano più del capo diventa molto più leggibile:

$$\pi_{\substack{\text{Impiegati.Matricola,Impiegati.Nome,Impiegati.Stipendio,}\\ \text{Capi.Matricola,Capi.Nome,Capi.Stipendio}}}\Big(\sigma_{\text{Impiegati.Stipendio}>\text{Capi.Stipendio}}\Big(\text{Capi} \bowtie_{\text{Capi.Matricola}=\text{Capo}} (\text{Supervisione} \bowtie_{\text{Impiegato}=\text{Impiegati.Matricola}} \text{Impiegati})\Big)\Big)$$

3.2 Calcolo Relazionale e Datalog

3.2.1 Calcolo Relazionale su Domini

Il **calcolo relazionale** è una famiglia di linguaggi **dichiarativi** basati sulla logica del primo ordine: si descrivono le proprietà del risultato, non i passi per ottenerlo. Ne vediamo due versioni: il **calcolo sui domini** e il **calcolo su tuple con dichiarazioni di range**.

Nel calcolo sui domini un'espressione ha la forma:

$$\{ A_1 : x_1, \ldots, A_k : x_k \mid f \}$$

- $A_1 : x_1, \ldots, A_k : x_k$ è la **target list**: A_i sono attributi distinti (lo schema del risultato), x_i variabili distinte;
- f è una **formula** con connettivi logici $(\wedge, \vee, \neg)$ e quantificatori $(\exists, \forall)$.

Il risultato è una relazione su $A_1, \ldots, A_k$ che contiene le tuple di valori per $x_1, \ldots, x_k$ che rendono vera f.

Rispetto alla logica dei predicati: i simboli di predicato sono le **relazioni** della base di dati e i predicati standard $(=, >, \ldots)$; non ci sono simboli di funzione; la notazione è **non posizionale** (si scrive il nome dell'attributo, non la posizione).

Usiamo Impiegati(<u>Matricola</u>, Nome, Età, Stipendio) e Supervisione(Capo, <u>Impiegato</u>).

- Impiegati che guadagnano più di 40 (equivale a $\sigma_{\text{Stipendio}>40}(\text{Impiegati})$):

$$\begin{aligned}&\{\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s \mid \\ &\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s) \wedge s > 40\}\end{aligned}$$

- Matricola, nome ed età di tutti gli impiegati (una proiezione): basta togliere s dalla target list.

$$\begin{aligned}&\{\text{Matricola} : m, \text{Nome} : n, \text{Età} : e \mid \\ &\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s)\}\end{aligned}$$

  Le variabili che non compaiono nella target list sono implicitamente quantificate con $\exists$.

- Capi di impiegati che guadagnano più di 40 (il join si ottiene usando **la stessa variabile** m in due predicati):

$$\begin{aligned}&\{\text{Capo} : c \mid \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m) \wedge \\ &\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s) \wedge s > 40\}\end{aligned}$$

- Impiegati che guadagnano più del proprio capo, con matricola, nome e stipendio di entrambi:

$$\begin{aligned}&\{\text{Matr} : m, \text{Nome} : n, \text{Stip} : s, \text{MatrC} : c, \text{NomeC} : nc, \text{StipC} : sc \mid \\ &\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s) \wedge \\ &\quad \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m) \wedge \\ &\quad \text{Impiegati}(\text{Matricola} : c, \text{Nome} : nc, \text{Età} : ec, \text{Stipendio} : sc) \wedge s > sc\}\end{aligned}$$

- Matricola e nome dei capi i cui impiegati guadagnano **tutti** più di 40: serve la negazione ("non esiste un impiegato di c che guadagna ≤ 40"):

$$\begin{aligned}&\{\text{Matricola} : c, \text{Nome} : n \mid \\ &\quad \text{Impiegati}(\text{Matricola} : c, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s) \wedge \\ &\quad \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m) \wedge \\ &\quad \neg\exists m' \exists n' \exists e' \exists s' \big(\\ &\qquad \text{Impiegati}(\text{Matricola} : m', \text{Nome} : n', \text{Età} : e', \text{Stipendio} : s') \wedge \\ &\qquad \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m') \wedge s' \leq 40 \big)\}\end{aligned}$$

Quale quantificatore usare? È indifferente: $\exists$ e $\forall$ sono intercambiabili grazie alle leggi di De Morgan:

$$\neg(A \wedge B) = \neg A \vee \neg B \qquad \neg(A \vee B) = \neg A \wedge \neg B$$
$$\neg\exists x\, A = \forall x\, \neg A \qquad \exists x\, A = \neg\forall x\, \neg A$$

Nell'ultimo esempio, $\neg\exists m' \ldots (P \wedge Q \wedge s' \leq 40)$ si può riscrivere come $\forall m' \ldots (\neg(P \wedge Q) \vee s' > 40)$.

Pregi e difetti. Il calcolo sui domini è dichiarativo, ma:

- è **verboso**: servono moltissime variabili;
- permette espressioni **senza senso**, come

$$\{A : x \mid \neg R(A : x)\} \qquad \{A : x, B : y \mid R(A : x)\}$$
$$\{A : x, B : y \mid R(A : x) \wedge y = y\}$$

  Il loro risultato dipende dal dominio (es. tutti i valori che **non** stanno in R, potenzialmente infiniti): si dicono **dipendenti dal dominio** e vanno evitate. L'algebra relazionale invece è **indipendente dal dominio**, quindi non può esprimerle.

Calcolo sui Domini e Algebra Relazionale

I due linguaggi sono **equivalenti**:

- per ogni espressione del calcolo sui domini indipendente dal dominio esiste un'espressione equivalente dell'algebra;
- per ogni espressione dell'algebra esiste un'espressione equivalente del calcolo sui domini (indipendente dal dominio).

3.2.2 Calcolo su Tuple con Dichiarazioni di Range

Questa versione nasce per superare i limiti del calcolo sui domini:

- meno variabili: **una variabile per ogni tupla**, non una per ogni attributo;
- tutti i valori devono provenire dalla base di dati: ogni variabile ha un **range**, cioè la relazione da cui prende le tuple.

Le espressioni hanno la forma:

$$\{\,\text{TargetList} \mid \text{RangeList} \mid \text{Formula}\,\}$$

- **RangeList**: elenca le variabili libere e la relazione da cui provengono, es. $i(\text{Impiegati})$;
- **TargetList**: elementi del tipo $Y : x.Z$ (attributi Z della variabile x, rinominati in Y), oppure $x.Z$, oppure $x.*$ (tutti gli attributi);
- **Formula**: confronti $x.A\,\theta\,c$ o $x.A\,\theta\,y.B$, connettivi logici e quantificatori con range: $\exists x(R)(\dots)$, $\forall x(R)(\dots)$.

Gli stessi esempi di prima diventano molto più compatti:

- Impiegati che guadagnano più di 40:

$$\{i.* \mid i(\text{Impiegati}) \mid i.\text{Stipendio} > 40\}$$

- Matricola, nome ed età di tutti:

$$\{i.(\text{Matricola}, \text{Nome}, \text{Età}) \mid i(\text{Impiegati}) \mid\}$$

- Capi di impiegati che guadagnano più di 40:

$$\begin{aligned}\{s.\text{Capo} \mid\ & i(\text{Impiegati}), s(\text{Supervisione}) \mid \\ & i.\text{Matricola} = s.\text{Impiegato} \wedge i.\text{Stipendio} > 40\}\end{aligned}$$

- Nome e stipendio di quei capi (servono due variabili su Impiegati, i e i'):

$$\begin{aligned}\{\text{NomeC}, \text{StipC} : i'.(\text{Nome}, \text{Stipendio}) \mid\ & \\ & i'(\text{Impiegati}), s(\text{Supervisione}), i(\text{Impiegati}) \mid \\ & i'.\text{Matricola} = s.\text{Capo} \wedge i.\text{Matricola} = s.\text{Impiegato} \wedge i.\text{Stipendio} > 40\}\end{aligned}$$

- Impiegati che guadagnano più del capo: come sopra, con target list $i.(\text{Matricola}, \text{Nome}, \text{Stipendio}), \text{MatrC}$ $i'.(\text{Matricola}, \text{Nome}, \text{Stipendio})$ e condizione finale $i.\text{Stipendio} > i'.\text{Stipendio}$.
- Capi i cui impiegati guadagnano tutti più di 40:

$$\begin{aligned}\{i.(\text{Matricola}, \text{Nome}) \mid\ & s(\text{Supervisione}), i(\text{Impiegati}) \mid \\ & i.\text{Matricola} = s.\text{Capo} \wedge \\ & \neg\big(\exists i'(\text{Impiegati})\big(\exists s'(\text{Supervisione})\big(\\ & \quad s.\text{Capo} = s'.\text{Capo} \wedge s'.\text{Impiegato} = i'.\text{Matricola} \wedge \\ & \quad i'.\text{Stipendio} \leq 40\big)\big)\big)\}\end{aligned}$$

Limite: l'unione. Questo calcolo **non può esprimere l'unione** $R_1(AB) \cup R_2(AB)$: ogni variabile ha un solo range, quindi una tupla del risultato non può provenire "o da R_1 o da R_2". Intersezione e differenza invece si esprimono. Per questo SQL, che si basa su questo calcolo, ha un operatore esplicito `UNION`.

3.2.3 Limiti di Algebra e Calcolo: la Chiusura Transitiva

Algebra e calcolo sono sostanzialmente equivalenti ed esprimono un insieme significativo di interrogazioni, ma alcune query utili restano fuori:

- si possono solo **estrarre** valori, non **calcolarne** di nuovi: né sulla singola tupla (conversioni, somme, differenze...) né su insiemi di tuple (somme, medie...). SQL aggiunge queste estensioni;
- non si possono esprimere **query ricorsive**, come la chiusura transitiva.

Chiusura transitiva. Data Supervisione(Impiegato, Capo), vogliamo per ogni impiegato **tutti i superiori**: il capo, il capo del capo, e così via. Se Rossi ha come capo Lupi e Lupi ha come capo Falchi, allora (Rossi, Falchi) fa parte del risultato.

Si possono trovare i superiori di secondo livello con un join della relazione con se stessa (più una ridenominazione), quelli di terzo livello con due join, e così via. Ma il numero di livelli dipende dai dati: **non c'è un limite** al numero di join necessari, quindi nessuna espressione fissa dell'algebra calcola la chiusura transitiva per qualunque istanza.

3.2.4 Datalog

Datalog è un linguaggio di programmazione logica orientato alle basi di dati, derivato dal Prolog. Usa due tipi di predicati:

- **estensionali**: le relazioni della base di dati;
- **intensionali**: corrispondono alle viste, e sono definiti tramite **regole**.

Una regola ha la forma

$$\textit{testa} \leftarrow \textit{corpo}$$

dove la testa è un predicato atomico intensionale e il corpo è una lista di predicati atomici in AND (separati da virgole). Le interrogazioni sono predicati atomici preceduti da **?**.

Esempi:

- Impiegati di 30 anni (basta una costante nel predicato):

$$? \ \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : 30, \text{Stipendio} : s)$$

- Impiegati che guadagnano più di 40: serve un predicato intensionale.

$$\begin{aligned} &\text{Ricchi}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s) \leftarrow \\ &\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s), \ s > 40 \\ &? \ \text{Ricchi}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s) \end{aligned}$$

- Capi di impiegati che guadagnano più di 40:

$$\begin{aligned} &\text{CapiDiRicchi}(\text{Capo} : c) \leftarrow \\ &\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s), \\ &\quad s > 40, \ \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m) \end{aligned}$$

- Capi i cui impiegati guadagnano tutti più di 40: serve la **negazione**. Prima si definiscono i capi che hanno almeno un impiegato "povero", poi si escludono:

$$\begin{aligned}
&\text{CapiDiNonRicchi}(\text{Capo} : c) \leftarrow \\
&\quad \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m), \\
&\quad \text{Impiegati}(\text{Matricola} : m, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s),\ s \leq 40 \\
&\text{CapiSoloDiRicchi}(\text{Matricola} : c, \text{Nome} : n) \leftarrow \\
&\quad \text{Impiegati}(\text{Matricola} : c, \text{Nome} : n, \text{Età} : e, \text{Stipendio} : s), \\
&\quad \text{Supervisione}(\text{Capo} : c, \text{Impiegato} : m), \\
&\quad \textbf{not}\ \text{CapiDiNonRicchi}(\text{Capo} : c)
\end{aligned}$$

- Per ogni impiegato, tutti i superiori: serve la **ricorsione**, che Datalog permette.

$$\begin{aligned}
&\text{Superiore}(\text{Impiegato} : i, \text{SuperCapo} : c) \leftarrow \\
&\quad \text{Supervisione}(\text{Impiegato} : i, \text{Capo} : c) \\
&\text{Superiore}(\text{Impiegato} : i, \text{SuperCapo} : c) \leftarrow \\
&\quad \text{Supervisione}(\text{Impiegato} : i, \text{Capo} : c'), \\
&\quad \text{Superiore}(\text{Impiegato} : c', \text{SuperCapo} : c)
\end{aligned}$$

  La prima regola dice che il capo diretto è un superiore; la seconda che un superiore del mio capo è anche un mio superiore.

Potere espressivo. La semantica delle query ricorsive è delicata, soprattutto con la negazione. In sintesi:

- Datalog **non ricorsivo senza negazione** equivale al calcolo senza negazione e senza quantificatore universale;
- Datalog **non ricorsivo con negazione** equivale a calcolo e algebra;
- Datalog **ricorsivo senza negazione** non è confrontabile con il calcolo;
- Datalog **ricorsivo con negazione** è **più espressivo** di calcolo e algebra.

3.3 Esercizi di algebra relazionale

3.3.1 Esercizi visti in classe

Esercizio 1 Data la relazione:

```
REGION(Name, Surface),
CITY(Code, Name, Population),
BELONGING(Region, City)
```

Scrivere l'espressione in algebra relazionale per ottenere i nomi delle regioni aventi città con più di 1.000.000 abitanti.

Soluzione: $\pi_{\text{Region}}(\rho_{\text{Code}\leftarrow\text{City}}(\text{BELONGING}) \bowtie \sigma_{\text{Population}>1000000}(\text{CITY}))$

Esercizio 2 Data la relazione:

`TRAIN(Code, Start, End, Miles)`

Scrivere l'espressione in algebra relazionale per ottenere tutte le rotte tra Boston e Chicago con uno scalo.

Soluzione:

$$\sigma_{\text{Start}='\text{Boston}' \wedge \text{End}='\text{Chicago}'}\Big(\rho_{\text{Code1, Switch, Miles1}\leftarrow\text{Code, End, Miles}}(\text{CODE}) \bowtie \rho_{\text{Code2, Switch, Miles2}\leftarrow\text{Code, Start, Miles}}(\text{TRAIN})\Big)$$

Soluzioni estese:

1. $$\rho_{\text{Code1, Switch, Miles1}\leftarrow\text{Code, End, Miles}}(\text{TRAIN}) \bowtie_{\text{Start}='\text{Boston}' \wedge \text{End}='\text{Chicago}'} \rho_{\text{Code2, Switch, Miles2}\leftarrow\text{Code, Start, Miles}}(\text{TRAIN})$$

2. $$\rho_{\text{Code1, Switch, Miles1}\leftarrow\text{Code, End, Miles}}\big(\sigma_{\text{Start}='\text{Boston}'}(\text{TRAIN})\big) \bowtie \rho_{\text{Code2, Switch, Miles2}\leftarrow\text{Code, Start, Miles}}\big(\sigma_{\text{End}='\text{Chicago}'}(\text{TRAIN})\big)$$

Esercizio 3 Date le seguenti relazioni:

`REGION(Name, Surface)`
`CITY(Code, Name, Population)`
`BELONGING(Region, City)`

Scrivere un'espressione in algebra relazionale che restituisca i nomi distinti delle città appartenenti a regioni con superficie superiore a 2.500 piedi quadrati. È consentito utilizzare esclusivamente le operazioni di ridenominazione, selezione, proiezione e join naturale.

Soluzione:

$$\pi_{\text{Name}}\Big(\rho_{\text{Region}\leftarrow\text{Name}}\big(\sigma_{\text{Surface}>2500}(\text{REGION})\big) \bowtie \text{BELONGING} \bowtie \rho_{\text{City}\leftarrow\text{Code}}(\text{CITY})\Big)$$

Soluzione alternativa:

$$\pi_{\text{Name}}\Big(\sigma_{\text{Surface}>2500}\big(\rho_{\text{Region}\leftarrow\text{Name}}(\text{REGION})\big) \bowtie \text{BELONGING} \bowtie \rho_{\text{City}\leftarrow\text{Code}}(\text{CITY})\Big)$$

Esercizio 4 Date le seguenti relazioni:

```
PERSON(Id, Name, Surname, Age)
ENROLLMENT(Person, Course)
COURSE(Name, Price)
```

Scrivere un'espressione in algebra relazionale che restituisca i nomi distinti delle persone che frequentano un corso il cui costo è superiore a 10$ per ogni anno della loro età (ovvero, Price $>$ Age $*$ 10).

Soluzione:

$$\pi_{\text{Name}}\Big(\sigma_{\text{Price}>\text{Age}*10}\big(\rho_{\text{Person}\leftarrow\text{Id}}(\text{PERSON}) \bowtie \text{ENROLLMENT} \bowtie \rho_{\text{Course}\leftarrow\text{Name}}(\text{COURSE})\big)\Big)$$

Esercizio 5 Date le seguenti relazioni:

```
STUDENT(Id, Name, Surname, School)
EXAM(Student, Subject, Mark, Date)
```

Scrivere un'espressione in algebra relazionale che restituisca nome e cognome degli studenti che hanno ottenuto 30 in almeno un esame.

Soluzione:

$$\pi_{\text{Name, Surname}}\Big(\text{STUDENT} \bowtie_{\text{Id=Student}} \sigma_{\text{Mark}=30}(\text{EXAM})\Big)$$

Esercizio 6 Date le seguenti relazioni:

```
SUPPLIER(Id, Name, Partners, Headquarter)
PRODUCT(Id, Name, Colour, Size, Store)
SUPPLYING(SupId, ProdId, Quantity)
```

Scrivere un'espressione in algebra relazionale che restituisca il nome dei fornitori che forniscono almeno un prodotto diverso da "PX274".

Soluzione:

$$\pi_{\text{Name}}\Big(\text{SUPPLIER} \bowtie_{\text{Id=SupId}} \sigma_{\text{ProdId}\neq'\text{PX274}'}(\text{SUPPLYING})\Big)$$

Esercizio 7 Date le seguenti relazioni:

```
SUPPLIER(Id, Name, Partners, Headquarter)
PRODUCT(Id, Name, Colour, Size, Store)
SUPPLYING(SupId, ProdId, Quantity)
```

Scrivere un'espressione in algebra relazionale che restituisca il codice dei fornitori che forniscono almeno due prodotti diversi.

Soluzione:

$$\pi_{\text{SupId1}}\Big(\rho_{\text{SupId1, ProdId1, Quantity1}\leftarrow\text{SupId, ProdId, Quantity}}(\text{SUPPLYING}) \bowtie_{\text{SupId1=SupId2}\,\wedge\,\text{ProdId1}\neq\text{ProdId2}} \rho_{\text{SupId2, ProdId2, Quantity2}\leftarrow\text{SupId, ProdId, Quantity}}(\text{SUPPLYING})\Big)$$

Esercizio 8 Date le seguenti relazioni:

```
STUDENT(Id, Name, Surname, School)
EXAM(Student, Subject, Mark, Date)
```

Scrivere un'espressione in algebra relazionale che restituisca nome e cognome degli studenti che non hanno mai ottenuto 30 in un esame.

Soluzione:

$$\pi_{\text{Name, Surname, Id}}(\text{STUDENT})$$
$$-$$
$$\pi_{\text{Name, Surname, Id}}\big(\text{STUDENT} \bowtie_{\text{Id=Student}} \sigma_{\text{Mark=30}}(\text{EXAM})\big)$$

Esercizio 9 Date le seguenti relazioni:

```
BOOK(BookId, Title, Author)
USER(UserId, Name, Surname)
BORROWING(UserId, BookId, Date)
```

Scrivere un'espressione in algebra relazionale che restituisca l'identificativo degli utenti che hanno preso in prestito un libro di Fleming.

Soluzione:

$$\pi_{\text{UserId}}\big(\sigma_{\text{Author='Fleming'}}(\text{BOOK}) \bowtie \text{BORROWING}\big)$$

Esercizio 10 Date le seguenti relazioni:

```
BOOK(BookId, Title, Author)
USER(UserId, Name, Surname)
BORROWING(UserId, BookId, Date)
```

Scrivere un'espressione in algebra relazionale che restituisca l'identificativo degli utenti che hanno preso in prestito esclusivamente libri di Fleming.

Soluzione:

$$\pi_{\text{UserId}}\big(\sigma_{\text{Author='Fleming'}}(\text{BOOK}) \bowtie \text{BORROWING}\big)$$
$$-$$
$$\pi_{\text{UserId}}\big(\sigma_{\text{Author}\neq\text{'Fleming'}}(\text{BOOK}) \bowtie \text{BORROWING}\big)$$

Esercizio 11 Date le seguenti relazioni:

```
EXAM(Code, Subject, Teacher)
ROOM(Code, Name, Building, Capacity)
BOOKING(Room, Date, Exam)
```

Scrivere un'espressione in algebra relazionale che restituisca il nome e l'edificio delle aule in cui si tiene oggi una prova del corso di Database.

Soluzione:

$$\pi_{\text{Name, Building}}\Big(\pi_{\text{Room}}\big(\sigma_{\text{Subject}='\text{Database}'}(\text{EXAM}) \bowtie_{\text{Code=Exam}} \sigma_{\text{Date}='\text{today}'}(\text{BOOKING})\big) \bowtie_{\text{Room=Code}} \text{ROOM}\Big)$$

Esercizio 12 Date le seguenti relazioni:

```
STUDENT(Id, Name, Surname, School)
EXAM(Student, Subject, Mark, Date)
```

Scrivere un'espressione in algebra relazionale che restituisca nome e cognome degli studenti che hanno ottenuto 30 in almeno un esame, riportando anche la data del primo di tali esami.

Soluzione:

$$R_1 := \pi_{\text{Student, Date}}(\sigma_{\text{Mark=30}}(\text{EXAM}))$$

$$R_2 := \rho_{\text{Date1}\leftarrow\text{Date}}(R_1)$$

$$R_3 := \rho_{\text{Date}\leftarrow\text{Date1}}\Big(R_2 - \pi_{\text{Student, Date1}}\big(\sigma_{\text{Date<Date1}}(R_1 \bowtie R_2)\big)\Big)$$

$$\pi_{\text{Name, Surname, Date}}\big(\text{STUDENT} \bowtie_{\text{Id=Student}} R_3\big)$$

Esercizio 13 Date le seguenti relazioni:

```
EXAM(Code, Subject, Teacher)
ROOM(Code, Name, Building, Capacity)
BOOKING(Room, Date, Exam)
```

Scrivere un'espressione in algebra relazionale che restituisca il nome delle aule con capienza maggiore o uguale a 100 posti che non risultano prenotate per oggi.

Soluzione:

$$\text{BIGROOM} := \sigma_{\text{Capacity}\geq 100}(\text{ROOM})$$

$$\pi_{\text{Name}}\Big(\pi_{\text{Name, Code}}(\text{BIGROOM}) - \pi_{\text{Name, Code}}\big(\text{BIGROOM} \bowtie_{\text{Code=Room}} \sigma_{\text{Date}='\text{today}'}(\text{BOOKING})\big)\Big)$$

Esercizio 14 Date le seguenti relazioni:

```
WINESHOP(Code, Name, Address, ZipC)
CATALOG(Wineshop, Wine, Price)
WINE(Code, Name, Color, Region)
```

Scrivere un'espressione in algebra relazionale che restituisca il codice delle enoteche che vendono vini rosé o rossi.

Soluzione:

$$\pi_{\text{Wineshop}}\Big(\text{CATALOG} \bowtie_{\text{Wine=Code}} \sigma_{\text{Color='Rose'} \vee \text{Color='Red'}}(\text{WINE})\Big)$$

Esercizio 15 Date le seguenti relazioni:

```
WINESHOP(Code, Name, Address, ZipC)
CATALOG(Wineshop, Wine, Price)
WINE(Code, Name, Color, Region)
```

Scrivere un'espressione in algebra relazionale che restituisca il nome delle enoteche che vendono sia vini rosé che vini rossi.

Soluzione:

$$\pi_{\text{Name}}\Big(\Big(\pi_{\text{Wineshop}}\big(\text{CATALOG} \bowtie_{\text{Wine=Code}} \sigma_{\text{Color='Rose'}}(\text{WINE})\big) \cap \pi_{\text{Wineshop}}\big(\text{CATALOG} \bowtie_{\text{Wine=Code}} \sigma_{\text{Color='Red'}}(\text{WINE})\big)\Big) \bowtie_{\text{Wineshop=Code}} \text{WINESHOP}\Big)$$

Esercizio 16 Date le seguenti relazioni:

```
PERSON(Name, Age, Gender)
FREQUENTS(Name, Pizzeria)
EATS(Name, Pizza)
SERVES(Pizzeria, Pizza, Price)
```

Scrivere un'espressione in algebra relazionale che restituisca i nomi di tutte le donne che mangiano sia la pizza ai funghi che la pizza ai peperoni.

Soluzione:

$$\pi_{\text{Name}}\big(\sigma_{\text{Gender}='\text{F}' \wedge \text{Pizza}='\text{pepperoni}'}(\text{PERSON} \bowtie \text{EATS})\big)$$
$$\cap$$
$$\pi_{\text{Name}}\big(\sigma_{\text{Gender}='\text{F}' \wedge \text{Pizza}='\text{mushroom}'}(\text{PERSON} \bowtie \text{EATS})\big)$$

Esercizio 17 Date le seguenti relazioni:

```
PERSON(Name, Age, Gender)
FREQUENTS(Name, Pizzeria)
EATS(Name, Pizza)
SERVES(Pizzeria, Pizza, Price)
```

Scrivere un'espressione in algebra relazionale che restituisca i nomi di tutte le pizzerie frequentate esclusivamente da donne o esclusivamente da uomini.

Soluzione:

$$\Big(\pi_{\text{Pizzeria}}\big(\sigma_{\text{Gender}='\text{F}'}(\text{PERSON}) \bowtie \text{FREQUENTS}\big)$$
$$-$$
$$\pi_{\text{Pizzeria}}\big(\sigma_{\text{Gender}='\text{M}'}(\text{PERSON}) \bowtie \text{FREQUENTS}\big)\Big)$$
$$\cup$$
$$\Big(\pi_{\text{Pizzeria}}\big(\sigma_{\text{Gender}='\text{M}'}(\text{PERSON}) \bowtie \text{FREQUENTS}\big)$$
$$-$$
$$\pi_{\text{Pizzeria}}\big(\sigma_{\text{Gender}='\text{F}'}(\text{PERSON}) \bowtie \text{FREQUENTS}\big)\Big)$$

Esercizio 18 Date le seguenti relazioni:

```
MAGAZINE(Code, Name, Editor)
ARTICLE(Code, Title, Topic, Magazine)
```

Scrivere un'espressione in algebra relazionale che restituisca il codice e il nome delle riviste che non hanno mai pubblicato articoli riguardanti il motociclismo.

Soluzione:

$$\pi_{\text{Code, Name}}\Big(\text{MAGAZINE} \bowtie \big(\pi_{\text{Code}}(\text{MAGAZINE})$$
$$-$$
$$\rho_{\text{Code} \leftarrow \text{Magazine}}\big(\pi_{\text{Magazine}}(\sigma_{\text{Topic}='\text{motorcycling}'}(\text{ARTICLE}))\big)\Big)$$

Esercizio 19 Date le seguenti relazioni:

```
SAILOR(Id, Name, Level, BirthD)
BOOKING(Sailor, Boat, Date)
BOAT(Id, Name, Colour)
```

Scrivere un'espressione in algebra relazionale che restituisca codici e nomi dei marinai che hanno prenotato sia una barca rossa che una barca verde.

Soluzione:

$$\pi_{\text{Sid, Sname}}\Big(\rho_{\text{Sid, Sname, Slevel, SbirthD}\leftarrow\text{Id, Name, Level, BirthD}}(\text{SAILOR}) \bowtie_{\text{Sid=Sailor1}} \Big(\rho_{\text{Sailor1}\leftarrow\text{Sailor}}\big(\text{BOOKING} \bowtie_{\text{Boat=Id}} \sigma_{\text{Colour='Red'}}(\text{BOAT})\big) \bowtie_{\text{Sailor1=Sailor2}} \rho_{\text{Sailor2}\leftarrow\text{Sailor}}\big(\text{BOOKING} \bowtie_{\text{Boat=Id}} \sigma_{\text{Colour='Green'}}(\text{BOAT})\big)\Big)\Big)$$

Esercizio 20 Date le seguenti relazioni:

```
PLANE(Id, Name, Range)
CERTIFICATE(Eid, Pid)
EMPLOYEE(Id, Name, Wage)
```

Scrivere un'espressione in algebra relazionale che restituisca codici e nomi dei dipendenti autorizzati a pilotare almeno due aerei capaci di coprire distanze superiori a 5000 miglia.

Soluzione:

$$\pi_{\text{Id, Name}}\Big(\text{EMPLOYEE} \bowtie_{\text{Id=Eid1}} \Big(\rho_{\text{Id1, Eid1}\leftarrow\text{Id, Eid}}\big(\sigma_{\text{Range}\geq 5000}(\text{PLANE}) \bowtie_{\text{Id=Pid}} \text{CERTIFICATE}\big) \bowtie_{\text{Eid1=Eid2}\,\wedge\,\text{Id1}\neq\text{Id2}} \rho_{\text{Id2, Eid2}\leftarrow\text{Id, Eid}}\big(\sigma_{\text{Range}\geq 5000}(\text{PLANE}) \bowtie_{\text{Id=Pid}} \text{CERTIFICATE}\big)\Big)\Big)$$

Appendice: Notazione per esami su piattaforma (EOL) Sintassi testuale equivalente utilizzata per l'inserimento delle query:

Operatori relazionali e logici:

- **Selezione:** `SEL[condizione] Relazione`
- **Proiezione:** `PRO[attributo1, ...] Relazione`

- **Join con condizione:** `R1 JOIN[condizione] R2`
- **Ridenominazione:** `RID[(nuovo_att, vecchio_att), ...] Relazione`
- **Insiemistici:** `R1 UNI R2` (Unione), `R1 INT R2` (Intersezione), `R1 DIF R2` (Differenza)
- **Logici:** `AND`, `OR`, `NOT`

Esempio di query d'esame:
Standard:

$$\pi_{\text{idModello, nome}}\big(\sigma_{\text{categoria}='\text{Berlina}' \wedge \text{nazionalita}='\text{Francia}'}(\text{Modello} \bowtie \rho_{\text{nomeM}\leftarrow\text{nome}}(\text{Marca}))\big)$$
$$-$$
$$\pi_{\text{idModello, nome}}\big(\sigma_{\text{anno}='2007'}(\text{Modello} \bowtie_{\text{idModello=modello}} \text{Valutazione})\big)$$

Notazione EOL:

```
(PRO [idModello, nome] (SEL [categoria='Berlina' AND nazionalita='Francia']
  (Modello JOIN (RID [(nomeM, nome)] Marca))))
DIF
(PRO [idModello, nome] (SEL [anno='2007']
  (Modello JOIN [idModello=modello] Valutazione)))
```

3.3.2 Pattern utili da imparare

Convenzioni e simboli utilizzati Nelle formule generali che seguono vengono utilizzate le seguenti convenzioni:

- R, S: relazioni (tabelle) di partenza o espressioni relazionali intermedie;
- Key: insieme di attributi identificativi (chiave primaria o chiave esterna che lega le relazioni);
- Target: insieme degli attributi finali richiesti dal testo (es. nomi, cognomi, codici);
- P, P_1, P_2: condizioni logico-booleane di selezione (es. Prezzo > 50, Città $=' \text{Roma}'$);
- $\rho, \sigma, \pi, \bowtie, -, \cap, \cup$: operatori standard di ridenominazione, selezione, proiezione, join, differenza, intersezione e unione.

Pattern 1: Esistenziale semplice ("Almeno uno con proprietà P") È la forma base: collegare due entità collegate da una relazione uno-a-molti e filtrare sulla proprietà desiderata.

- **Schema logico:**
$$\pi_{\text{Target}}\big(R \bowtie \sigma_P(S)\big)$$
- **Strategia:** Eseguire la selezione σ sulla tabella che contiene il vincolo prima del join (push-down della selezione), unire con la tabella anagrafica R contenente le informazioni da mostrare, e infine proiettare.
- **Esercizi di riferimento:** Es. 1, Es. 5, Es. 6, Es. 9, Es. 14.
- **Esempio applicativo:**
Schema: `DOCENTE(Id, Nome, Dipartimento)`, `CORSO(Codice, Titolo, CFU, Docente)`
Query: Restituire i nomi dei docenti che tengono almeno un corso con più di 6 CFU.
$$\pi_{\text{Nome}}\big(\text{DOCENTE} \bowtie_{\text{Id=Docente}} \sigma_{\text{CFU}>6}(\text{CORSO})\big)$$

Pattern 2: Cardinalità multipla ("Almeno due elementi distinti") Richiede di verificare l'esistenza di tuple distinte che fanno capo alla medesima entità padre, confrontando una relazione con se stessa.

- **Schema logico:**
$$\pi_{\text{Key1}}\big(\rho_1(R) \bowtie_{\text{Key1=Key2} \wedge \text{Val1} \neq \text{Val2}} \rho_2(R)\big)$$
- **Regola chiave:** La ridenominazione degli attributi delle due copie è obbligatoria per evitare ambiguità. La condizione Val1 $\neq$ Val2 (oppure Val1 $<$ Val2 per rompere la simmetria) garantisce che non si tratti della medesima tupla.
- **Esercizi di riferimento:** Es. 7, Es. 20.
- **Esempio applicativo:**
Schema: `CLIENTE(`<u>`Id`</u>`, Nome)`, `ORDINE(`<u>`NumOrdine`</u>`, Cliente, Articolo, Data)`
Query: Restituire l'Id dei clienti che hanno acquistato almeno due articoli differenti.
$$\pi_{\text{Cliente1}}\Big(\rho_{\text{Num1, Cliente1, Articolo1, Data1} \leftarrow \text{NumOrdine, Cliente, Articolo, Data}}(\text{ORDINE}) \bowtie_{\text{Cliente1=Cliente2} \wedge \text{Articolo1} \neq \text{Articolo2}} \rho_{\text{Num2, Cliente2, Articolo2, Data2} \leftarrow \text{NumOrdine, Cliente, Articolo, Data}}(\text{ORDINE})\Big)$$

Pattern 3: Congiunzione su righe distinte ("Sia A che B") Si presenta quando un'entità deve possedere contemporaneamente due proprietà che compaiono su tuple separate di una tabella dipendente (es. due acquisti, due iscrizioni).

- **Schema logico (Intersezione):**
$$\pi_{\text{Key}}(\sigma_{P_1}(S)) \ \cap \ \pi_{\text{Key}}(\sigma_{P_2}(S))$$
- **Schema logico (Join naturale / Theta-join):**
$$\pi_{\text{Key1}}\big(\rho_1(\sigma_{P_1}(S)) \bowtie_{\text{Key1=Key2}} \rho_2(\sigma_{P_2}(S))\big)$$
- **Regola chiave:** L'operatore $\cap$ richiede l'unione-compatibilità: bisogna prima proiettare sullo stesso insieme di attributi (solitamente la chiave dell'entità). Eventuali anagrafiche vanno unite all'esterno dopo l'intersezione.
- **Esercizi di riferimento:** Es. 15, Es. 16, Es. 19.
- **Esempio applicativo:**
Schema: `ATLETA(`<u>`Matricola`</u>`, Nome)`, `ISCRIZIONE(`<u>`Matricola`</u>`, `<u>`Gara`</u>`)`
Query: Restituire nome e matricola degli atleti iscritti sia ai '100m' che ai '200m'.
$$\pi_{\text{Matricola, Nome}}\Big(\text{ATLETA} \bowtie \big(\pi_{\text{Matricola}}(\sigma_{\text{Gara}='100m'}(\text{ISCRIZIONE})) \cap \pi_{\text{Matricola}}(\sigma_{\text{Gara}='200m'}(\text{ISCRIZIONE}))\big)\Big)$$

Pattern 4: Negazione assoluta ("Non ha mai", "Nessuno", "Senza") L'algebra relazionale non supporta filtri negativi diretti tra tuple indipendenti. La negazione si realizza come sottrazione dell'insieme di chi viola il vincolo dall'insieme universo di candidati.

- **Schema logico:**
$$\pi_{\text{Key}}(R) \ - \ \pi_{\text{Key}}\big(R \bowtie \sigma_{\text{Condizione}}(S)\big)$$
- **Regola chiave:** I due rami della differenza ($-$) devono essere **compatibili per schema**. Proiettare sempre preventivamente sulla medesima chiave/insieme di attributi. Non sottrarre prima di aver isolato le chiavi.
- **Esercizi di riferimento:** Es. 8, Es. 13, Es. 18.
- **Esempio applicativo:**
Schema: `LETTORE(Id, Nome)`, `PRESTITO(IdLettore, IdLibro, Anno)`
Query: Restituire l'Id dei lettori che non hanno mai preso in prestito alcun libro nel 2024.
$$\pi_{\text{Id}}(\text{LETTORE}) \ - \ \pi_{\text{IdLettore}}(\sigma_{\text{Anno}=2024}(\text{PRESTITO}))$$

Pattern 5: Condizione universale / Esclusività ("Esclusivamente", "Solo") Seleziona le entità che soddisfano la proprietà P e non hanno mai assunto una proprietà diversa da P.

- **Schema logico:**
$$\pi_{\text{Key}}(\sigma_P(S)) \ - \ \pi_{\text{Key}}(\sigma_{\neg P}(S))$$
- **Spiegazione:** Il primo termine individua i soggetti con almeno una tupla valida per P (evita di selezionare entità vuote o inattive); il secondo termine individua chiunque abbia almeno una tupla che contraddice P. La loro differenza lascia solo i soggetti puramente conformi.
- **Esercizi di riferimento:** Es. 10, Es. 17.
- **Esempio applicativo:**
Schema: `MEDICO(Id, Nome)`, `VISITA(IdVisita, Medico, Specialita)`
Query: Restituire l'Id dei medici che effettuano visite esclusivamente nel campo della 'Cardiologia'.
$$\pi_{\text{Medico}}(\sigma_{\text{Specialita}=\text{'Cardiologia'}}(\text{VISITA})) \ - \ \pi_{\text{Medico}}(\sigma_{\text{Specialita}\neq\text{'Cardiologia'}}(\text{VISITA}))$$

Pattern 6: Calcolo dell'estremo (Minimo / Massimo / Primo evento) In assenza delle funzioni aggregate di SQL (min, max), l'estremo si individua sottraendo dall'insieme totale dei candidati tutti quelli per cui esiste un altro elemento strettamente migliore (o peggiore).

- **Schema logico per determinare il minimo di un attributo V:**
  1. Proiettare le coppie (Key, V): $T_1 \leftarrow \pi_{\text{Key},V}(R)$
  2. Ridenominare: $T_2 \leftarrow \rho_{V' \leftarrow V}(T_1)$
  3. Trovare gli elementi non minimi (esiste un $V < V'$ per lo stesso Key):
  $T_{\text{non_min}} \leftarrow \pi_{\text{Key},V'}\big(\sigma_{V<V'}(T_1 \bowtie T_2)\big)$
  4. Isolare il minimo per differenza:
  $T_{\text{min}} \leftarrow \rho_{V \leftarrow V'}(T_2) - \rho_{V \leftarrow V'}(T_{\text{non_min}})$

- **Esercizi di riferimento:** Es. 12.
- **Esempio applicativo:**
  Schema: `PRODOTTO(Codice, Nome, Prezzo)`
  Query: Restituire il codice e il prezzo del prodotto con il prezzo minimo assoluto.

$$P_1 := \pi_{\text{Codice, Prezzo}}(\text{PRODOTTO})$$
$$P_2 := \rho_{\text{Cod2, Prezzo2} \leftarrow \text{Codice, Prezzo}}(P_1)$$
$$P_{\text{non_min}} := \pi_{\text{Cod2, Prezzo2}}\big(\sigma_{\text{Prezzo} < \text{Prezzo2}}(P_1 \times P_2)\big)$$
$$P_{\text{min}} := P_2 - P_{\text{non_min}}$$
$$\pi_{\text{Cod2, Prezzo2}}(P_{\text{min}})$$

Pattern 7: Cammini e catene (Transitività con scalo / Nodi intermedi) Utilizzato quando una relazione modella un grafo orientato (nodi e archi) e l'interrogazione richiede percorsi composti da due o più passi consecutivi.

- **Schema logico (Percorso a 2 passi con 1 nodo intermedio):**

$$\rho_1(R) \bowtie_{\text{Destinazione1=Origine2}} \rho_2(R)$$

- **Filtraggio estremi:** applicare la selezione $\sigma_{\text{Origine1}='\text{Start}' \wedge \text{Destinazione2}='\text{End}'}$ per bloccare il punto di partenza e il traguardo.
- **Esercizi di riferimento:** Es. 2.
- **Esempio applicativo:**
  Schema: `VOLO(Codice, Partenza, Destinazione, Durata)`
  Query: Trovare tutti i voli da 'Roma' a 'Tokyo' con esattamente uno scalo intermedio.

$$\sigma_{\text{Partenza1}='\text{Roma}' \wedge \text{Destinazione2}='\text{Tokyo}'}\Big($$
$$\rho_{\text{Cod1, Partenza1, Scalo, Durata1} \leftarrow \text{Codice, Partenza, Destinazione, Durata}}(\text{VOLO})$$
$$\bowtie_{\text{Scalo=Partenza2}}$$
$$\rho_{\text{Cod2, Partenza2, Destinazione2, Durata2} \leftarrow \text{Codice, Partenza, Destinazione, Durata}}(\text{VOLO})\Big)$$

4 SQL di base

4.1 Definizione dei Dati (DDL)

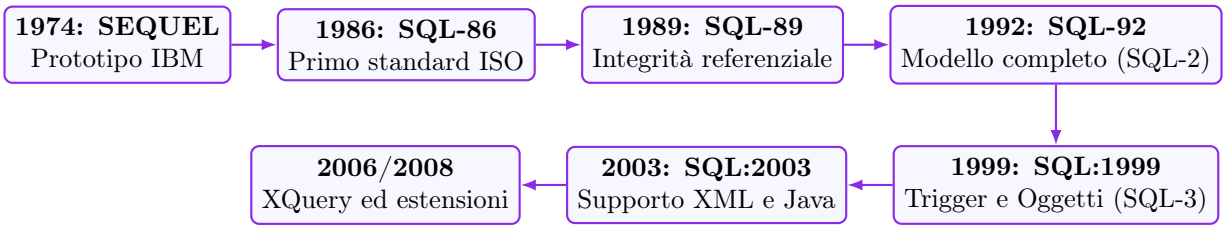
4.1.1 Origine, Standardizzazione ed Evoluzione del Linguaggio

Definizione e Natura di SQL. La sigla **SQL** nasceva originariamente come acronimo di *Structured Query Language*; nel tempo è divenuta un nome proprio a tutti gli effetti. Rappresenta il linguaggio standard per l'interazione con i database relazionali ed è un linguaggio completo: integra sia comandi di definizione della struttura (**DDL**, *Data Definition Language*) sia comandi di manipolazione e interrogazione dei dati (**DML**, *Data Manipulation Language*).

Cenni Storici. Le radici di SQL risalgono al **1974**, quando nei laboratori IBM venne sviluppato **SEQUEL** all'interno del progetto pionieristico *System R*. Le prime implementazioni commerciali apparvero all'inizio degli anni '80 con Oracle (1981) e SQL/DS di IBM. A partire dal **1983** il linguaggio si impose come standard *de facto* nel panorama industriale, portando gli enti di standardizzazione ANSI e ISO a formalizzarne le specifiche ufficiali.

L'Evoluzione degli Standard ISO/ANSI. Nel corso degli anni lo standard è stato costantemente arricchito per rispondere alle crescenti esigenze applicative:

- **SQL-86 (SQL-base)**: primo standard ufficiale. Conteneva le parole chiave fondamentali per l'interrogazione, ma offriva un supporto ancora rudimentale per la modifica degli schemi.
- **SQL-89**: estese lo standard introducendo formalmente il supporto nativo ai **vincoli di integrità referenziale** (chiavi esterne).
- **SQL-92 (SQL-2)**: consolidò il modello relazionale, introdusse funzioni condizionali avanzate (`CASE`, `COALESCE`, `NULLIF`) e definì tre livelli formali di conformità per i produttori (*entry*, *intermediate*, *full*).
- **SQL:1999 (SQL-3)**: segnò il passaggio all'approccio oggetto-relazionale, introducendo tipi composti, trigger e funzioni esterne.
- **SQL:2003 e versioni successive**: integrarono l'interoperabilità con linguaggi a oggetti (come Java mediante SQL/JRT), la gestione di dati semistrutturati (**SQL/XML**), linguaggi di interrogazione gerarchica come **XQuery** (SQL:2006) e raffinamenti sintattici sui trigger (SQL:2008).



Standard Ufficiale vs Dialetti Reali. Sebbene esista uno standard internazionale rigoroso, nella pratica ogni DBMS commerciale o open source (PostgreSQL, Oracle, MySQL, SQL Server, SQLite) implementa una propria grammatica proprietaria (dialetto). Piccole differenze sintattiche o funzioni aggiuntive rendono spesso necessaria la consultazione della documentazione specifica del motore adottato.

Nome Ufficioso	Nome Ufficiale	Caratteristiche Principali Introdottesi
SQL-Base	SQL-86	Sintassi fondamentale di selezione; creazione schemi embrionale.
	SQL-89	Supporto completo all'integrità referenziale (*Foreign Key*).
SQL-2	SQL-92	Modello relazionale completo, 3 livelli di conformità (*entry, intermediate, full*), costrutto `CASE`.
SQL-3	SQL:1999	Modello oggetto-relazionale, trigger nativi, funzioni esterne.
	SQL:2003	Estensioni per dati semistrutturati (`SQL/XML`) e integrazione Java.
	SQL:2006	Supporto a XQuery per interrogare documenti XML.
	SQL:2008	Ritocchi sintattici (ad es. clausola `INSTEAD OF` per i trigger).

4.1.2 Database, Schemi, Tabelle e Vincoli

Il Livello DDL. Il **Data Definition Language** comprende tutte le istruzioni con cui si creano, modificano o cancellano le strutture logiche della base di dati (database, schemi, tabelle, domini e indici).

Creazione di un Database. L'ambiente di livello più alto è il database fisico, contenente l'insieme di tabelle, viste e procedure:

```
CREATE DATABASE nome_database;
```

In ambienti embedded o specializzati la sintassi può variare: ad esempio, in SQLite la creazione avviene aprendo la connessione al file con `sqlite3_open_v2()`, mentre in DBMS come Mimer si usa la dicitura `CREATE DATABANK`.

Lo Schema Logico (`CREATE SCHEMA`). Uno schema SQL raccoglie e qualifica un insieme omogeneo di tabelle, domini e vincoli appartenenti a un contesto applicativo o a un utente:

```
CREATE SCHEMA anagrafica_aziendale;
```

È possibile legare direttamente la titolarità dello schema a un utente autorizzato specificando la clausola `AUTHORIZATION`:

```
CREATE SCHEMA anagrafica_aziendale AUTHORIZATION 'responsabile_hr';
```

Creazione delle Tabelle (`CREATE TABLE`). È l'istruzione cardine del DDL: alloca una nuova relazione logica, ne istanzia una struttura inizialmente priva di tuple e ne definisce attributi, tipi di dato e vincoli di integrità:

```
CREATE TABLE Impiegato (
    Matricola CHAR(6) PRIMARY KEY,
    Nome      CHAR(20) NOT NULL,
    Cognome   CHAR(20) NOT NULL,
    Dipart    CHAR(15),
    Stipendio NUMERIC(9) DEFAULT 0,
    FOREIGN KEY (Dipart) REFERENCES Dipartimento(NomeDipart),
    UNIQUE (Cognome, Nome)
);
```

Questa dichiarazione stabilisce in modo compatto:

- La chiave primaria (`Matricola`), garantita automaticamente univoca e non nulla.
- L'obbligatorietà dei campi `Nome` e `Cognome` (`NOT NULL`).
- Un valore predefinito (`DEFAULT 0`) assegnato allo stipendio qualora non venga indicato in fase di inserimento.
- Un vincolo di integrità referenziale verso la tabella `Dipartimento`.
- Un vincolo di unicità composito sulla coppia (`Cognome`, `Nome`).

Tipi di Dato Elementari. In SQL i tipi di dato corrispondono ai domini dell'algebra relazionale. Lo standard mette a disposizione diversi tipi predefiniti:

- **Stringhe di caratteri**:
  - `CHAR(n)`: stringa a lunghezza fissa di n caratteri (eventuali spazi mancanti vengono aggiunti automaticamente a destra).
  - `VARCHAR(n)`: stringa a lunghezza variabile, con limite massimo pari a n caratteri (occupa solo i byte effettivi del testo).
- **Tipi numerici**:
  - Interi: `SMALLINT`, `INTEGER`, `BIGINT`.
  - Numeri con virgola fissa o esatti: `NUMERIC(precisione, scala)` o `DECIMAL(p, s)`, ideali per importi monetari e contabilità.
  - Numeri in virgola mobile approssimati: `FLOAT`, `REAL`, `DOUBLE PRECISION`.
- **Date e intervalli temporali**:
  - `DATE`: memorizza anno, mese e giorno (es. `'2026-10-18'`).
  - `TIME`: memorizza ore, minuti e secondi.
  - `INTERVAL`: rappresenta una durata temporale (es. 30 giorni).
- **Estensioni di SQL-3**:
  - `BOOLEAN`: ammette i valori di verità `TRUE`, `FALSE` e l'indeterminato `UNKNOWN`.
  - `BLOB` (*Binary Large Object*) e `CLOB` (*Character Large Object*): pensati per memorizzare grandi volumi di dati binari non strutturati (immagini, video) o lunghi testi documentali.

I Domini Personalizzati (`CREATE DOMAIN`). Spesso i tipi elementari sono troppo generici per modellare i vincoli del mondo reale. Per evitare di riscrivere le medesime condizioni su molte colonne, SQL consente di definire tipi di dato personalizzati (**domini**):

```
1 CREATE DOMAIN VotoAccademico AS SMALLINT
2     DEFAULT NULL
3     CHECK (VALUE >= 18 AND VALUE <= 30);
```

Una volta definito, `VotoAccademico` può essere usato come tipo in qualsiasi tabella: il controllo di validità compreso tra 18 e 30 sarà applicato in modo uniforme e centralizzato dal DBMS.
Collocazione Sintattica dei Vincoli. I vincoli possono essere dichiarati in due modalità equivalenti:

- **Vincolo di colonna (in-line)**: scritto immediatamente a fianco della definizione dell'attributo. Si usa se il vincolo riguarda quel singolo campo.
- **Vincolo di tabella (stand-alone)**: scritto come elemento separato dopo la lista degli attributi. È obbligatorio quando il vincolo coinvolge due o più colonne insieme.

Le Clausole di Integrità Principali.

- `NOT NULL`: impedisce che l'attributo assuma il valore nullo. Si definisce sempre come vincolo di colonna.
- `UNIQUE`: impone che non esistano due righe con lo stesso valore non nullo per gli attributi indicati. Definisce una chiave candidata.
- `PRIMARY KEY`: specifica la chiave primaria della tabella. Implica sia l'unicità sia l'assoluto divieto di valori `NULL`. Può esistere **una sola** chiave primaria per relazione.
- `CHECK (condizione)`: impone un predicato booleano che ogni riga deve soddisfare per essere inserita o modificata.

Le due seguenti formulazioni per la chiave primaria sono del tutto equivalenti quando la chiave è composta da un solo attributo:

```
-- Sintassi a livello di colonna:
Matricola CHAR(6) PRIMARY KEY

-- Sintassi a livello di tabella:
Matricola CHAR(6),
PRIMARY KEY (Matricola)
```

Attenzione: Unicità Singola vs Unicità Composita

Esiste una differenza cruciale tra dichiarare il vincolo `UNIQUE` sulle singole colonne e dichiararlo su una tupla di colonne:

```
-- Caso 1: Vincolo composito di tabella
Nome    CHAR(20) NOT NULL,
Cognome CHAR(20) NOT NULL,
UNIQUE (Cognome, Nome)
```

Significato: la combinazione di cognome e nome deve essere univoca. Possono coesistere due persone che si chiamano "Mario Rossi" e "Paolo Rossi" (stesso cognome) o "Mario Bianchi" (stesso nome).

```
-- Caso 2: Due vincoli separati a livello di colonna
Nome    CHAR(20) NOT NULL UNIQUE,
```

```
3 Cognome CHAR(20) NOT NULL UNIQUE
```

Significato: in tutta la tabella non potrà **mai** ripetersi il nome "Mario" (anche con cognome diverso), né potrà mai ripetersi il cognome "Rossi". Sono due vincoli completamente differenti.

4.1.3 Integrità Referenziale ed Evoluzione dello Schema

Definizione delle Chiavi Esterne. Il vincolo di integrità referenziale stabilisce che i valori presenti in una o più colonne di una tabella (*tabella referenziante*) debbano corrispondere a valori già esistenti nella chiave primaria di un'altra tabella (*tabella referenziata*).

```
CREATE TABLE Infrazioni (
    CodiceVerbale CHAR(6) PRIMARY KEY,
    Giorno        DATE NOT NULL,
    Agente        INTEGER NOT NULL REFERENCES Vigile(Matricola),
    StatoAuto     CHAR(2),
    TargaAuto     CHAR(7),
    FOREIGN KEY (StatoAuto, TargaAuto)
        REFERENCES Auto(Stato, Targa)
);
```

In questo esempio:

- L'attributo singolo `Agente` referenzia direttamente la chiave primaria della tabella `Vigile` (sintassi in-line).
- La coppia di attributi (`StatoAuto`, `TargaAuto`) referenzia la chiave primaria composita della tabella `Auto` (sintassi di tabella).

Politiche di Reazione alle Violazioni. Se un utente tenta di **cancellare** o **aggiornare** un record genitore che ha delle righe collegate nella tabella figlia, il DBMS interviene applicando una politica definita con le clausole:

```
ON [DELETE | UPDATE] [CASCADE | SET NULL | SET DEFAULT | NO ACTION]
```

Azione	Effetto in Caso di DELETE	Effetto in Caso di UPDATE
CASCADE	Elimina automaticamente tutte le righe figlie collegate al record rimosso.	Aggiorna la chiave esterna nelle righe figlie con il nuovo valore del padre.
SET NULL	Pone a `NULL` il campo della chiave esterna nelle righe figlie (richiede che il campo ammetta NULL).	Pone a `NULL` il campo della chiave esterna nelle righe figlie.
SET DEFAULT	Imposta la chiave esterna al valore predefinito definito nello schema per quella colonna.	Imposta la chiave esterna al valore predefinito definito nello schema.
NO ACTION	**Blocca l'operazione**: la cancellazione del padre è rifiutata se esistono figli collegati.	**Blocca l'operazione**: l'aggiornamento del padre è rifiutato se esistono figli collegati.

Esempio di dichiarazione completa con politiche reattive:

```
CREATE TABLE Infrazioni (
    CodiceVerbale CHAR(6) PRIMARY KEY,
    Giorno        DATE NOT NULL,
    Agente        INTEGER REFERENCES Vigile(Matricola)
                  ON DELETE SET NULL
                  ON UPDATE CASCADE
);
```

Se un agente viene rimosso dal personale, i suoi verbali passano ad agente `NULL`; se la sua matricola viene modificata, i verbali vengono aggiornati a cascata.

Modifica Dinamica dello Schema. Non è quasi mai possibile prevedere a priori tutte le necessità future di una base di dati. SQL mette a disposizione istruzioni DDL per alterare strutture preesistenti senza perdere i dati già registrati.

Modifica dei Domini (`ALTER DOMAIN`). Consente di modificare valori di default o vincoli su un dominio:

```
-- Impostare un nuovo valore predefinito:
ALTER DOMAIN VotoAccademico SET DEFAULT 30;

-- Rimuovere il valore predefinito:
ALTER DOMAIN VotoAccademico DROP DEFAULT;

-- Aggiungere un vincolo con un nome specifico:
ALTER DOMAIN VotoAccademico
    ADD CONSTRAINT ControlloLode CHECK (VALUE <= 30);

-- Rimuovere un vincolo esistente:
ALTER DOMAIN VotoAccademico DROP CONSTRAINT ControlloLode;
```

Modifica delle Tabelle (`ALTER TABLE`). Permette di aggiungere colonne, modificare attributi o gestire i vincoli:

```
-- Aggiungere una nuova colonna allo schema:
ALTER TABLE Impiegato ADD COLUMN LivelloQualifica VARCHAR(10);

-- Modificare una colonna esistente (es. imporre NOT NULL):
ALTER TABLE Impiegato ALTER COLUMN Matricola SET NOT NULL;

-- Aggiungere un vincolo di integrita con nome:
ALTER TABLE Impiegato
    ADD CONSTRAINT LunghMatricola CHECK (CHAR_LENGTH(Matricola) = 6);

-- Rimuovere un vincolo:
ALTER TABLE Impiegato DROP CONSTRAINT LunghMatricola;
```

Eliminazione di Colonne: RESTRICT vs CASCADE. Quando si cancella una colonna da una tabella con `DROP COLUMN`, è fondamentale specificare l'impatto sugli altri oggetti del database:

```
-- Rimuove la colonna solo se nessun altro oggetto (viste, vincoli)
   dipende da essa:
```

```
ALTER TABLE Impiegato DROP COLUMN LivelloQualifica RESTRICT;

-- Rimuove la colonna ed elimina a cascata tutte le viste o i vincoli
   correlati:
ALTER TABLE Impiegato DROP COLUMN LivelloQualifica CASCADE;
```

Eliminazione Definitiva: Comandi `DROP`. Per cancellare definitivamente uno schema, un dominio o un'intera tabella:

```
-- Elimina il dominio se non piu utilizzato da alcuna colonna:
DROP DOMAIN VotoAccademico;

-- Rimuove interamente la tabella: cancella sia lo schema sia tutte le
   tuple contenute:
DROP TABLE Infrazioni;
```

Indici Fisici (`CREATE INDEX`). Per velocizzare le operazioni di ricerca e join, il DDL consente di definire indici su uno o più attributi:

```
CREATE INDEX idx_CognomeAgente ON Vigile (Cognome);
```

L'indice opera a **livello fisico**: non altera la semantica logica dei dati né il risultato delle query, ma riduce i tempi di accesso al disco da una scansione sequenziale $\mathcal{O}(n)$ a una ricerca su albero $\mathcal{O}(\log n)$.

Il DDL nella Pratica Professionale. Nei progetti industriali moderni, sebbene la progettazione avvenga spesso attraverso strumenti grafici (GUI come pgAdmin, MySQL Workbench, DBeaver) o tramite migrazioni automatiche nei framework software (ORM), la comprensione puntuale del DDL testuale resta fondamentale per garantire il controllo e l'ottimizzazione dell'architettura dati.

4.2 Interrogazioni in SQL

4.2.1 Il Comando SELECT: Struttura, Proiezione e Selezione

Natura Dichiarativa delle Interrogazioni. A differenza dell'algebra relazionale, che richiede di specificare l'esatta sequenza procedurale di operazioni per ricavare un risultato, SQL adotta un approccio prevalentemente **dichiarativo**: chi interroga il sistema specifica *cosa* intende estrarre, lasciando al motore interno del DBMS il compito di stabilire *come* procedere.

Sintassi Fondamentale. L'istruzione per l'interrogazione dei dati è il blocco `SELECT-FROM-WHERE`:

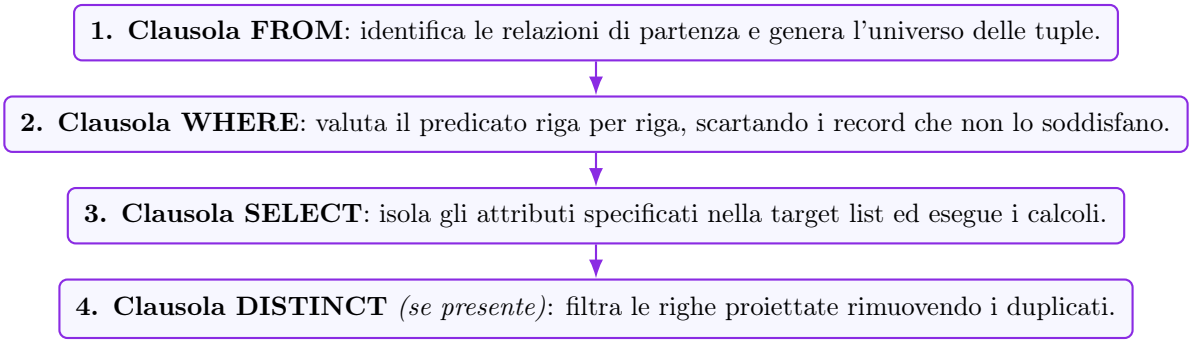
```
SELECT <ListaAttributi>
FROM   <ListaTabelle>
[ WHERE  <Condizione> ];
```

Le tre clausole assolvono compiti distinti:

- `SELECT` (*Target List*): specifica quali colonne devono comparire nel risultato finale (corrisponde concettualmente alla proiezione π).
- `FROM`: indica le tabelle sorgente da cui prelevare i record. Se sono presenti più relazioni, ne determina il prodotto cartesiano o il join.
- `WHERE` (*opzionale*): definisce il predicato logico di filtraggio che ogni tupla combinata deve soddisfare (corrisponde alla selezione σ).

Ordine Lessicale vs Ordine Semantico di Esecuzione. Un errore comune consiste nel confondere l'ordine in cui si scrive la query con l'ordine con cui il motore la elabora concettualmente. Sebbene la parola chiave `SELECT` compaia per prima nella stringa, essa viene valutata logicamente **dopo** le altre clausole:

```
3 SELECT Matricola, Nome
1 FROM   Agente
2 WHERE  Cognome = 'Rossi';
```



L'Ottimizzatore di Interrogazioni (*Query Planner*). Sapere che concettualmente la clausola `FROM` calcola un prodotto cartesiano non significa che il DBMS lo esegua davvero fisicamente. I DBMS moderni integrano un **ottimizzatore di query**:

- Le selezioni (`WHERE`) vengono spinte ed eseguite il più presto possibile (*push-down*), riducendo le dimensioni dei dati intermedi prima di effettuare confronti complessi.
- I prodotti cartesiani associati a filtri di uguaglianza vengono commutati in algoritmi di join efficienti (sfruttando indici fisici, tabelle hash o ordinamenti preventivi).
- L'utente non deve preoccuparsi di "scrivere una query efficiente a basso livello", ma deve concentrarsi sulla **leggibilità** e sulla **correttezza semantica** della formulazione logica.

Per comprendere operativamente il comportamento di SQL, faremo riferimento a una relazione campione denominata **Persone**:

Relazione Persone			
Nome	**Cognome**	**Età**	**Reddito**
Mario	Rossi	27	21
Giacomo	Bianchi	25	15
Luisa	Verdi	55	42
Filippo	Neri	26	30
Luigi	Ferrari	50	40
Stefano	Rossi	85	35

Selezione Pura. Equivale all'operatore di selezione dell'algebra relazionale $\sigma_{\text{Età}<30}(\text{Persone})$. Tutte le colonne vengono mantenute inalterate:

```sql
SELECT *
FROM   Persone
WHERE  Eta < 30;
```

Il simbolo asterisco (*) è una scorciatoia sintattica per indicare la totalità degli attributi definiti nello schema della relazione sorgente.

Proiezione Pura. Corrisponde all'operatore di proiezione $\pi_{\text{Nome,Reddito}}(\text{Persone})$. Non viene imposta alcuna clausola di filtraggio:

```sql
SELECT Nome, Reddito
FROM   Persone;
```

Composizione di Selezione e Proiezione. Quando si estraggono solo determinati campi da un sottoinsieme mirato di tuple, si realizza la combinazione $\pi_{\text{Nome,Reddito}}\big(\sigma_{\text{Età}<30}(\text{Persone})\big)$:

```sql
SELECT Nome, Reddito
FROM   Persone
WHERE  Eta < 30;
```

Risultato Proiezione e Selezione

Nome	Reddito
Mario	21
Giacomo	15
Filippo	30

Ridenominazione con la Parola Chiave `AS`. È possibile rinominare le colonne del risultato o assegnare un alias compatto alle tabelle nella clausola `FROM`:

```sql
SELECT p.Nome     AS NomeDipendente,
       p.Reddito  AS CompensoAnnuo
FROM   Persone AS p
WHERE  p.Eta < 30;
```

L'assegnazione di alias tramite `AS` non muta lo schema salvato su disco; si limita a etichettare le colonne nella vista generata a video, facilitando la lettura o evitando ambiguità quando più tabelle condividono attributi con lo stesso nome.

Espressioni di Filtraggio Composte. All'interno della clausola `WHERE` i confronti atomici ($=, <>, <, >, <=, >=$) possono essere concatenati mediante i connettivi logici `AND`, `OR` e `NOT`. L'uso delle parentesi tonde è fondamentale per esplicitare l'ordine di valutazione ed evitare ambiguità di precedenza:

```sql
SELECT *
FROM   Persone
WHERE  Reddito > 25
  AND (Eta < 30 OR Eta > 60);
```

Questa interrogazione estrae le persone con reddito superiore a 25 che siano contemporaneamente o molto giovani (sotto i 30 anni) oppure anziane (sopra i 60).

Pattern Matching sulle Stringhe: Il Predicato `LIKE`. Per effettuare ricerche flessibili su colonne testuali senza richiedere un'uguaglianza esatta, SQL fornisce l'operatore `LIKE`, che riconosce due caratteri speciali (*wildcard*):

- **Trattino basso (_)**: rappresenta esattamente un singolo carattere arbitrario.
- **Percentuale (%)**: rappresenta una sequenza qualsiasi di caratteri (compresa la stringa vuota di zero caratteri).

```
-- Cerca chi ha un nome che inizia per 'J',
-- ha un carattere intermedio qualunque, seguito da 'm' e poi altri
   caratteri:
SELECT *
FROM   Persone
WHERE  Nome LIKE 'J_m%';
```

Questo pattern identifica con successo nomi come `Jim` o `James`, mentre scarterebbe `John` o `Jack`.

Gestione del Valore Mancante: `IS NULL` e `IS NOT NULL`. Poiché il valore speciale `NULL` rappresenta l'assenza di dato (o un'informazione sconosciuta), non appartiene al dominio ordinario dei valori. Di conseguenza:

- Il confronto convenzionale `Età = NULL` oppure `Età <> NULL` non è valido: in SQL restituisce sempre `UNKNOWN` (indeterminato), con il risultato che la riga viene sistematicamente scartata.
- Per verificare formalmente l'assenza o la presenza del dato si devono impiegare unicamente i costrutti dedicati `IS NULL` e `IS NOT NULL`.

```
-- Dipendenti con eta superiore a 40 anni oppure con eta ignota:
SELECT *
FROM   Dipendenti
WHERE  Eta > 40
   OR  Eta IS NULL;
```

Insiemi vs Multinsiemi (*Bags*). Nella teoria pura dell'algebra relazionale, ogni relazione è formalizzata come un insieme matematico di tuple: non possono mai esistere due righe duplicate all'interno del medesimo risultato.

In SQL commerciale, per motivi di pura **efficienza computazionale**, la clausola `SELECT` opera per impostazione predefinita su **multinsiemi**: se una query proietta solo alcuni attributi e più righe assumono gli stessi valori, tali righe vengono restituite duplicate tante volte quante figuravano nell'origine, poiché eliminare i duplicati richiederebbe un costoso algoritmo di ordinamento o hashing su disco.

Controllo dei Duplicati: Clausola DISTINCT

Per imporre che il risultato contenga unicamente tuple distinte (ristabilendo la semantica dell'algebra relazionale), si posiziona la parola chiave `DISTINCT` subito dopo `SELECT`:

```
SELECT DISTINCT Cognome, Filiale
FROM   Dipendenti;
```

Se `DISTINCT` viene omesso (comportamento equivalente alla parola chiave implicita `ALL`), eventuali righe identiche vengono preservate nel risultato.

Output con `SELECT` predefinito

Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma
Rossi	Roma

Output con `SELECT DISTINCT`

Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma

4.2.2 Interrogazioni su Più Relazioni e Join

Prodotto Cartesiano e Selezione. Quando l'informazione desiderata è distribuita su più tabelle, è sufficiente elencare i rispettivi nomi separandoli da virgole all'interno della clausola `FROM`. In assenza di filtri, questa formulazione produce il pieno **prodotto cartesiano**:

```
SELECT *
FROM   TabellaA, TabellaB;
```

Se `TabellaA` possiede N righe e `TabellaB` contiene M righe, il risultato restituirà $N \times M$ tuple complessive.

Il Join Implicito in SQL. Per collegare i dati coerentemente, si affianca all'elenco delle tabelle nella clausola `FROM` un predicato di uguaglianza inserito nella clausola `WHERE`. Questa combinazione traduce l'equi-join o join naturale dell'algebra:

```
SELECT DISTINCT R1.A1, R2.A4
FROM   R1, R2
WHERE  R1.A2 = R2.A3;
```

Il legame formale tra le due notazioni si scompone in tre fasi sequenziali:

1. **FROM** `R1, R2`: calcola il prodotto cartesiano $R_1 \times R_2$.
2. **WHERE** `R1.A2 = R2.A3`: applica la selezione $\sigma_{R_1.A2=R_2.A3}$.
3. **SELECT DISTINCT** `R1.A1, R2.A4`: conclude con la proiezione $\pi_{A1,A4}$.

L'espressione algebrica globale equivalente è quindi:

$$\pi_{A_1,A_4}\big(\sigma_{A_2=A_3}(R_1 \times R_2)\big)$$

Lavorare con Relazioni Ricorsive o Replicate. Quando una query richiede di confrontare una riga con altre righe della **stessa tabella**, occorre includere la tabella più volte nella clausola `FROM`. Per evitare ambiguità, è obbligatorio assegnare alle singole istanze nomi di variabile distinti (*alias di correlazione*).

Si considerino due relazioni: **Persone**(Nome, Età, Reddito) e **Paternità**(Padre, Figlio), dove `Padre` e `Figlio` fanno riferimento a nomi registrati in `Persone`.

Esempio 1: Padri di persone che guadagnano più di 20. Vogliamo estrarre i nomi dei padri i cui figli hanno un reddito superiore a 20. In algebra relazionale:

$$\pi_{\text{Padre}}\big(\text{Paternità} \bowtie_{\text{Figlio=Nome}} \sigma_{\text{Reddito}>20}(\text{Persone})\big)$$

In SQL, combinando `Persone` e `Paternità`:

```sql
SELECT DISTINCT p.Padre
FROM   Persone pe, Paternita p
WHERE  p.Figlio = pe.Nome
  AND  pe.Reddito > 20;
```

Esempio 2: Individui che guadagnano più del proprio padre. La query richiede di visualizzare il nome della persona, il suo reddito e il reddito di suo padre, selezionando solo i casi in cui il reddito del figlio sia strettamente maggiore di quello paterno.

Per risolvere l'interrogazione, la relazione **Persone** deve essere consultata due volte con ruoli distinti:

- Un'istanza con alias `c` (*child*, il figlio).
- Un'istanza con alias `f` (*father*, il padre).

```sql
SELECT c.Nome     AS NomeFiglio,
       c.Reddito  AS RedditoFiglio,
       f.Reddito  AS RedditoPadre
FROM   Persone f,
       Paternita pat,
       Persone c
WHERE  f.Nome = pat.Padre
  AND  pat.Figlio = c.Nome
  AND  c.Reddito > f.Reddito;
```

In algebra relazionale pura questa interrogazione risulterebbe estremamente complessa da formulare e leggere, poiché richiederebbe la ridenominazione preventiva di tutti gli attributi del padre per evitare conflitti di collisione sui nomi di colonna:

$$\pi_{\text{Nome,Reddito,RedditoP}}\Big(\sigma_{\text{Reddito}>\text{RedditoP}}\big(\rho_{\dots}(\text{Persone}) \bowtie \text{Paternità} \bowtie \text{Persone}\big)\Big)$$

La notazione dichiarativa con alias di SQL (`f` e `c`) consente invece di esprimere il vincolo di parentela e il confronto tra redditi in modo pulito, naturale e privo di ambiguità.

Calcoli ed Espressioni nella Clausola SELECT. La target list di una query non è limitata alla semplice proiezione di colonne memorizzate: è possibile inserire espressioni aritmetiche, manipolazioni di stringhe o costanti. I valori calcolati vengono generati al volo durante l'interrogazione senza alterare i dati su disco:

```sql
-- Calcola la meta del reddito per una specifica persona:
SELECT Reddito / 2 AS RedditoDimezzato
FROM   Persone
WHERE  Nome = 'Luigi';
```

L'assegnazione di un alias con `AS` è caldamente consigliata per assegnare un nome significativo alla colonna risultante, che altrimenti verrebbe etichettata dal motore con il testo dell'espressione o lasciata anonima.

Dal Join Implicito al Join Esplicito. Nelle versioni iniziali di SQL, il join tra tabelle veniva espresso elencando le relazioni nella clausola `FROM` e specificando la condizione di accoppiamento nel `WHERE` (*join implicito*). Lo standard SQL-92 ha introdotto una sintassi dedicata ed esplicita per separare nettamente le condizioni di collegamento tra tabelle dai filtri sulle righe:

```sql
-- Sintassi implicita (tradizionale):
SELECT m.Madre, f.Figlio, f.Padre
FROM   Maternita m, Paternita f
WHERE  m.Figlio = f.Figlio;

-- Sintassi esplicita (standard SQL-92):
SELECT m.Madre, f.Figlio, f.Padre
FROM   Maternita m
       JOIN Paternita f ON m.Figlio = f.Figlio;
```

La sintassi formale generale del blocco con join esplicito è:

```sql
SELECT <ListaAttributi>
FROM   TabellaSinistra [INNER] JOIN TabellaDestra
       ON <CondizioneDiJoin>
[ WHERE  <AltriFiltri> ];
```

La parola chiave `INNER` è opzionale: se si scrive semplicemente `JOIN`, il DBMS assume per impostazione predefinita un inner join.

Natural Join (`NATURAL JOIN`). Se due tabelle condividono attributi con lo stesso identico nome, è possibile ricorrere al **join naturale**. Il DBMS individua in automatico le colonne omonime, impone l'uguaglianza tra di esse ed elimina le colonne duplicate dal risultato:

```sql
SELECT Madre, Figlio, Padre
FROM   Maternita NATURAL JOIN Paternita;
```

Nota pratica: sebbene compatto, l'uso del `NATURAL JOIN` nella programmazione reale è spesso sconsigliato, poiché se in futuro viene aggiunta a una delle due tabelle una colonna con un nome coincidente per puro caso, il join cambierà comportamento in modo silenzioso e inatteso.

Il Limite dell'Inner Join. Nell'inner join convenzionale, se una tupla non trova alcuna corrispondenza nella tabella con cui viene combinata, essa viene scartata dal risultato finale. Questo comportamento comporta una perdita di informazione quando si desidera visualizzare l'intero elenco di un'entità, indipendentemente dal fatto che abbia o meno record associati.

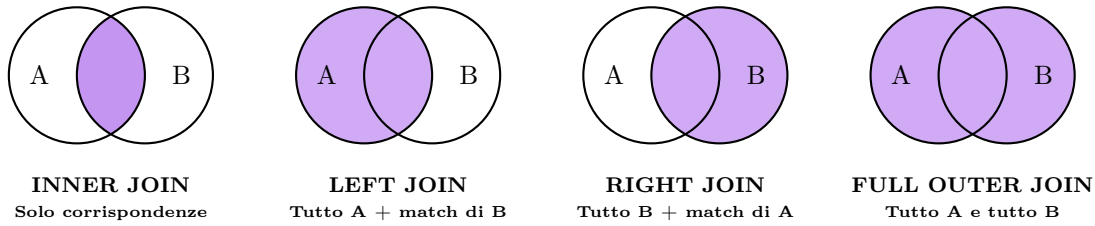
Le Varianti di Outer Join. L'**outer join** supera questo limite: mantiene nel risultato anche le tuple che non soddisfano la condizione di join, riempiendo con valori `NULL` le colonne della tabella associata mancante. Ne esistono tre declinazioni:

- **LEFT [OUTER] JOIN**: mantiene **tutte** le tuple della tabella di sinistra (primo operando). Se non c'è corrispondenza a destra, gli attributi di destra assumono valore `NULL`.
- **RIGHT [OUTER] JOIN**: mantiene **tutte** le tuple della tabella di destra (secondo operando). Se manca l'abbinamento a sinistra, compaiono valori `NULL`.
- **FULL [OUTER] JOIN**: mantiene **tutte** le tuple di entrambe le tabelle, accoppiando dove possibile e inserendo `NULL` ovunque manchi la corrispondenza (unione delle due strategie precedenti).

```sql
-- Estrae tutti i figli con il padre noto e, se presente, anche la madre:
SELECT f.Figlio, f.Padre, m.Madre
FROM   Paternita f
       LEFT OUTER JOIN Maternita m ON f.Figlio = m.Figlio;
```

Risultato di un LEFT JOIN tra Paternità e Maternità			
Figlio	**Padre**	**Madre**	
Giacomo	Stefano	*NULL*	(madre non registrata)
Filippo	Luigi	Luisa	
Mario	Franco	Anna	

Rappresentazione Insimeistica dei Join. La figura seguente sintetizza visivamente la differenza semantica tra le tipologie fondamentali di join:



Filtraggio delle Sole Tuple Orfane (Anti-Join). Combinando un outer join con una clausola `WHERE ... IS NULL`, è possibile isolare con precisione chirurgica i record di una tabella che **non hanno alcun riscontro** nell'altra:

```
-- Persone registrate che non risultano essere padri di nessuno:
SELECT p.Nome
FROM   Persone p
       LEFT JOIN Paternita pat ON p.Nome = pat.Padre
WHERE  pat.Padre IS NULL;
```

4.2.3 Ordinamento e Operatori Insiemistici

Assenza di Ordine Intrinseco. In base alla teoria delle relazioni, il risultato di un'interrogazione è un insieme non ordinato di tuple. L'ordine con cui le righe appaiono a schermo dopo una `SELECT` non è mai garantito dal DBMS, a meno che non venga richiesto esplicitamente mediante la clausola `ORDER BY`.

Sintassi e Direzione dell'Ordinamento. La clausola `ORDER BY` va posizionata come ultimo elemento dell'istruzione:

```
SELECT Nome, Reddito
FROM   Persone
WHERE  Eta <= 30
ORDER BY Nome ASC;
```

- `ASC` (crescente, *ascending*): ordina dal valore più piccolo al più grande (dalla A alla Z per stringhe, dal più piccolo al più grande per numeri). Nello standard SQL è il comportamento predefinito se omesso.
- `DESC` (decrescente, *descending*): inverte l'ordine (dalla Z alla A, o dal numero maggiore al minore).

Ordinamento Crescente (`ORDER BY Nome ASC`)

Nome	Reddito
Filippo	30
Giacomo	15
Mario	21

Ordinamento Decrescente (`ORDER BY Nome DESC`)

Nome	Reddito
Mario	21
Giacomo	15
Filippo	30

È possibile ordinare su più attributi in cascata specificando una lista separata da virgole: le tuple vengono ordinate in base al primo campo; a parità di valore, si valuta il secondo, e così via:

```
ORDER BY Cognome ASC, Reddito DESC;
```

Compatibilità tra Schemi e Notazione Posizionale. SQL mette a disposizione i tre classici operatori insiemistici per combinare i risultati di due blocchi `SELECT`:

- `UNION`: unione insiemistica.
- `INTERSECT`: intersezione insiemistica.
- `EXCEPT` (denominato `MINUS` in dialetti come Oracle): differenza insiemistica.

A differenza del modello relazionale matematico in cui i vincoli dipendono dai nomi degli attributi, gli operatori insiemistici in SQL adottano una **notazione rigidamente posizionale**:

1. Le due query devono proiettare lo **stesso numero di colonne**.
2. Le colonne corrispondenti nella sequenza (prima con prima, seconda con seconda) devono avere **tipi di dato tra loro compatibili**.
3. I nomi delle colonne nel risultato finale vengono ereditati automaticamente dalla **prima query**.

Eliminazione dei Duplicati vs Conservazione: Clausola `ALL`. Di base, `UNION`, `INTERSECT` ed `EXCEPT` operano in modalità insiemistica pura: **eliminano i duplicati** dal risultato finale. Se si desidera preservare le righe duplicate (lavorando su multinsiemi per risparmiare tempo di calcolo), si aggiunge la clausola `ALL`:

```
-- Unione insiemistica (ogni figlio compare una sola volta):
SELECT Figlio FROM Maternita
UNION
SELECT Figlio FROM Paternita;

-- Unione su multinsieme (un figlio compare due volte se ha madre e padre noti):
SELECT Figlio FROM Maternita
UNION ALL
SELECT Figlio FROM Paternita;
```

Differenza ed Intersezione: Esempi.

```sql
-- Dipendenti il cui nome NON compare tra i cognomi aziendali:
SELECT Nome FROM Impiegato
EXCEPT
SELECT Cognome AS Nome FROM Impiegato;

-- Dipendenti il cui nome coincide con un cognome registrato:
SELECT Nome FROM Impiegato
INTERSECT
SELECT Cognome AS Nome FROM Impiegato;
```

L'operatore `INTERSECT` può essere sempre riscritto tramite un equi-join equivalente della tabella con se stessa, mentre l'operazione `EXCEPT` trova una traduzione alternativa e potente nelle query annidate con negazione.

4.2.4 Interrogazioni Annidate e Visibilità delle Variabili

Il Concetto di Sottoquery. Una delle caratteristiche più rilevanti di SQL è la possibilità di inserire un blocco d'interrogazione completo (`SELECT-FROM-WHERE`) all'interno della clausola `WHERE` di un'altra query più esterna. Si parla in questo caso di **query annidata** (*subquery* o sottointerrogazione).

Le sottoquery rendono il codice più leggibile e modulare, evitando l'uso massiccio di join cartesiani ed eliminando la necessità di inventare continuamente alias di tabella quando si effettuano verifiche d'appartenenza.

Sottoquery Scalari (Confronto Elementare). Quando una sottoquery restituisce come output un'unica riga contenente un'unica colonna (un singolo valore o *scalare*), è possibile impiegare gli ordinari operatori di confronto ($=, <>, <, >, <=, >=$):

```sql
-- Trova nome e reddito del padre di 'Franco':
-- (Versione con equijoin implicito):
SELECT p.Nome, p.Reddito
FROM   Persone p, Paternita pat
WHERE  p.Nome = pat.Padre
  AND  pat.Figlio = 'Franco';

-- (Versione equivalente con sottoquery scalare):
SELECT Nome, Reddito
FROM   Persone
WHERE  Nome = (SELECT Padre
               FROM   Paternita
               WHERE  Figlio = 'Franco');
```

Attenzione: se la sottoquery tra parentesi dovesse restituire più di una riga (ad esempio nel caso di una persona registrata con due padri), il confronto scalare fallirebbe sollevando un errore a runtime.

Operatori su Insiemi: `IN`, `ANY` e `ALL`. Quando la sottoquery restituisce una colonna con zero, una o più righe, si ricorre a operatori insiemistici dedicati:

Costrutto	Condizione di Verità per la Tupla Esterna
`Attributo IN (Expr)`	Il valore dell'attributo è presente all'interno dell'insieme restituito dalla sottoquery (equivale a `= ANY`).
`Attributo NOT IN (Expr)`	Il valore dell'attributo è rigorosamente assente dall'insieme restituito.
`Attributo` θ `ANY (Expr)`	La condizione $A\ \theta\ x$ è vera per **almeno uno** degli elementi x restituiti da `Expr` ($\theta \in \{=, <>, <, >, <=, >=\}$).
`Attributo` θ `ALL (Expr)`	La condizione $A\ \theta\ x$ è verificata contemporaneamente per **tutti** gli elementi x restituiti da `Expr`.

Esempio Pratico con `IN` ed `ANY`. Vogliamo estrarre nome e reddito dei padri i cui figli guadagnano più di 20:

```
SELECT Nome, Reddito
FROM   Persone
WHERE  Nome IN (
    SELECT Padre
    FROM   Paternita
    WHERE  Figlio = ANY (
        SELECT Nome
        FROM   Persone
        WHERE  Reddito > 20
    )
);
```

In questa formulazione, a differenza di quanto accade con l'inner join, non è necessario applicare `DISTINCT` all'esterno: ogni padre compare una sola volta, anche qualora avesse più figli che soddisfano la condizione di reddito.

Predicati Esistenziali. Spesso una condizione non dipende dal valore specifico restituito da una colonna, ma dal semplice fatto che una certa proprietà ammetta o meno **almeno una tupla** di riscontro. Per questo scopo SQL fornisce il predicato:

```
EXISTS ( <Sottoquery> )
```

Il predicato `EXISTS` restituisce **Vero** se la sottoquery tra parentesi produce almeno un record; restituisce **Falso** se l'insieme risultante è vuoto. La target list della sottoquery è irrilevante (si usa convenzionalmente la dicitura `SELECT *`).

Sottoquery Correlate. Nella quasi totalità dei casi, `EXISTS` viene impiegato all'interno di **sottoquery correlate**: l'interrogazione interna fa riferimento a una colonna della query più esterna, venendo rivalutata logicamente riga per riga per ogni tupla esaminata dal blocco principale.

```
-- Persone che hanno almeno un figlio (padri o madri):
SELECT *
FROM   Persone p
WHERE  EXISTS (SELECT * FROM Paternita pat WHERE pat.Padre = p.Nome)
   OR  EXISTS (SELECT * FROM Maternita mat WHERE mat.Madre = p.Nome);
```

La Potenza di `NOT EXISTS` per Esprimere "TUTTI". Nella logica del primo ordine, asserire che "tutti gli elementi soddisfano una data proprietà" equivale a dire che "**non esiste** alcun elemento

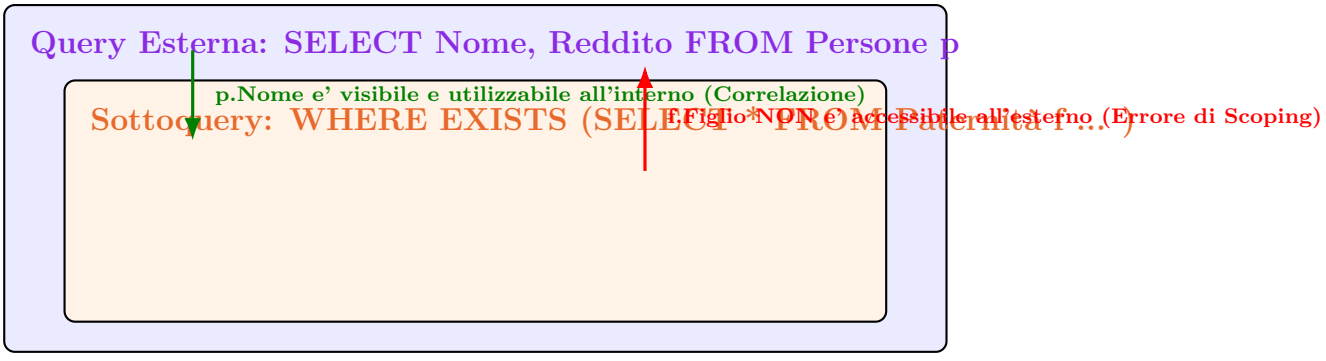
che violi la proprietà". SQL non possiede un quantificatore universale diretto; il costrutto `NOT EXISTS` rappresenta lo strumento standard per tradurre questo pattern:

```sql
-- Padri i cui figli guadagnano TUTTI piu di 20:
-- (Ovvero: padri per cui NON ESISTE un figlio che guadagna <= 20)
SELECT DISTINCT f.Padre
FROM   Paternita AS f
WHERE  NOT EXISTS (
    SELECT *
    FROM   Persone p
    WHERE  p.Nome = f.Figlio
      AND  p.Reddito <= 20
);
```

Regole di Scoping nei Blocchi Annidati

Il passaggio di parametri e la visibilità delle tabelle tra query concentriche segue regole rigorose analoghe a quelle dei linguaggi di programmazione a blocchi:

1. **Dall'interno verso l'esterno**: una sottoquery interna può leggere gli attributi e gli alias definiti nei blocchi che la contengono (variabili dell'ambiente esterno).
2. **Dall'esterno verso l'interno (Divieto)**: una query esterna **non può in alcun modo accedere** o proiettare attributi definiti esclusivamente nel `FROM` di una sottoquery interna.
3. **Risoluzione dei Nomi**: se un attributo viene citato senza specificare la tabella, il DBMS lo associa alla tabella più vicina dichiarata nel `FROM` locale (*nearest scope*).
4. **Invisibilità tra Fratelli**: due sottoquery parallele collegate da operatori logici (`AND`, `OR`) si trovano allo stesso livello gerarchico e **non possono condividere** le rispettive variabili locali.



Analisi di un Errore Comune di Scoping. Si consideri il seguente frammento SQL non valido:

```sql
-- Esempio Errato: violazione della visibilita tra sottoquery parallele
SELECT *
FROM   Impiegato
WHERE  Dipart IN (SELECT NomeDipart
```

```
                  FROM   Dipartimento D1
                  WHERE  NomeDipart = 'Produzione')
  OR  Dipart IN (SELECT NomeDipart
                  FROM   Dipartimento D2
                  WHERE  D2.Citta = D1.Citta); -- ERRORE!
```

La seconda sottoquery tenta di accedere alla colonna `D1.Città`. Poiché l'alias `D1` è stato istanziato all'interno del primo blocco parallelo, il suo ciclo di vita si è concluso alla chiusura della parentesi tonda. La variabile non è visibile al ramo fratello e il parser SQL rigetta la query con un errore di identificatore sconosciuto.

4.2.5 Aggregazione e Raggruppamento

Dal Calcolo su Singola Riga all'Aggregazione. Le espressioni viste finora operano orizzontalmente all'interno della medesima tupla. Nella maggior parte delle analisi informative è tuttavia necessario estrarre grandezze statistiche complessive (totali, conteggi, medie) calcolate verticalmente su insiemi di righe. SQL risponde a questa esigenza mediante le **funzioni aggregate** (*aggregate functions*):

- `COUNT`: calcola il numero di righe o di valori non nulli.
- `SUM`: calcola la somma aritmetica dei valori numerici.
- `AVG`: calcola la media aritmetica dei valori numerici.
- `MIN`: determina il valore minimo (funziona sia su numeri sia su date e stringhe).
- `MAX`: determina il valore massimo.

Sintassi Fondamentale. Le funzioni aggregate compaiono nella target list della clausola `SELECT` e adottano la seguente sintassi generale:

```
NomeFunzione( [DISTINCT] Attributo | * )
```

L'asterisco (`*`) è ammesso esclusivamente con la funzione `COUNT(*)` e serve a conteggiare il numero totale di tuple restituite dalla query, a prescindere dal loro contenuto.

Differenza Chiave: `COUNT(*)` vs `COUNT(Attributo)` vs `COUNT(DISTINCT)`. Si consideri la seguente istanza campione della relazione **Persone**:

Relazione Persone			
Nome	**Cognome**	**Età**	**Reddito**
Mario	Rossi	27	21
Giacomo	Bianchi	25	*NULL*
Luisa	Verdi	55	21
Filippo	Neri	50	35

Applicando le diverse forme di conteggio sulla colonna `Reddito`:

```sql
SELECT COUNT(*)                    AS ConteggioTotale,
       COUNT(Reddito)              AS ConteggioValidi,
       COUNT(DISTINCT Reddito)     AS ConteggioValoriDistinti
FROM   Persone;
```

- `COUNT(*)` = 4: conta tutte le tuple fisiche presenti, includendo anche la riga di Giacomo che contiene un valore nullo.
- `COUNT(Reddito)` = 3: valuta unicamente i valori noti della colonna, **ignorando i valori** `NULL` (prende in esame solo 21, 21 e 35).
- `COUNT(DISTINCT Reddito)` = 2: elimina preliminarmente le repliche e scarta i `NULL`, conteggiando unicamente i valori distinti (21 e 35).

Comportamento delle Funzioni Aggregate con i Valori NULL. La regola generale applicata da SQL per `SUM`, `AVG`, `MIN` e `MAX` prevede l'**eliminazione preliminare silenziosa di tutti i valori NULL** prima dell'esecuzione del calcolo:

```sql
SELECT AVG(Reddito) AS RedditoMedio
FROM   Persone;
```

Il DBMS calcola la media sommando i soli valori presenti ($21 + 21 + 35 = 77$) e dividendo per il numero di tuple valide (3):

$$\text{RedditoMedio} = \frac{77}{3} \approx 25{,}66$$

Il record con valore `NULL` viene del tutto ignorato (non viene conteggiato come zero al denominatore). Se la colonna contiene unicamente valori `NULL`, il risultato dell'aggregazione sarà a sua volta `NULL`.

Regola di Omogeneità nella Target List: Un Errore Frequente. Un errore concettuale molto comune consiste nel tentare di selezionare contemporaneamente un attributo scalare e una funzione di aggregazione globale:

```sql
-- QUERY ERRATA: genera un errore sintattico a runtime!
SELECT Nome, MAX(Reddito)
FROM   Persone;
```

Perché è errata? La funzione `MAX(Reddito)` comprime l'intera tabella in un'unica riga riassuntiva scalare, mentre l'attributo `Nome` produrrebbe tante righe quante sono le persone archiviate. Il DBMS non può sapere a quale nome associare il valore massimo calcolato.

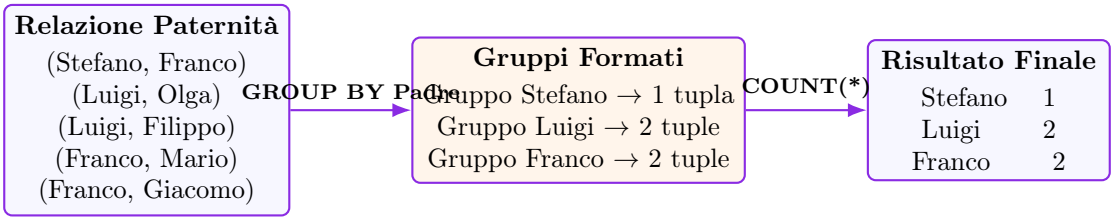
Per estrarre sia il dato scalare sia l'estremo cercato, occorre formulare una **sottoquery di confronto**:

```sql
-- FORMULAZIONE CORRETTA:
SELECT *
FROM   Persone
WHERE  Reddito = (SELECT MAX(Reddito) FROM Persone);
```

Necessità del Raggruppamento. Spesso non si desidera calcolare un aggregato sull'intera tabella, bensì partizionare i dati in sottoinsiemi omogenei (ad esempio calcolare il reddito medio per ciascuna città, o il numero di figli per ciascun padre). Questo comportamento si realizza con la clausola `GROUP BY`.

Sintassi ed Esempio Operativo. Data la tabella **Paternità**(Figlio, Padre):

```sql
SELECT   Padre, COUNT(*) AS NumeroFigli
FROM     Paternita
GROUP BY Padre;
```



Semantica di Valutazione del GROUP BY. Il processo concettuale seguito dal motore SQL si articola in due passi:

1. Le tuple che hanno superato i filtri della clausola `WHERE` vengono suddivise in partizioni disgiunte: tutte le righe che presentano gli stessi identici valori per gli attributi di raggruppamento confluiscono nel medesimo gruppo.

2. Le funzioni aggregate specificate nella `SELECT` vengono applicate **separatamente a ciascun gruppo**, producendo esattamente una singola riga di output per ogni partizione.

La Regola Aurea della Clausola GROUP BY

Se in un'interrogazione è presente la clausola `GROUP BY`, **ogni singolo attributo** che compare nella clausola `SELECT` deve soddisfare tassativamente almeno una di queste due condizioni:

- È racchiuso come argomento all'interno di una **funzione aggregata** (es. `COUNT`, `AVG`, `SUM`);
- Compare esplicitamente nell'elenco degli attributi del `GROUP BY`.

Qualsiasi colonna non aggregata lasciata fuori dal `GROUP BY` rende la query non valida, poiché il DBMS non può determinare in modo univoco quale valore estrarre qualora il gruppo contenga tuple con valori eterogenei per quel campo.

Raggruppamento e Valori NULL. Se una colonna impiegata nel `GROUP BY` contiene valori `NULL`, lo standard SQL stabilisce che **tutti i valori NULL collassano in un unico gruppo**:

Tabella R(A, B)

A	B
1	11
2	11
3	*NULL*
4	*NULL*

$\Longrightarrow$

Risultato della Query

`SELECT B, COUNT(*) FROM R GROUP BY B;`

B	COUNT(*)
11	2
NULL	2

Filtrare Gruppi Interi. La clausola `WHERE` non può filtrare i risultati delle funzioni aggregate, poiché viene valutata logicamente **prima** che i gruppi vengano formati. Per imporre vincoli sul risultato delle funzioni statistiche calcolate sui gruppi, SQL introduce la clausola `HAVING`.

Esempio di Utilizzo. Si consideri la richiesta: *«Estrarre i padri i cui figli percepiscono un reddito medio superiore a 25»*:

```
SELECT   p.Padre, AVG(pe.Reddito) AS RedditoMedioFigli
FROM     Persone pe
         JOIN Paternita p ON pe.Nome = p.Figlio
GROUP BY p.Padre
HAVING   AVG(pe.Reddito) > 25;
```

Confronto Diretto: WHERE vs HAVING. Comprendere la distinzione tra queste due clausole è fondamentale per formulare correttamente query complesse:

Caratteristica	Clausola WHERE	Clausola HAVING
Momento di Valutazione	Agisce a monte, **prima** della formazione dei gruppi.	Agisce a valle, **dopo** il raggruppamento e l'aggregazione.
Ambito di Filtraggio	Filtra le singole righe individuali della tabella.	Filtra intere partizioni o gruppi consolidati.
Uso di Aggregati	**Vietato**: non può contenere funzioni come `AVG()` o `COUNT()`.	**Ammesso**: valuta predicati basati sui risultati delle funzioni aggregate.

Uso Congiunto di WHERE ed HAVING. Le due clausole possono coesistere nella medesima istruzione. Si consideri la query: *«Fornire i padri i cui figli con meno di 30 anni hanno un reddito medio superiore a 20»*:

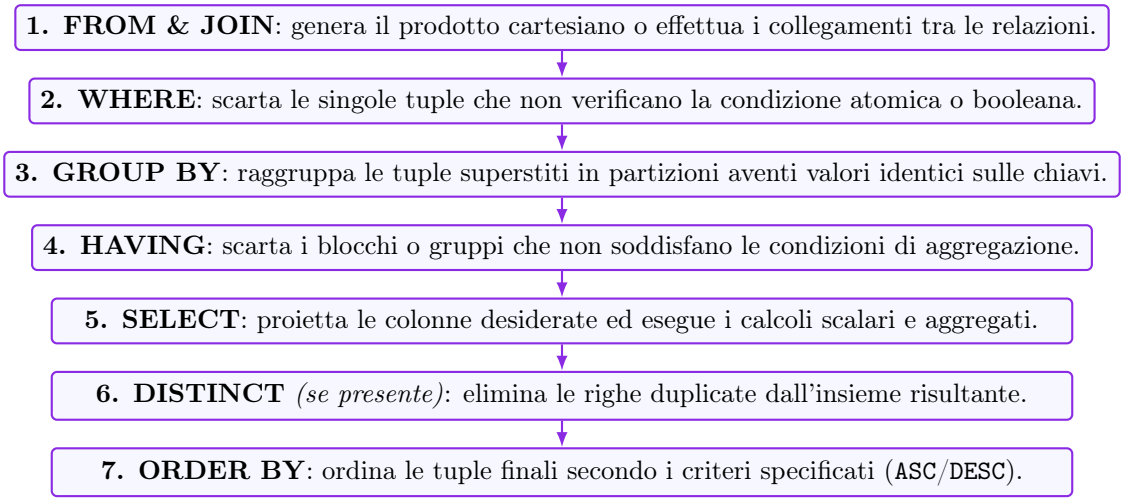
```
SELECT   p.Padre, AVG(pe.Reddito) AS RedditoMedioGiovani
FROM     Persone pe
         JOIN Paternita p ON pe.Nome = p.Figlio
WHERE    pe.Eta < 30
GROUP BY p.Padre
HAVING   AVG(pe.Reddito) > 20;
```

La clausola `WHERE pe.Eta < 30` scarta preventivamente tutti i figli trentenni o più anziani; il `GROUP BY` partiziona i soli figli giovani rimasti in base al padre; infine `HAVING` seleziona unicamente i padri la cui media calcolata supera la soglia di 20.

La Forma Canonica Generale. Riassumendo tutte le clausole esaminate, l'interrogazione SQL completa si struttura formalmente nella seguente sequenza:

```
SELECT   [DISTINCT] <ListaAttributi ed Espressioni di Aggregazione>
FROM     <ListaTabelle con eventuali costrutti JOIN>
[ WHERE    <Predicato di filtraggio sulle singole righe> ]
[ GROUP BY <ListaAttributi di raggruppamento> ]
[ HAVING   <Predicato di filtraggio sui gruppi calcolati> ]
[ ORDER BY <ListaAttributi di ordinamento> [ASC | DESC] ];
```

Il Pipeline Concettuale di Esecuzione. L'ordine logico secondo cui il motore relazionale elabora i dati non coincide con l'ordine di scrittura del testo, ma segue rigorosamente questa pipeline:



4.3 Operazioni di Modifica dei Dati (DML): INSERT, DELETE e UPDATE

Oltre alle interrogazioni in sola lettura, il linguaggio SQL prevede tre istruzioni operative fondamentali per manipolare lo stato estensionale della base di dati: inserimento (`INSERT`), cancellazione (`DELETE`) e aggiornamento (`UPDATE`).

1. Inserimento di Tuple: Il Comando `INSERT`. L'inserimento può avvenire specificando direttamente i valori della singola tupla (`VALUES`) oppure popolando la tabella in blocco a partire dal risultato di un'interrogazione (`SELECT`):

```
-- Forma base: specifica tutti i valori rispettando l'ordine dello schema:
INSERT INTO Persone
VALUES ('Giovanni', 'Rossi', 25, 32);

-- Forma con elenco attributi esplicito:
INSERT INTO Persone (Nome, Eta, Reddito)
VALUES ('Giacomo', 28, 40);

-- Inserimento parziale: gli attributi omessi assumono il valore DEFAULT o NULL:
INSERT INTO Persone (Nome, Reddito)
VALUES ('Roberto', 55);

-- Inserimento massivo derivato da una sottoquery:
INSERT INTO Persone (Nome)
SELECT Padre
FROM   Paternita
WHERE  Padre NOT IN (SELECT Nome FROM Persone);
```

Regole operative: se la lista delle colonne viene omessa, il comando presuppone che i valori siano forniti per tutti gli attributi nello stesso identico ordine stabilito durante la `CREATE TABLE`. Se invece viene specificata una lista parziale, tutti i campi esclusi devono obbligatoriamente ammettere il valore `NULL` o possedere una clausola `DEFAULT`, altrimenti il DBMS rigetta l'operazione.

2. Cancellazione di Tuple: Il Comando `DELETE`. Rimuove le righe da una tabella in base a un predicato logico:

```
-- Rimuove le persone con eta inferiore a 35 anni:
DELETE FROM Persone
WHERE  Eta < 35;

-- Cancellazione con condizione dipendente da un'altra tabella:
DELETE FROM Paternita
WHERE  Figlio NOT IN (SELECT Nome FROM Persone);

-- ATTENZIONE: cancellazione totale di tutte le tuple della tabella:
DELETE FROM Paternita;
```

Proprietà della cancellazione:

- Se la clausola `WHERE` viene omessa, essa viene intesa implicitamente come `WHERE TRUE`: **tutte le righe della tabella vengono eliminate**, lasciando lo schema vuoto ma intatto.
- L'eliminazione di un record genitore può scatenare la rimozione a catena di record figli se sulla relativa chiave esterna è configurata l'azione reattiva `ON DELETE CASCADE`.

3. Modifica di Tuple: Il Comando `UPDATE`. Altera il contenuto di una o più colonne nelle tuple che soddisfano il criterio di selezione:

```
-- Modifica puntuale di un attributo con valore costante:
UPDATE Persone
SET    Reddito = 45
WHERE  Nome = 'Roberto';

-- Aggiornamento dinamico tramite espressione aritmetica (+10% ai giovani
   ):
UPDATE Persone
SET    Reddito = Reddito * 1.1
WHERE  Eta < 30;

-- Assegnazione del valore mediante una sottoquery scalare correlata:
UPDATE Persone
SET    Reddito = (SELECT Reddito FROM Persone WHERE Nome = 'Mario')
WHERE  Nome = 'Giacomo';

-- Assegnazione al valore NULL o al valore DEFAULT definito nello schema:
UPDATE Persone
SET    Reddito = NULL
WHERE  Eta < 30;

UPDATE Persone
SET    Reddito = DEFAULT
WHERE  Eta < 30;
```

Anche per l'aggiornamento vale la regola di sicurezza: omettere la clausola `WHERE` comporterà l'applicazione della modifica specificata sull'intera totalità delle tuple presenti nella relazione.

5 SQL avanzato

5.1 Vincoli Complessi e Viste

5.1.1 Vincoli CHECK e Asserzioni

Definizione e Ambito. La clausola `CHECK` consente di specificare vincoli di integrità personalizzati su singole tuple (o combinazioni di attributi della stessa tabella), definendo condizioni booleane che ogni riga deve obbligatoriamente soddisfare:

```
CHECK ( Predicato )
```

Qualsiasi operazione di inserimento (`INSERT`) o aggiornamento (`UPDATE`) che renda il predicato `FALSE` viene bloccata dal DBMS. Se il predicato restituisce `UNKNOWN` (a causa di valori `NULL`), il vincolo si considera non violato per specifica dello standard SQL.

Controlli Aritmetici e di Dominio Intra-Tupla. Il caso d'uso più comune prevede confronti tra colonne appartenenti alla medesima tupla:

```
CREATE TABLE Impiegato (
    Matricola     CHAR(6) PRIMARY KEY,
    Cognome       VARCHAR(20) NOT NULL,
    Nome          VARCHAR(20) NOT NULL,
    Sesso         CHAR(1) NOT NULL CHECK (Sesso IN ('M', 'F')),
    Stipendio     INTEGER CHECK (Stipendio >= 0),
    Ritenute      INTEGER,
    Netto         INTEGER,
    Supervisore   CHAR(6),
    CHECK (Netto = Stipendio - Ritenute)
);
```

Listing 5: Vincoli CHECK intra-tupla

Verifica dei Vincoli CHECK: Esempi di Inserimento

Consideriamo tre tentativi di inserimento nella tabella precedente:

Istruzione INSERT	Motivazione Semantica	Esito
`(1, 'Doe', 'John', '', 100, 20, 80)`	`Sesso = ''` non appartiene al dominio `('M', 'F')`.	**Respinto**
`(2, 'Smith', 'Jimmy', 'M', 100, 10, 80)`	Errore aritmetico: $80 \neq 100 - 10 = 90$.	**Respinto**
`(3, 'Rossini', 'Luca', 'M', 70, 20, 50)`	$50 = 70 - 20$ e sesso valido.	**Accettato**

CHECK con Sottoquery e Limitazioni nei DBMS. Lo standard SQL ammette teoricamente predicati complessi contenenti sottoquery per confrontare la tupla con altri record della tabella:

```
CREATE TABLE Impiegato (
    Matricola     INTEGER PRIMARY KEY,
    Stipendio     INTEGER,
```

```
    Supervisore  INTEGER,
    CHECK (Stipendio <= (
        SELECT Stipendio
        FROM Impiegato J
        WHERE Supervisore = J.Matricola
    ))
);
```

Listing 6: CHECK con sottoquery (non supportato universalmente)

Nota di Supporto Reale sui Vincoli CHECK

L'inserimento di clausole `SELECT` all'interno di un vincolo `CHECK` **non è supportato da molti DBMS commerciali** (tra cui MySQL e PostgreSQL) per motivi di efficienza: richiederebbe l'esecuzione di una sottoquery a ogni singola scrittura. Nella pratica aziendale, controlli complessi tra tuple diverse si implementano tramite **Trigger**.

Definizione e Ruolo Globale. Quando una regola di integrità coinvolge più tabelle o l'intero stato della base di dati, i vincoli di tabella non sono sufficienti. Lo standard SQL-92 definisce le **Asserzioni** (`ASSERTION`): vincoli indipendenti definiti a livello di schema.

```
CREATE ASSERTION NomeAsserzione CHECK ( Predicato );
```

```
CREATE ASSERTION AlmenoUnImpiegato
CHECK (1 <= (SELECT COUNT(*) FROM Impiegato));
```

Listing 7: Esempio: garantire almeno un impiegato nel database

Confronto Operativo: CHECK vs ASSERTION

Proprietà	CHECK (di Tabella)	ASSERTION (di Schema)
Associazione	Legato a una specifica relazione.	Entità autonoma dello schema.
Validazione	Valutato solo su inserimenti/modifiche della tabella di appartenenza.	Valutato su qualunque modifica che possa alterare il predicato globale.
Supporto Reale	Ampiamente supportato per controlli intra-tupla.	**Quasi assente nei DBMS reali** per via del costo di computazione eccessivo.

5.1.2 Viste: Interrogazione, Aggiornamento e Annidamento

Natura delle Viste. Una **vista** è una relazione virtuale il cui contenuto non è memorizzato fisicamente su disco, ma è calcolato dinamicamente tramite un'interrogazione SQL eseguita al momento dell'accesso:

```
CREATE VIEW NomeVista [ ( ListaColonne ) ] AS
    QuerySELECT
    [ WITH [ LOCAL | CASCADED ] CHECK OPTION ];
```

```sql
CREATE VIEW ImpiegatiAmm (Nome, Cognome, Stipendio) AS
    SELECT Nome, Cognome, Stipendio
    FROM   Impiegato
    WHERE  Dipart = 'Amministrazione'
      AND  Stipendio > 10;
```

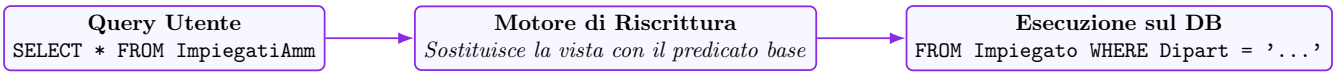
Listing 8: Creazione di una vista filtrata

Interrogazione Trasparente e Riscrittura. Le viste si interrogano esattamente come qualsiasi altra tabella reale:

```sql
SELECT *
FROM   ImpiegatiAmm;
```

Il DBMS riceve l'interrogazione e opera una **riscrittura logica trasparente**, integrando il corpo della vista nella query:

```sql
-- Riscrittura interna eseguita dal motore del DBMS:
SELECT Nome, Cognome, Stipendio
FROM   Impiegato
WHERE  Dipart = 'Amministrazione'
  AND  Stipendio > 10;
```



Aggiornabilità delle Viste. Le operazioni di modifica (`INSERT`, `UPDATE`, `DELETE`) sulle viste presentano ambiguità: devono riflettersi sulle tabelle sottostanti.

- **Viste Aggiornabili**: definite su una singola tabella di base, senza funzioni aggregate, `DISTINCT`, o clausole `GROUP BY`.
- **Problema di Inconsistenza**: un utente potrebbe inserire o aggiornare una riga assegnandole valori che non rispettano la clausola `WHERE` della vista, facendola "sparire" dalla vista stessa ma lasciandola registrata sul DB fisico.

```sql
CREATE VIEW ImpiegatiPoveri AS
    SELECT *
    FROM   ImpiegatiAmm
    WHERE  Stipendio < 50
    WITH CHECK OPTION;
```

Listing 9: Vista definita con WITH CHECK OPTION

Il Ruolo di `WITH CHECK OPTION`. Impedisce qualsiasi operazione di `INSERT` o `UPDATE` che produca tuple non conformi ai criteri di visibilità della vista:

```sql
-- Operazione bloccata dal DBMS:
UPDATE ImpiegatiPoveri
SET    Stipendio = 60
WHERE  Nome = 'Anna';
-- ERRORE: 60 viola il vincolo (Stipendio < 50) imposto dal CHECK OPTION!
```

Listing 10: Tentativo di modifica respinto

Quando una vista è definita a partire da un'altra vista preesistente, la clausola di controllo può essere configurata su due livelli di rigidità:

- `LOCAL CHECK OPTION`: convalida unicamente le condizioni `WHERE` definite sulla vista corrente, senza propagare il vincolo alle viste sottostanti (a meno che non abbiano esplicitamente il proprio `CHECK OPTION`).
- `CASCADED CHECK OPTION` (*comportamento predefinito*): propaga i controlli a ritroso lungo tutta la gerarchia, imponendo il rispetto simultaneo delle condizioni della vista corrente e di tutte le viste genitrici.

Riepilogo: Propagazione dei Vincoli nelle Viste Annidate

Scenario di Gerarchia	WITH LOCAL CHECK OPTION	WITH CASCADED CHECK OPTION
Vista B definita sopra Vista A	Controlla solo i filtri propri di B. I filtri di A vengono ignorati se A non ha specificato check option.	**Controlla entrambi i filtri**: la tupla deve soddisfare i requisiti di B e i requisiti di A.
Garanzia sui Dati	Flessibilità locale, rischio di effetti collaterali a livelli inferiori.	**Massima integrità logica**: nessuna tupla illegale può raggiungere la tabella di base.

5.2 Query Complesse, Ricorsione e Funzioni Scalari

5.2.1 Query Complesse e Scomposizione con Viste Intermedie

Il Limite dell'Annidamento degli Aggregati. In SQL standard non è consentito annidare direttamente funzioni aggregate all'interno della stessa clausola `SELECT`.

Si consideri la richiesta: *«Fornire il numero medio di uffici per dipartimento»*. L'approccio immediato e intuitivo:

```
-- ERRORE SINTATTICO: SQL vieta l'annidamento di funzioni aggregate
SELECT AVG(COUNT(DISTINCT Office))
FROM   Employee
GROUP BY Dept;
```

Listing 11: Query sintatticamente errata: annidamento di aggregati

Motivazione dell'errore: l'operatore `COUNT` lavora comprimendo le singole righe all'interno di ogni partizione creata dal `GROUP BY`, mentre `AVG` dovrebbe operare su un insieme di risultati già calcolati. Il motore SQL non supporta due livelli logici di riduzione scalare nello stesso blocco di selezione.

Risoluzione con Vista Intermedia a Due Livelli. Per calcolare aggregazioni su aggregazioni senza incorrere in errori di sintassi, si scompone l'elaborazione creando una **vista intermedia**:

```
-- Passo 1: la vista calcola il primo livello di aggregazione per reparto
CREATE VIEW DeptOffices (NameDept, OffNo) AS
```

```
    SELECT Dept, COUNT(DISTINCT Office)
    FROM   Employee
    GROUP BY Dept;

-- Passo 2: la query finale calcola la media sui valori gia aggregati
SELECT AVG(OffNo) AS MediaUfficiPerDipartimento
FROM   DeptOffices;
```

Listing 12: Query corretta con vista intermedia di aggregazione

Il Problema delle Clausole HAVING Annidate. Si consideri una seconda richiesta: *«Individuare il dipartimento con la spesa salariale complessiva più elevata»*. Un tentativo frequente consiste nell'annidare una sottoquery direttamente all'interno della clausola `HAVING`:

```
-- Formulazione problematica: clausola HAVING con sottoquery aggregata
SELECT Dept
FROM   Employee
GROUP BY Dept
HAVING SUM(Salary) >= ALL (
    SELECT SUM(Salary)
    FROM   Employee
    GROUP BY Dept
);
```

Listing 13: Query con HAVING annidato: non supportata da molti DBMS

Criticità: sebbene teoricamente concepita dallo standard, la presenza di una clausola `HAVING` contenente una sottoquery raggruppata e quantificata con `>= ALL` non è universalmente ammessa da molti motori relazionali o produce piani di esecuzione inefficienti.

Soluzione Robusta: Vista Intermedia di Budget. Si isola il totale salariale di ciascun dipartimento in una vista, riducendo l'estrazione a una semplice condizione scalare di massimo:

```
-- Passo 1: isola la somma degli stipendi per ogni dipartimento
CREATE VIEW BudgetSalary (Dep, TotalSalary) AS
    SELECT Dept, SUM(Salary)
    FROM   Employee
    GROUP BY Dept;

-- Passo 2: estrae il dipartimento confrontando il valore con il massimo
SELECT Dep
FROM   BudgetSalary
WHERE  TotalSalary = (
    SELECT MAX(TotalSalary)
    FROM   BudgetSalary
);
```

Listing 14: Soluzione standard e portabile con vista intermedia

Regola Metodologica per Query Aggregate Complesse

Ogniqualvolta una richiesta richieda calcoli statistici a gradini (es. massimo di somme, media di conteggi) o confronti globali su gruppi:

1. **Non forzare la query singola**: l'annidamento estremo genera codice illeggibile e spesso non conforme ai compilatori dei singoli DBMS.
2. **Modularizzare con le Viste**: definire una vista intermedia che calcoli le grandezze parziali e interrogare successivamente la vista per ottenere il risultato globale.

5.2.2 Query Ricorsive e Chiusura Transitiva (SQL:1999)

I Limiti dell'Algebra e la Chiusura Transitiva. Nelle basi di dati reali i record possono instaurare relazioni ad albero o a grafo di profondità arbitraria (gerarchie aziendali, alberi genealogici, percorsi stradali). Con i soli operatori di join convenzionali è possibile risalire a un numero prefissato di livelli (es. padre, nonno), ma **non è possibile calcolare la chiusura transitiva** su una gerarchia di profondità ignota.

Si consideri la relazione **Paternità** (*Fatherhood*):

Relazione Paternità (*Fatherhood*)

Father	Child
Carl	Frank
Louis	Olga
Louis	Bob
Frank	Alex
Frank	Alfred

Obiettivo: determinare **tutti gli antenati** di ciascuna persona (genitori, nonni, bisnonni, ecc.).

Fondamento Teorico: La Ricorsione in Datalog. Nella logica dei predicati e in Datalog, la chiusura transitiva si scompone formalmente in una regola base e una regola ricorsiva induttiva:

$$\text{Ancestors}(p, f) \leftarrow \text{Fatherhood}(p, f)$$
$$\text{Ancestors}(a, d) \leftarrow \text{Fatherhood}(a, f),\ \text{Ancestors}(f, d)$$

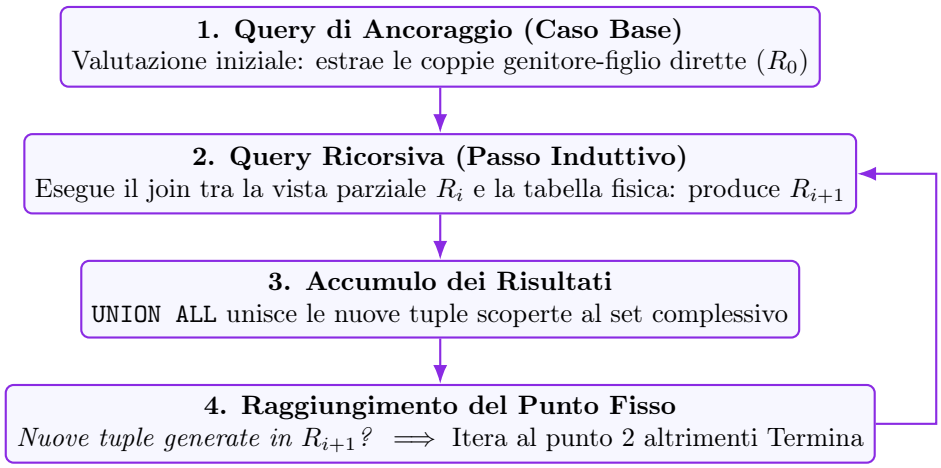
- **Passo Base**: ogni padre diretto è un antenato del proprio figlio.
- **Passo Ricorsivo**: se a è padre di f ed f è antenato di d, allora anche a è un antenato di d.

Implementazione in SQL:1999 con WITH RECURSIVE. Lo standard SQL:1999 ha introdotto le *Common Table Expressions* (CTE) ricorsive per implementare il calcolo a punto fisso direttamente nel linguaggio:

```
WITH RECURSIVE Ancestors (Ancestor, Descendant) AS (
    -- 1. Query di ancoraggio (Anchor Member - Caso Base)
    SELECT Father, Child
    FROM   Fatherhood
```

```
    UNION ALL
    -- 2. Query ricorsiva (Recursive Member - Passo Induttivo)
    SELECT a.Ancestor, f.Child
    FROM   Ancestors a
           JOIN Fatherhood f
             ON a.Descendant = f.Father
)
SELECT *
FROM   Ancestors;
```

Listing 15: Calcolo completo degli antenati tramite CTE ricorsiva



Esempio Aziendale: Catena di Supervisione Gerarchica. Dato lo schema aziendale Employee(No, Surna vogliamo individuare la totalità dei supervisori (diretti e indiretti) del dipendente "John Doe":

```
WITH RECURSIVE InCharge (No, Supervisor) AS (
    -- Caso base: supervisori diretti di tutti i dipendenti
    SELECT No, Supervisor
    FROM   Employee
    UNION
    -- Passo ricorsivo: risale la catena di comando
    SELECT e.No, ic.Supervisor
    FROM   Employee e
           JOIN InCharge ic
             ON e.Supervisor = ic.No
)
SELECT e.Name, e.Surname, ic.Supervisor
FROM   Employee e
       JOIN InCharge ic
         ON e.No = ic.No
WHERE  e.Name = 'John'
  AND  e.Surname = 'Doe';
```

Listing 16: Catena dei supervisori di John Doe

5.2.3 Funzioni Scalari Predefinite e Manipolazione Dati

Natura delle Funzioni Scalari. A differenza delle funzioni aggregate (che operano verticalmente comprimendo più righe in un unico valore aggregato), le **funzioni scalari** operano orizzontalmente sul valore del singolo attributo di ogni riga, restituendo esattamente un valore per ciascuna tupla esaminata.

Caratteristica	Funzioni Scalari	Funzioni Aggregate
Ambito di Azione	Valutate tupla per tupla (orizzontale).	Valutate su insiemi di tuple (verticale).
Righe di Output	Restituiscono un valore per ogni tupla esaminata.	Comprimono il gruppo/tabella in un solo valore riassuntivo.
Posizionamento	In qualsiasi clausola (`SELECT`, `WHERE`, ecc.).	Solo nella clausola `SELECT` o `HAVING`.
Esempi Tipici	`EXTRACT`, `LOWER`, `CAST`.	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

1. Funzioni Temporali e Manipolazione Date. Lo standard definisce funzioni di sistema per interagire con date e timestamp:

- `CURRENT_DATE`: restituisce la data corrente del sistema.
- `EXTRACT(campo FROM espressione)`: isola una singola porzione numerica da una data (es. `YEAR`, `MONTH`, `DAY`, `HOUR`).

```
-- Estrae l'anno degli ordini emessi nella giornata odierna:
SELECT EXTRACT(YEAR FROM OrderDate) AS AnnoOrdine
FROM   Orders
WHERE  DATE(OrderDate) = CURRENT_DATE;
```

Listing 17: Uso di EXTRACT e CURRENT_DATE

2. Funzioni di Manipolazione Testuale. Consentono trasformazioni sulle stringhe di caratteri:

- `CHAR_LENGTH(stringa)`: restituisce il numero intero di caratteri che compongono il testo.
- `LOWER(stringa)` / `UPPER(stringa)`: converte l'intero testo in caratteri minuscoli o maiuscoli, fondamentale per eseguire confronti uniformi e indipendenti dalle maiuscole (*case-insensitive*).

3. Conversione di Tipo Esplicita: La Funzione CAST. Nei database relazionali può essere necessario convertire dinamicamente un attributo da un dominio a un altro:

```
CAST ( Espressione AS NuovoTipoDato )
```

```
-- Normalizzazione di stringhe e calcolo della lunghezza:
SELECT Matricola,
       LOWER(Cognome)       AS CognomeMinuscolo,
       CHAR_LENGTH(Cognome) AS NumeroLettere
FROM   Employee;

```

```sql
-- Forzare la divisione reale tra due interi:
SELECT Matricola,
       CAST(Stipendio AS DECIMAL(10,2)) / 12 AS StipendioMensile
FROM   Employee;
```

Listing 18: Esempi di utilizzo di CAST e funzioni su stringhe

5.3 Espressioni Condizionali e Sicurezza dei Dati

5.3.1 Espressioni Condizionali: COALESCE, NULLIF e CASE

Logica Condizionale in SQL. Nello standard SQL-92 sono state introdotte funzioni scalari per gestire la presenza di valori nulli e implementare logiche decisionali di tipo *if-then-else* direttamente all'interno delle interrogazioni, senza dover demandare tali controlli al software applicativo esterno.

1. La Funzione COALESCE. Riceve in ingresso una sequenza di espressioni e restituisce il **primo valore non nullo** incontrato nell'elenco. Se tutti i parametri valgono `NULL`, la funzione restituisce `NULL`:

```sql
COALESCE(valore_1, valore_2, ..., valore_n)
```

La funzione trova due utilizzi tipici nella pratica:

- **Selezione di recapiti a cascata**: recuperare il primo contatto disponibile da colonne alternative:

```sql
-- Restituisce il cellulare se presente; altrimenti il fisso di casa:
SELECT Matricola,
       COALESCE(Cellulare, TelefonoCasa) AS RecapitoPrincipale
FROM   Impiegato;
```

- **Sostituzione di valori di default**: rimpiazzare l'assenza di un valore (`NULL`) con una stringa o un valore sentinella leggibile:

```sql
-- Se il dipartimento e' NULL, restituisce la stringa fissa 'Nessuno
   ':
SELECT Cognome, Nome,
       COALESCE(Dipart, 'Nessuno') AS DipartimentoAssegnato
FROM   Impiegato;
```

2. La Funzione NULLIF. Confronta i due argomenti passati in input: se sono **uguali**, restituisce `NULL`; se sono **diversi**, restituisce il valore del primo argomento:

```sql
NULLIF(espressione_1, espressione_2)
```

Viene impiegata principalmente per ripulire i dati da valori sentinella preesistenti o per evitare errori di divisione per zero:

```sql
-- Trasforma la dicitura 'Sconosciuto' nel valore nativo NULL:
SELECT Cognome,
       NULLIF(Dipart, 'Sconosciuto') AS DipartimentoPulito
FROM   Impiegato;

```

```sql
-- Prevenzione della divisione per zero (restituisce NULL se Divisore =
   0):
SELECT Dividendo / NULLIF(Divisore, 0) AS QuozienteSicuro
FROM   DatiContabili;
```

Listing 19: Uso di NULLIF per normalizzare dati non validi

3. Il Costrutto CASE. Fornisce una struttura di diramazione condizionale completa a livello di riga. Esiste in due varianti sintattiche:

- **CASE Semplice**: confronta un'espressione iniziale con una serie di valori costanti:

```sql
CASE espressione
    WHEN valore_1 THEN risultato_1
    WHEN valore_2 THEN risultato_2
    [ ELSE risultato_default ]
END
```

- **CASE Ricercato (*Searched*)**: valuta predicati booleani eterogenei in sequenza:

```sql
CASE
    WHEN condizione_1 THEN risultato_1
    WHEN condizione_2 THEN risultato_2
    [ ELSE risultato_default ]
END
```

```sql
-- Calcolo progressivo del bollo auto in base a tipo e potenza:
SELECT Targa,
       (CASE Tipo
            WHEN 'Auto' THEN 2.58 * PotenzaKW
            WHEN 'Moto' THEN 22.00 + (1.00 * PotenzaKW)
            ELSE NULL
        END) AS ImportoBollo
FROM   Veicolo
WHERE  AnnoImmatricolazione > 1975;
```

Listing 20: Calcolo dell'imposta sui veicoli con CASE

Sintesi: Funzioni Condizionali in SQL

Funzione	Logica Operativa	Tipico Caso d'Uso
COALESCE	Primo argomento $\neq$ NULL.	Fallback su contatti secondari o etichette di default.
NULLIF	Se $A = B \implies$ NULL; altrimenti A.	Bonifica di dati sentinella ed eliminazione divisioni per zero.
CASE	Struttura *if-then-else* ramificata.	Classificazione fasce di reddito, tariffe e reportistica dinamica.

5.3.2 Sicurezza, Privilegi e Ruoli

Requisiti di Riservatezza. La condivisione centralizzata dei dati comporta la necessità di proteggere le informazioni sensibili da accessi non autorizzati. Il sottosistema di sicurezza di SQL consente di definire quali utenti abbiano il diritto di accedere a specifiche porzioni della base di dati e quali operazioni possano eseguire.

Granularità e Titolarità delle Risorse.

- **Oggetti Proteggibili**: relazioni, viste, singoli attributi di tabelle e domini personalizzati.
- **Superutente Amministratore**: in ogni DBMS esiste un account amministrativo predefinito (es. `_system`, `postgres`, `root`) dotato a priori di tutti i privilegi globali sul server.
- **Proprietario della Risorsa (*Owner*)**: l'utente che crea un oggetto ne diviene automaticamente proprietario, acquisendo su di esso tutti i diritti d'uso e la facoltà di concederli ad altri.

I Cinque Elementi Costitutivi di un Privilegio

Nel modello di sicurezza relazionale, ogni autorizzazione è descritta formalmente da cinque coordinate:

1. **Risorsa**: l'oggetto logico su cui opera il permesso (tabella, vista, colonna).
2. **Soggetto Erogante (*Grantor*)**: l'utente che conferisce l'autorizzazione.
3. **Soggetto Ricevente (*Grantee*)**: l'utente o ruolo che beneficia del permesso.
4. **Azione Consentita**: l'operazione specifica ammessa sul dato.
5. **Diritto di Propagazione**: la facoltà per il ricevente di estendere il privilegio ricevuto ad altri utenti (`WITH GRANT OPTION`).

I Privilegi Nativi di SQL. Lo standard definisce una serie di operazioni elementari concedibili:

Privilegio	Operazione Consentita sulla Risorsa
SELECT	Lettura ed estrazione delle tuple mediante interrogazioni.
INSERT	Aggiunta di nuove righe all'interno della tabella o vista.
UPDATE	Modifica del contenuto delle tuple (può essere ristretto a singole colonne).
DELETE	Rimozione fisica delle righe dalla tabella.
REFERENCES	Definizione di vincoli di chiave esterna rivolti verso le colonne della tabella.
USAGE	Utilizzo di una definizione logica condivisa (es. un dominio o una sequenza).

Assegnazione dei Privilegi: Il Comando GRANT. Consente al proprietario di una risorsa (o all'amministratore) di trasferire permessi a uno o più soggetti:

```
GRANT < ListaPrivilegi | ALL PRIVILEGES >
ON    Risorsa
TO    < ListaUtenti | PUBLIC >
[ WITH GRANT OPTION ];
```

```
-- Concessione del permesso di sola lettura su una tabella:
GRANT SELECT ON Dipartimento TO Jack;

-- Concessione di permessi multipli con facolta di propagazione:
GRANT SELECT, INSERT ON Impiegato TO Mario
WITH GRANT OPTION;

-- Concessione di modifica limitata a singole colonne:
GRANT UPDATE (Stipendio, Dipart) ON Impiegato TO ResponsabileHR;

-- Concessione pubblica a qualsiasi utente registrato nel sistema:
GRANT SELECT ON CatalogoProdotti TO PUBLIC;
```

Listing 21: Esempi di comando GRANT

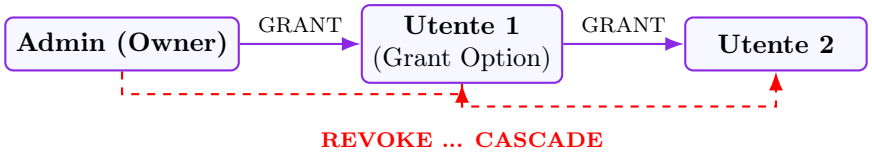
Revoca dei Privilegi: Il Comando REVOKE. Per rimuovere autorizzazioni precedentemente assegnate si usa l'istruzione `REVOKE`:

```
REVOKE ListaPrivilegi
ON     Risorsa
FROM   ListaUtenti
[ RESTRICT | CASCADE ];
```

Regola Fondamentale: i diritti possono essere revocati **esclusivamente dallo stesso utente che li ha concessi in precedenza**.

Gestione degli Effetti Collaterali: RESTRICT vs CASCADE.

- **RESTRICT** (*comportamento predefinito*): la revoca fallisce se l'utente ricevente ha a sua volta propagato il permesso ad altre persone tramite `WITH GRANT OPTION`.
- **CASCADE**: innesca una **reazione a catena**: il permesso viene tolto all'utente specificato e viene contestualmente rimosso a tutti gli altri utenti a cui tale permesso era stato concesso indirettamente lungo l'albero di propagazione.



Difesa da Attacchi per Inferenza ed Enumerazione. Un principio cardine della sicurezza nei DBMS impone che il sistema non debba rivelare alcuna informazione sull'esistenza o la consistenza delle risorse non accessibili:

- Se un utente prova a interrogare una tabella inesistente: riceve un messaggio di errore.
- Se un utente prova a interrogare una tabella esistente su cui **non possiede il privilegio di lettura**: il sistema deve restituire **lo stesso identico messaggio di errore**.

Fornire messaggi differenziati (es. "*Accesso negato alla tabella Impiegato*") confermerebbe all'attaccante che la tabella esiste, favorendo l'esplorazione malevola della struttura della base di dati.

Le Viste come Meccanismo di Controllo degli Accessi. Quando un utente non deve avere visibilità su tutte le righe o su tutte le colonne di una relazione, la tecnica architetturale standard prevede l'uso combinato di **Viste** e **Privilegi**:




```sql
-- 1. Creazione della vista con filtraggio orizzontale e verticale:
CREATE VIEW ImpiegatiRepartoVendite AS
    SELECT Matricola, Cognome, Nome
    FROM   Impiegato
    WHERE  Dipart = 'Vendite';

-- 2. Concessione dell'accesso solo alla vista (nessun diritto su
   Impiegato):
GRANT SELECT ON ImpiegatiRepartoVendite TO ResponsabileVendite;
```

Listing 22: Configurazione di sicurezza tramite vista

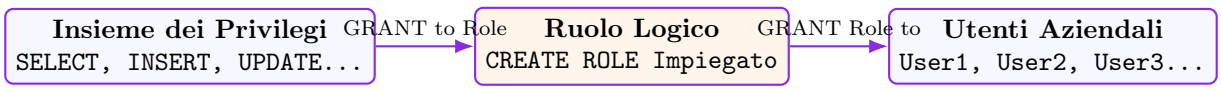
L'utente `ResponsabileVendite` visualizzerà unicamente il personale del proprio settore, senza poter accedere ai record degli altri reparti né visualizzare l'importo degli stipendi.

Limiti dell'Assegnazione Diretta dei Privilegi. Nei sistemi aziendali complessi con centinaia di operatori, assegnare i singoli permessi utente per utente risulta ingestibile e causa gravi vulnerabilità:

- Ogni inserimento di nuovo personale richiede decine di istruzioni `GRANT`.
- Se un dipendente cambia mansione, occorre rintracciare e revocare manualmente tutti i singoli permessi pregressi.

Il Modello RBAC (Role-Based Access Control). Introdotto formalmente in SQL:1999 (SQL-3), il modello RBAC introduce il concetto di **Ruolo**:

- Un **Ruolo** agisce da contenitore logico di privilegi associati a una determinata funzione lavorativa aziendale (es. *Amministrativo*, *Docente*, *Cassiere*).
- I privilegi sulle risorse vengono assegnati direttamente ai ruoli.
- Agli utenti non vengono concessi i singoli permessi sulle tabelle, ma viene assegnato uno o più ruoli.
- In ogni istante, un utente dispone sia dei propri eventuali privilegi individuali sia di tutti i privilegi ereditati tramite i ruoli a lui assegnati.



Sintassi e Comandi Operativi per la Gestione dei Ruoli.

```
-- 1. Creazione di un nuovo ruolo:
CREATE ROLE RuoloImpiegato;

-- 2. Concessione di privilegi specifici al ruolo:
GRANT CREATE TABLE TO RuoloImpiegato;
GRANT SELECT, INSERT ON Dipartimento TO RuoloImpiegato;

-- 3. Assegnazione del ruolo a un utente registrato:
GRANT RuoloImpiegato TO Mario;

-- 4. Attivazione del ruolo nella sessione utente corrente:
SET ROLE RuoloImpiegato;

-- 5. Revoca di un permesso dal ruolo (si propaga a tutti i membri del
   ruolo):
REVOKE CREATE TABLE FROM RuoloImpiegato;
```

Listing 23: Flusso operativo completo con RBAC

Vantaggi Architetturali del Modello RBAC

- **Manutenibilità Semplificata**: per aggiungere o rimuovere un permesso da un'intera categoria di dipendenti è sufficiente un'unica operazione di `GRANT` o `REVOKE` sul ruolo.
- **Gestione del Personale Immediata**: quando arriva un nuovo dipendente, gli si assegna il ruolo di competenza con un solo comando; se il dipendente lascia l'azienda, è sufficiente revocare il ruolo.
- **Principio del Minimo Privilegio**: gli utenti acquisiscono unicamente l'insieme di privilegi strettamente necessario a svolgere la propria mansione attiva.

5.4 Gestione delle Transazioni

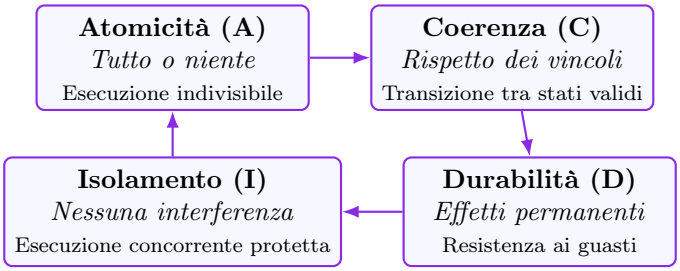
5.4.1 Transazioni e Proprietà ACID

Natura della Transazione. Una **transazione** è un programma in esecuzione che costituisce un'unità logica elementare di elaborazione all'interno della base di dati. Essa rappresenta un'operazione atomica: raggruppa una o più istruzioni di manipolazione dati che devono essere gestite dal DBMS come un unico blocco indivisibile.

I Quattro Requisiti Fondamentali (ACID). Per garantire l'affidabilità e la correttezza dei dati anche in presenza di accessi concorrenti o di guasti improvvisi del sistema, ogni transazione deve soddisfare quattro proprietà essenziali:

- **Atomicità (*Atomicity*)**: l'unità di lavoro deve essere eseguita per intero oppure per nulla.
- **Preservazione della Coerenza (*Consistency*)**: l'esecuzione completa della transazione deve trasferire la base di dati da uno stato coerente a un altro stato coerente.

- **Isolamento (*Isolation*)**: l'esecuzione di transazioni concorrenti non deve provocare interferenze reciproche sui dati.
- **Persistenza o Durabilità (*Durability*)**: i risultati prodotti da una transazione completata con successo devono persistere stabilmente nel tempo.



1. Atomicità (*All or Nothing*). La transazione costituisce un'unità di elaborazione non frazionabile: deve essere eseguita completamente oppure non deve essere applicata affatto.

- *Caso d'uso classico*: il bonifico bancario tra due conti correnti A e B.
- Il denaro deve essere prelevato da A e contestualmente accreditato su B.
- Se il sistema subisce un blocco dopo l'addebito ma prima dell'accredito, nessuna delle due operazioni deve rimanere registrata.

2. Preservazione della Coerenza (*Consistency*). Se la transazione viene eseguita dall'inizio alla fine senza interferenze, la base di dati passa da uno stato consistente a un nuovo stato consistente.

- Durante le fasi intermedie di elaborazione possono verificarsi violazioni temporanee di vincoli o bilanci contabili.
- Tali inconsistenze transitorie non possono persistere al momento della chiusura della transazione.
- Se al termine dell'elaborazione i vincoli d'integrità risultano ancora violati, la transazione viene abortita (*abort*) e l'intero stato viene ripristinato.

3. Isolamento (*Isolation*). Anche in presenza di un numero elevato di transazioni eseguite simultaneamente, ciascuna transazione deve operare come se fosse l'unica attiva all'interno del sistema.

- L'esecuzione concorrente non deve causare interferenze o anomalie di lettura/scrittura.
- *Esempio*: due operazioni concorrenti di prelievo che insistono nello stesso istante sul medesimo conto corrente devono essere regolate in modo che i saldi intermedi non si sovrascrivano a vicenda.

4. Durabilità degli Effetti (*Durability / Permanency*). Le modifiche apportate alla base di dati da una transazione confermata con successo (*committed*) devono essere conservate in modo permanente.

- I dati salvati non possono andare perduti, nemmeno a fronte di guasti critici a livello hardware o crash del sistema operativo.
- La persistenza viene garantita dal DBMS mediante la scrittura preventiva delle operazioni su supporti non volatili (meccanismo del *Write-Ahead Logging*).

Quadro Sinottico: Le Quattro Proprietà ACID

Proprietà	Significato Sistemistico	Comportamento in Caso di Errore
Atomicità	Unità di calcolo atomica e indivisibile.	Annullamento completo delle operazioni parziali (*Rollback*).
Coerenza	Preservazione di tutti i vincoli di schema.	Annullamento della transazione se permangono violazioni (*Abort*).
Isolamento	Indipendenza da transazioni concorrenti.	Controllo della concorrenza tramite locking o versioning dei record.
Durabilità	Effetti definitivi e non volatili su disco.	Ripristino automatico tramite registri di log post-riavvio.

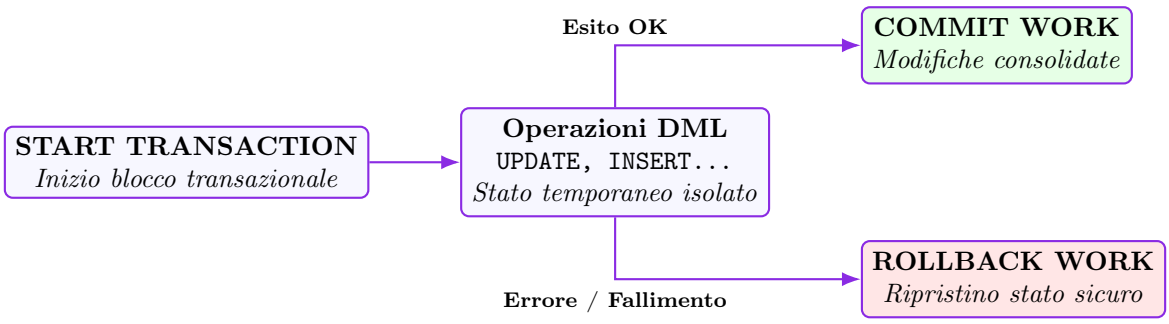
5.4.2 Comandi Transazionali ed Esempio Completo

Ciclo di Vita di una Transazione SQL. Una volta stabilita la sessione con la base di dati, il controllo dell'unità di lavoro fa uso di parole chiave specifiche:

- `START TRANSACTION`: avvia formalmente una nuova transazione logica (comando opzionale in gran parte degli RDBMS commerciali, dove la prima istruzione DML apre implicitamente il blocco).
- `COMMIT [WORK]`: conferma con successo le operazioni eseguite, rendendo le modifiche visibili agli altri utenti e salvandole stabilmente nella base di dati.
- `ROLLBACK [WORK]`: rifiuta tutte le modifiche apportate dall'inizio della transazione, scartando i dati temporanei e riportando il database al precedente stato consistente.

La Modalità Autocommit. Per impostazione predefinita, molti sistemi RDBMS (come MySQL) operano in modalità **autocommit**:

- Ogni singola istruzione SQL viene trattata come una transazione a sé stante.
- Al termine dell'esecuzione del comando, il DBMS esegue automaticamente un `COMMIT` immediato.
- Per coordinare sequenze di aggiornamenti correlate occorre disattivare l'autocommit oppure aprire esplicitamente il blocco con `START TRANSACTION`.



Si consideri un'operazione contabile che prevede il trasferimento di 10 euro dal conto corrente identificato dal numero 42177 verso il conto 12202.

L'operazione richiede due aggiornamenti congiunti che non possono essere separati:

```
-- Apertura esplicita della transazione (opzionale in alcuni DBMS):
START TRANSACTION;

-- 1. Prelievo di 10 euro dal conto sorgente:
UPDATE ContoBancario
SET    Saldo = Saldo - 10
WHERE  NumeroConto = 42177;

-- 2. Accredito di 10 euro sul conto di destinazione:
UPDATE ContoBancario
SET    Saldo = Saldo + 10
WHERE  NumeroConto = 12202;

-- Consolidamento definitivo delle modifiche nel database:
COMMIT WORK;
```

Listing 24: Esempio di transazione bancaria completa

Analisi delle Garanzie Operative:

- Se si verifica un calo di tensione o un'interruzione di rete dopo il primo comando di `UPDATE`, l'accredito non ha luogo; il DBMS interviene ed esegue automaticamente il **rollback**, ripristinando il saldo originario del conto 42177.
- Solo nel momento in cui il comando `COMMIT WORK` viene registrato, entrambe le variazioni di saldo diventano permanenti e visibili alle altre connessioni concorrenti.

5.5 Concorrenza, affidabilità e indici

5.5.1 Controllo di Concorrenza: Anomalie, Schedule e Serializzabilità

Il DBMS esegue molte transazioni in parallelo, alternandone le operazioni, per sfruttare le risorse. Senza controllo, l'esecuzione concorrente produce **anomalie**. Notazione: $r_i(x)$ e $w_i(x)$ sono lettura e scrittura di x da parte di T_i; c_i e a_i sono commit e abort.

Anomalia	Schedule tipico	Cosa succede
Perdita di aggiornamento	$r_1(x)\, r_2(x)\, w_1(x)\, w_2(x)$	entrambe leggono x, la scrittura di T_2 cancella quella di T_1 (conflitto WW)
Lettura sporca	$w_1(x)\, r_2(x)\, a_1$	T_2 legge un valore di T_1 che poi fa abort: un valore mai esistito (WR)
Letture inconsistenti	$r_1(x)\, w_2(x)\, c_2\, r_1(x)$	T_1 legge due volte x e ottiene valori diversi (RW)
Aggiornamento fantasma	—	T_1 legge dati che violano un vincolo perché T_2 ne ha aggiornati solo una parte (es. somma $x+y+z$ letta a metà di un trasferimento)
Inserimento fantasma	—	T_1 calcola un aggregato (es. media degli stipendi), T_2 *inserisce* una tupla che soddisfa la condizione, e una seconda lettura di T_1 dà un altro risultato

Uno **schedule** è una sequenza di operazioni di più transazioni che rispetta l'ordine interno di ciascuna. È **seriale** se le transazioni sono eseguite una dopo l'altra, senza alternanza: uno schedule seriale non ha anomalie. Lo scheduler deve accettare solo schedule **serializzabili**, cioè equivalenti a uno seriale.

- Due operazioni sono in **conflitto** se appartengono a transazioni diverse, operano sullo stesso oggetto e almeno una è una scrittura (coppie RW, WR, WW).
- Uno schedule è **conflict-serializzabile** (CSR) se è conflict-equivalente (stesse coppie in conflitto, nello stesso ordine) a uno seriale.
- **Grafo dei conflitti**: un nodo per transazione e un arco $T_i \to T_j$ se un'operazione di T_i precede un'operazione di T_j in conflitto con essa. Lo schedule è CSR **se e solo se il grafo è aciclico**.

Esempio: schedule non serializzabile

In $w_1(A)\, w_2(A)\, w_2(B)\, w_1(B)$ ci sono l'arco $T_1 \to T_2$ (su A) e l'arco $T_2 \to T_1$ (su B): il grafo ha un ciclo, quindi lo schedule *non* è serializzabile.

5.5.2 Lock, Locking a Due Fasi e Livelli di Isolamento

Nei DBMS reali la serializzabilità si ottiene con i **lock**: prima di leggere una risorsa si chiede un **lock condiviso** (S), prima di scriverla un **lock esclusivo** (X). Più lock S possono coesistere, mentre X è incompatibile con tutto; se il lock non è concesso la transazione attende.

- **2PL (locking a due fasi)**: dopo aver rilasciato un lock una transazione non può acquisirne altri (fase crescente, poi calante). Garantisce la serializzabilità.
- **2PL stretto**: i lock si rilasciano solo dopo commit o abort. Evita anche le letture sporche e gli abort a cascata; è la variante usata in pratica.

Lettura sporca risolta con 2PL stretto: $X_1(x)\, w_1(x)\, S_2(x)\, [\text{attesa}]\, a_1\, [\text{rilascio}]\, r_2(x)\, c_2$: T_2 legge x solo dopo la fine di T_1.

Deadlock Con i lock due transazioni possono attendersi a vicenda. Si risolve con *timeout*, con la rilevazione dei cicli nel *grafo delle attese* o con la prevenzione; una transazione viene abortita e rieseguita.

Livelli di isolamento in SQL SQL permette di rinunciare a parte dell'isolamento per guadagnare prestazioni (`SET TRANSACTION ISOLATION LEVEL ...`). In tutti i livelli i lock in scrittura sono tenuti fino al commit; cambia la gestione delle letture:

Livello	Letture	Anomalie ancora possibili
`READ UNCOMMITTED`	nessun lock in lettura	lettura sporca, letture inconsistenti, fantasmi
`READ COMMITTED`	lock in lettura brevi	letture inconsistenti, aggiornamento e inserimento fantasma
`REPEATABLE READ`	lock in lettura lunghi (sulle tuple)	inserimento fantasma
`SERIALIZABLE`	lock lunghi + lock di predicato	nessuna

Per scegliere il livello minimo: lettura sporca $\Rightarrow$ `READ COMMITTED`; letture non ripetibili o perdita di aggiornamento ($r_1 r_2 w_1 w_2$) $\Rightarrow$ `REPEATABLE READ`; inserimento fantasma $\Rightarrow$ `SERIALIZABLE`.

5.5.3 Affidabilità: Log, Ripresa dai Guasti e ARIES

Il gestore dell'affidabilità garantisce atomicità e durabilità tramite il **log**, un file sequenziale su memoria stabile:

- record di transazione: $B(T)$ begin, $C(T)$ commit, $A(T)$ abort; $U(T, O, BS, AS)$ update con *before state* e *after state*; $I(T, O, AS)$ insert; $D(T, O, BS)$ delete;
- record di sistema: **checkpoint** $CK(T_1, \ldots, T_n)$ con le transazioni attive in quel momento, e **dump**, copia completa del DB.

Regole fondamentali del log

- **WAL (write-ahead log)**: la parte *before state* va scritta su disco *prima* del dato corrispondente, così si può sempre fare UNDO.
- **Commit-precedenza**: la parte *after state* va scritta *prima* del commit, così si può sempre fare REDO.

Il commit avviene quando $C(T)$ è scritto in modo sincrono nel log: da lì la transazione è definitiva.

- **Ripresa a caldo** (guasto di sistema, memoria centrale persa): si risale il log fino all'ultimo checkpoint, si costruiscono l'insieme UNDO (transazioni attive senza commit) e l'insieme REDO (transazioni con commit), si disfano le prime all'indietro e si rifanno le seconde in avanti.
- **Ripresa a freddo** (guasto di dispositivo): si ripristina l'ultimo dump, si rifanno in avanti le azioni del log fino al guasto, poi si esegue una ripresa a caldo.

ARIES È l'algoritmo di recovery più diffuso. Ogni record di log ha un *LSN* e ogni pagina conserva il *pageLSN* dell'ultimo aggiornamento che contiene. Al riavvio esegue tre fasi:

1. **Analisi**: dal checkpoint determina le transazioni attive al crash (le *loser*) e le pagine forse non scritte su disco.
2. **REDO** ("ripetere la storia"): riesegue in avanti le operazioni del log il cui effetto non è già sulla pagina (LSN > pageLSN), comprese quelle delle loser.
3. **UNDO**: annulla all'indietro le operazioni delle loser, scrivendo nel log un **CLR** (*compensation log record*) per ogni operazione annullata, così un nuovo crash non le annulla due volte.

Esempio ARIES

$A = 10$; una transazione senza commit esegue 4 volte $A := A + 2$. Il log va su disco dopo l'operazione 1 e dopo la 3, A va su disco dopo la 1, poi crash dopo la 4.

- Log su disco: **1, 2, 3** (la 4 era solo in memoria); A su disco vale **12**.
- REDO: si rieseguono **2, 3** (la 1 è già su disco): $A = 16$.
- UNDO (nessun commit): **3', 2', 1'**. Log finale: 1 2 3 3' 2' 1'; A torna a **10**.

5.5.4 Strutture Fisiche di Accesso e Indici

Gli **indici** migliorano i tempi di risposta delle interrogazioni e sono definiti a livello *fisico*, non logico: `CREATE INDEX idx_Surname ON Officer (Surname)`.
I dati stanno in memoria secondaria, organizzata in **blocchi**: il costo di un'operazione si misura in *numero di blocchi letti o scritti*. Un file di B blocchi può essere:

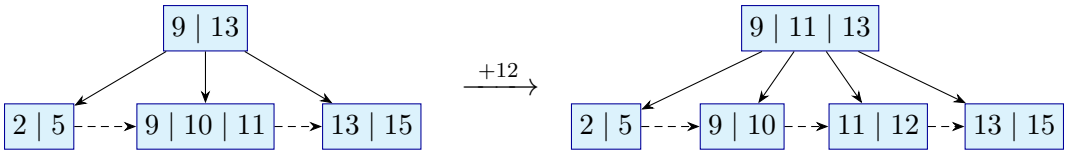
- **Heap (seriale)**: record in fondo, senza ordine; inserimento economico, ricerca per scansione.
- **Sequenziale ordinato**: record ordinati su un campo; ricerca binaria e scansioni ordinate, ma inserimenti costosi (spazio libero o overflow).
- **Hash**: la posizione si calcola con una funzione hash sulla chiave; accesso diretto per uguaglianza.

Classificazione degli indici Un indice associa ai valori della *chiave di ricerca* i puntatori ai record:

- **Primario / secondario**: primario se definito sul campo che ordina il file; secondario altrimenti (sempre denso).
- **Denso / sparso**: denso con un'entrata per ogni valore della chiave; sparso con meno entrate (tipicamente una per blocco), possibile solo su file ordinato.
- **Clustered / unclustered**: clustered se l'ordine fisico dei record segue quello dell'indice; efficiente per gli intervalli.
- **Monolivello / multilivello**: un indice sull'indice produce una struttura ad albero.

Alberi B e B+ Gli indici dinamici dei DBMS sono alberi bilanciati con *fan-out* elevato, quindi bassi (3–4 livelli bastano per milioni di record). Ogni nodo ha fino a $F-1$ chiavi ordinate e F puntatori, ed è pieno almeno a metà (radice esclusa); tutte le foglie sono allo stesso livello.

- **B-tree**: i nodi intermedi puntano anche ai dati.
- **B+tree**: *tutte* le chiavi stanno nelle foglie, collegate in lista; i nodi intermedi hanno solo chiavi separatrici. La lista rende efficienti le ricerche su intervalli.
- **Ricerca**: dalla radice si sceglie il puntatore tra le due chiavi che racchiudono il valore ($k_i \le v < k_{i+1}$) fino alla foglia; costo $\approx \log_F N$ accessi.
- **Inserimento**: se la foglia è piena si fa uno **split**; la prima chiave della nuova foglia è *copiata* nel padre (che può dividersi a sua volta; lo split di un nodo intermedio *sposta* la chiave mediana in alto). Se si divide la radice l'albero cresce di un livello.
- **Cancellazione**: se la foglia va in **underflow** (meno di metà piena) si fa una **ridistribuzione** con un fratello che ha chiavi in eccesso (aggiornando la separatrice nel padre), altrimenti un **merge** con il fratello, eliminando la separatrice dal padre.



Inserimento di 12 (nodi da 3 chiavi): la foglia $[9, 10, 11]$ si divide in $[9, 10]$ e $[11, 12]$ e 11 sale nella radice.

Strutture hash Una funzione $h(k)$ dà il numero di un *bucket*. Ottime per ricerche per *uguaglianza* (circa un accesso), inutili per gli intervalli. Con l'**hashing statico** il numero di bucket è fisso e i record in più vanno in catene di overflow; con l'**hashing dinamico/lineare** i bucket crescono col file, dividendosi a turno tramite un puntatore di split.

Costo delle operazioni Blocchi letti in media, file di B blocchi:

Operazione	**Organizzazione**	**Costo**
Uguaglianza sulla chiave	heap	$0.5\,B$
Ricerca su campo non chiave	heap	B (scansione completa)
Uguaglianza sul campo di ordinamento	file ordinato	$\log_2 B$
Uguaglianza su un altro campo	file ordinato	$0.5\,B$, come un heap
Uguaglianza	hash (senza overflow)	≈ 1
Uguaglianza o intervallo	B+tree	altezza + blocchi letti